



算法设计与分析

Lecture 9: Network Flow and Matching

曹刘娟

厦门大学信息学院计算机系

caoliujuan@xmu.edu.cn



Agenda

- 9.1 最大流问题
 - Ford Fulkerson方法
 - 最短路径增广算法
- 9.2 最小费用流问题
 - 消除回路算法
- 9.3 匹配问题
 - 二分图的匹配



MAX-FLOW PROBLEM

最大流问题



Network 网络

Examples of a network

- Liquids flowing through pipes 水流
- Parts through assembly lines 装配线
- Current through electrical network 电路
- Information through communication network 通信网络
- Signal through neural network in our brain 大脑神经信号
- Goods transported on the road... 货品传输

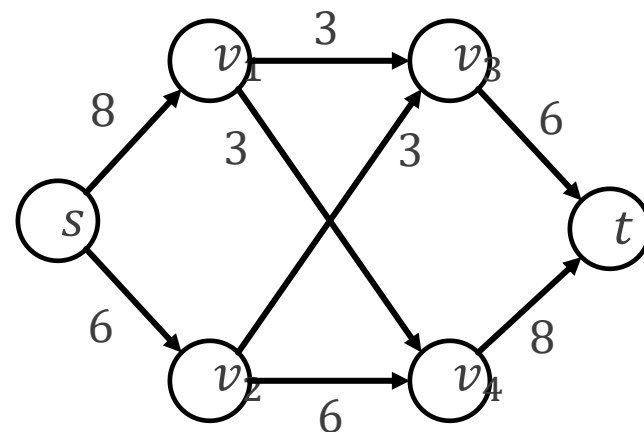


Definition 9.1

A **network** $G = (V, E)$ is a **directed graph** in which each edge $(u, v) \in E$ has a **nonnegative capacity** (容量) $c(u, v) \geq 0$. If $(u, v) \notin E$, we assume that $c(u, v) = 0$.

We distinguish two vertices in a flow network: a **source** (源点) s and a **sink** (汇点) t , where s only has **outedges** (出边) and t only has **inedges** (入边).

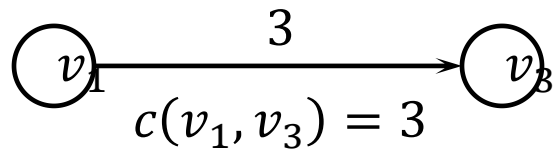
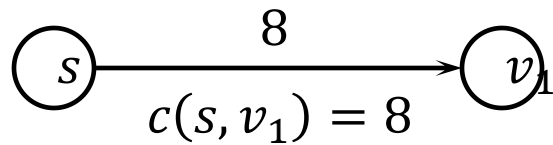
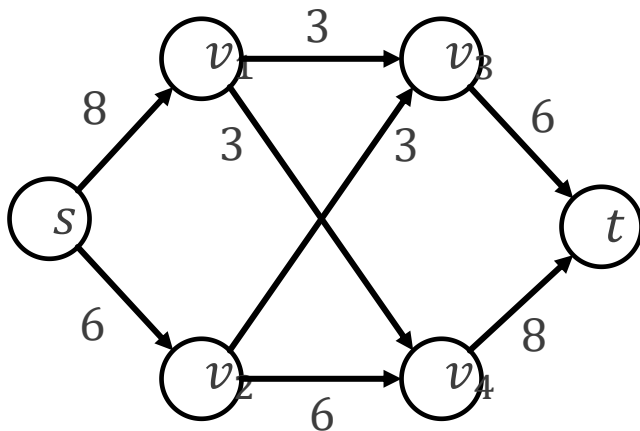
Every vertex lies on some path from the source to the sink.



$$c(v_1, v_3) = 3$$

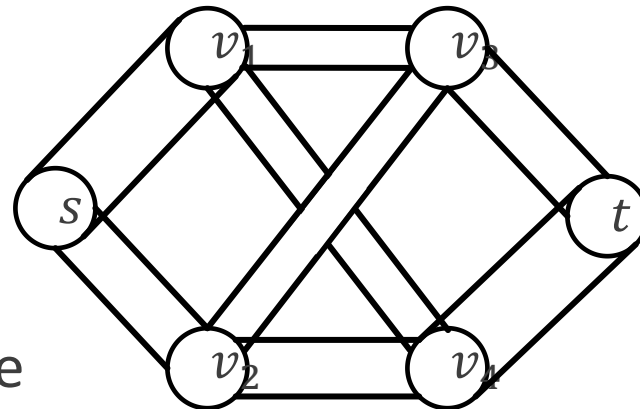
$$c(v_1, v_2) = 0$$

Flow network

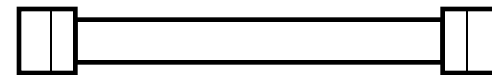


source

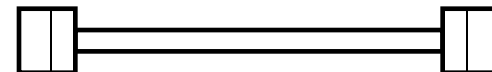
Example: Oil Pipeline



sink

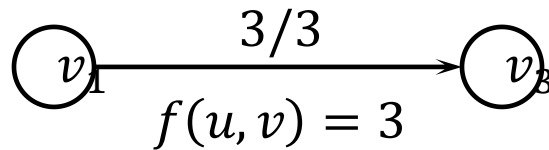
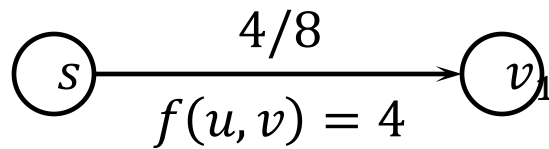
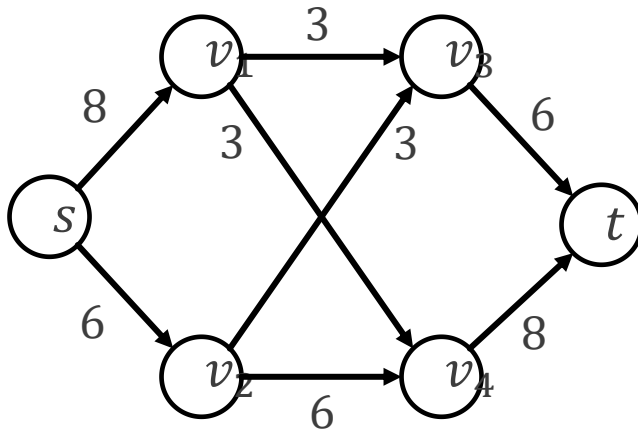


Big pipe



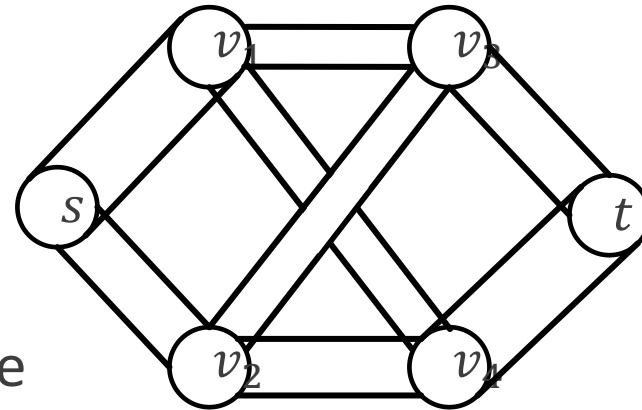
Small pipe

Flow network



source

Example: Oil Pipeline



sink



Flow below capacity

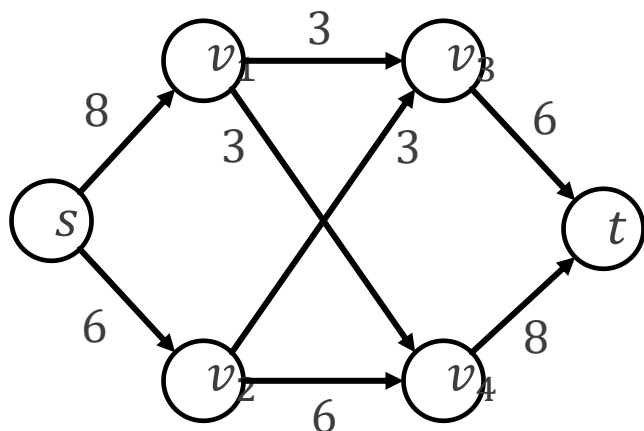


Maximum flow

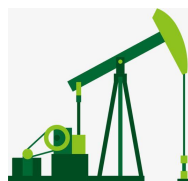


Max-Flow Problem

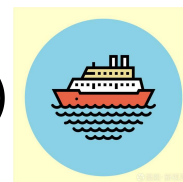
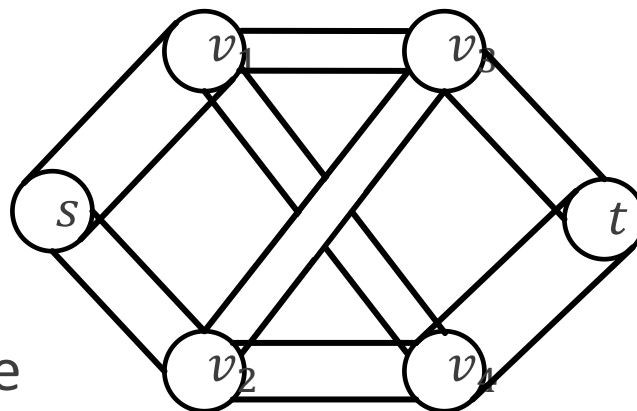
Flow network



Example: Oil Pipeline



source



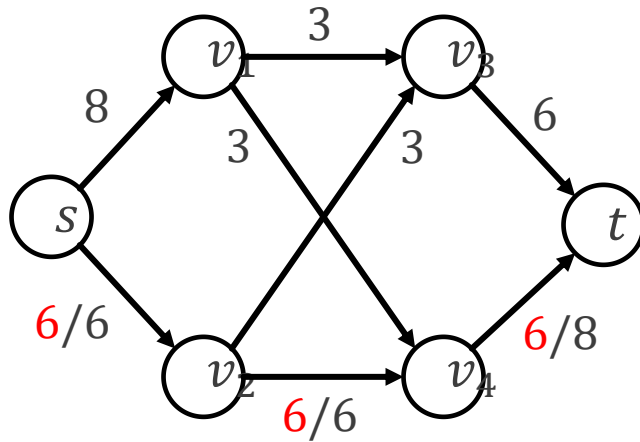
sink

- Informal definition of the max-flow problem:** What is the greatest rate at which material can be shipped from the source to the sink without violating any capacity constraints?

非正式的定义最大流问题：在遵守各条边容量限制的前提下，从源点到汇点最大的流量速率可能是多少？

Max-Flow Problem

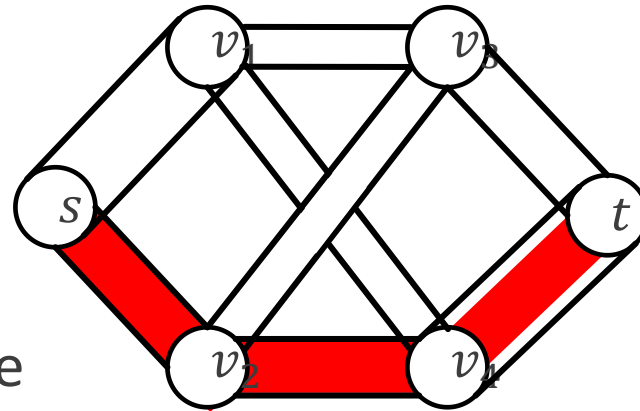
Flow network



Example: Oil Pipeline



source



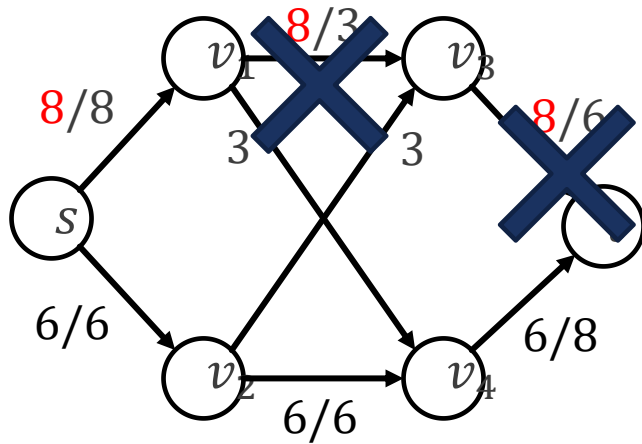
sink

flow = 6



Max-Flow Problem

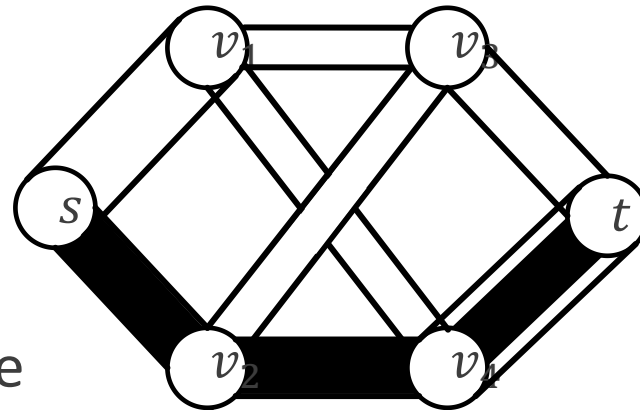
Flow network



Example: Oil Pipeline



source

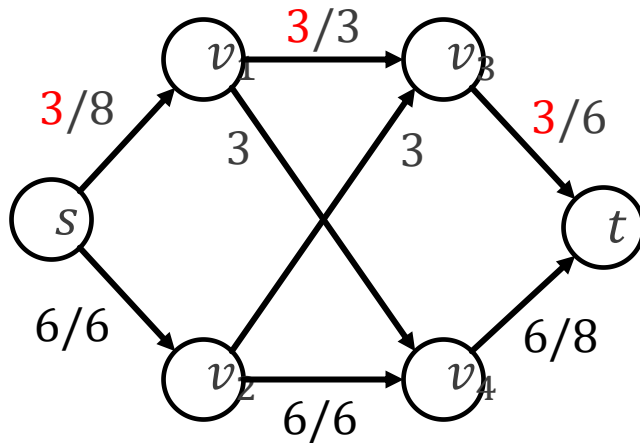


sink

Flow can't exceed capacity!

Max-Flow Problem

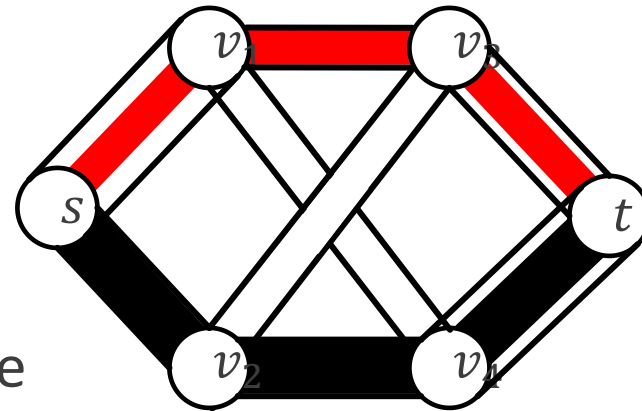
Flow network



Example: Oil Pipeline



source

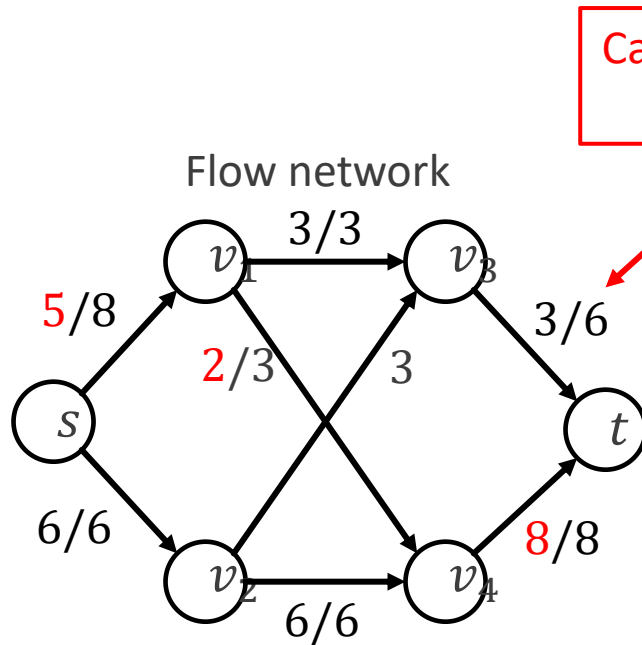


sink

flow = 9



Max-Flow Problem

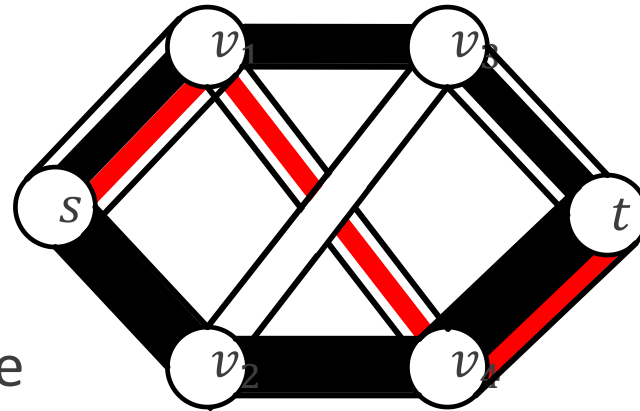


Can we improve
over here?



source

Example: Oil Pipeline



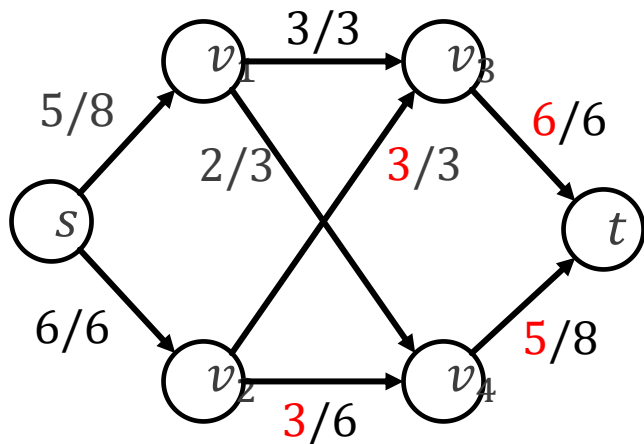
sink

$flow = 11$



Max-Flow Problem

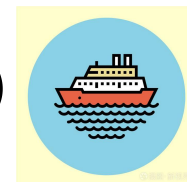
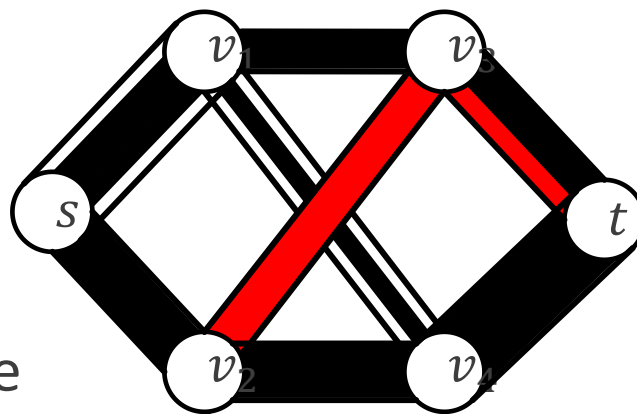
Flow network



Example: Oil Pipeline



source

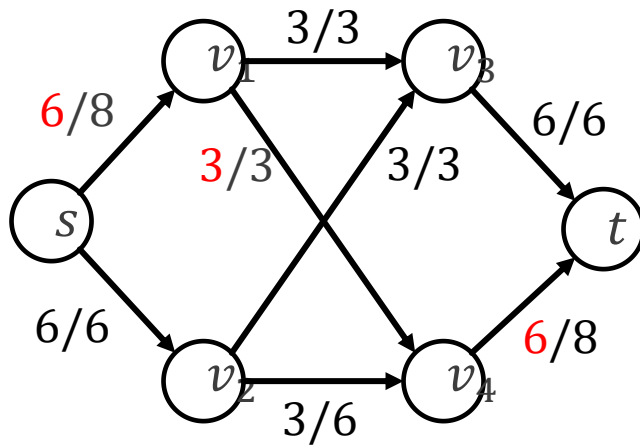


sink

flow = 11

Max-Flow Problem

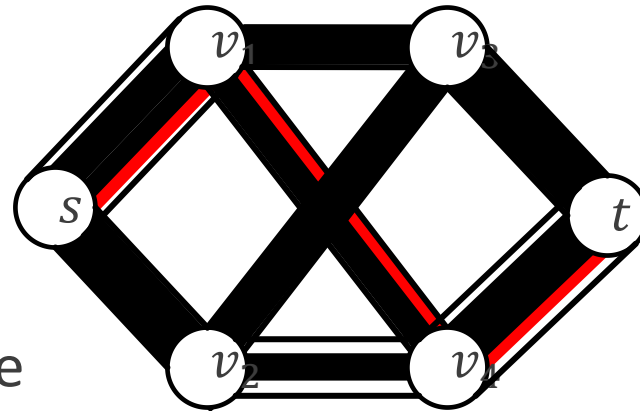
Flow network



Example: Oil Pipeline



source



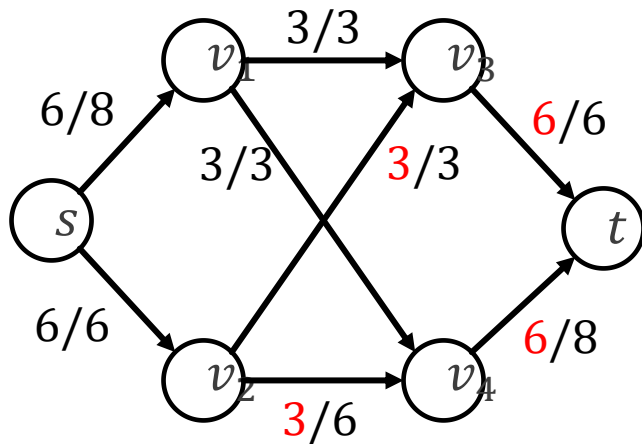
sink

flow = 12

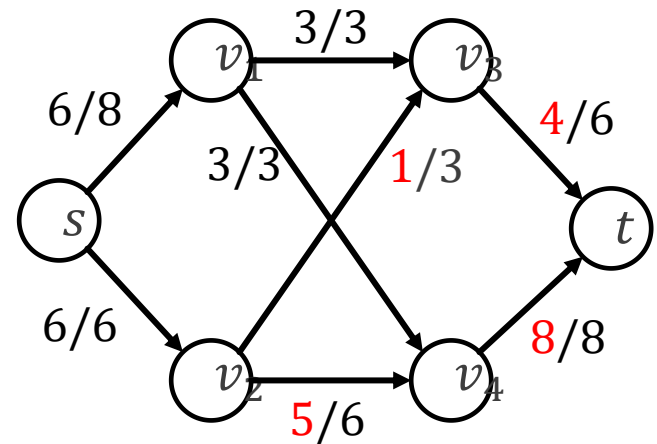


Max-Flow Problem

Flow network



Flow network



flow = 12



Definition 9.2

A **real valued function** $f: V \times V \rightarrow R$ in $G = (V, E)$ is **called flow (流)** if it satisfies 3 properties:

流从数学的意义上来讲就是找到**边和实数的映射**，并且这个映射满足以下3个性质

(1) **Capacity constraint (容量约束)**: $\forall u, v \in V$:

$$f(u, v) \leq c(u, v).$$

(2) **Skew symmetry (反对称)**: $\forall u, v \in V$:

$$f(u, v) = -f(v, u).$$

(3) **Flow conservation (流守恒)**: $\forall u \in V - \{s, t\}$:

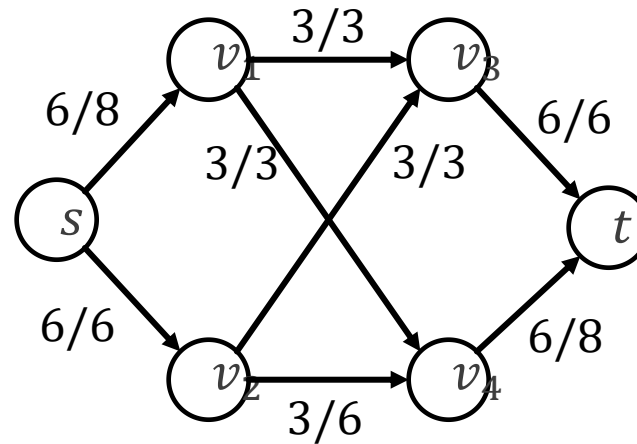
$$\sum_{v \in V} f(u, v) = 0.$$

where $f(u, v)$ is called **the flow** from vertex u to vertex v . If $f(u, v) = c(u, v)$, we call (u, v) a **saturated edge (饱和边)**.



Flow Properties

Flow conservation (流守恒)



We don't draw negative flow and zero capacity on the graph!

For v_1 :

$$f(v_1, s) = -6$$

$$f(v_1, v_3) = 3$$

$$f(v_1, v_4) = 3$$

$$\sum_{v \in V} f(v_1, v) = 0$$

For v_3 :

$$f(v_3, v_1) = -3$$

$$f(v_3, v_2) = -3$$

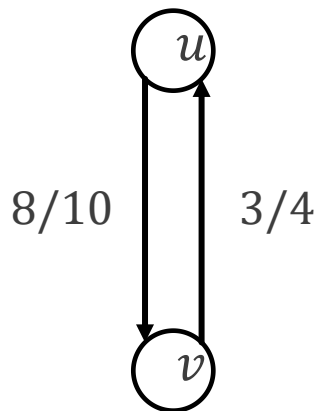
$$f(v_3, t) = 6$$

$$\sum_{v \in V} f(v_3, v) = 0$$



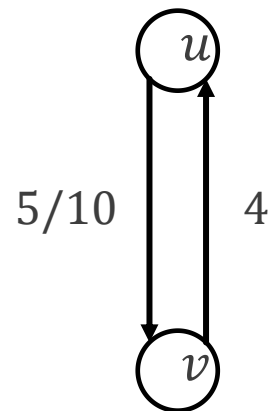
Cancellation

反对称性的定义 $f(u, v) = -f(v, u)$



$$\begin{aligned} f(u, v) &= 8 \\ f(v, u) &= 3 \end{aligned}$$

Cancellation



$$\begin{aligned} f(u, v) &= 5 \\ f(v, u) &= -5 \end{aligned}$$

Not satisfy skew symmetry

左边这个不满足反对称性，将这个修改成为右边的



Definition 9.2

The value of a flow f (f 的流量) is defined as

流量是指从源点 s 流向其他节点 v 的流量 $f(s, v)$ 的总和

$$|f| = \sum_{v \in V} f(s, v),$$

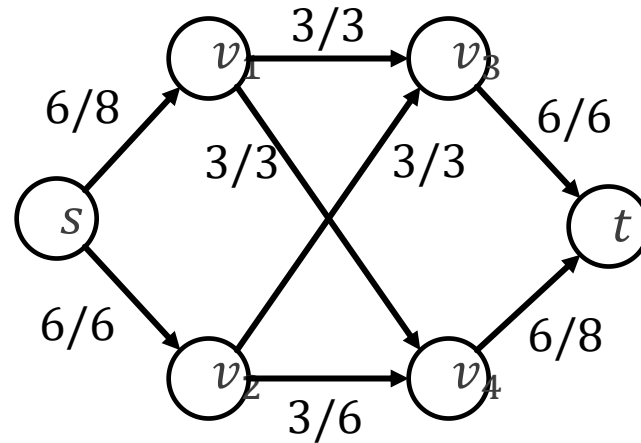
that is, the total flow out of the source s .

- Here, the $|\cdot|$ notation denotes flow value, not absolute value or cardinality.
- **Formal definition of the max-flow problem:** The max-flow problem is to **find a valid flow** for a given weighted directed graph G , that has the maximum value over all valid flows.

我们考虑的最大流问题就是在给定的带权有向图 G 上找到每条边对应 (u, v) 的流量 $f(u, v)$ （必须满足流的3个性质），并且使得源节点流出的流量最大



Flow



$$|f| = \sum_{v \in V} f(s, v) = f(s, v_1) + f(s, v_2) = 6 + 6 = 12.$$



MAX-FLOW PROBLEM

THE FORD-FULKERSON METHOD



The Ford-Fulkerson Method

- The Ford-Fulkerson method (可以简称为FF方法) is a way to find the max-flow.
- It has three important concepts:
 - (1) Residual networks (剩余网络)
 - (2) Augmenting paths (增广路径)
 - (3) Cuts of flow networks (网络的割)



Definition 9.3

Given a flow f on G with capacity function c , the residual capacity (剩余容量) function c_f on the set of pairs of vertices is defined as:

$$c_f(u, v) = c(u, v) - f(u, v)$$

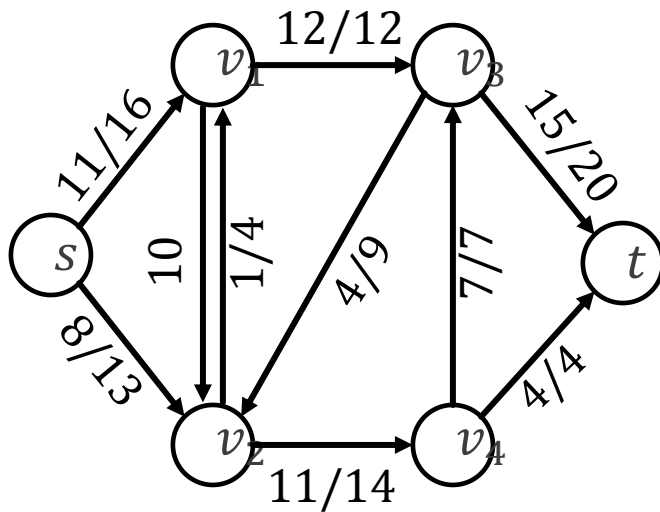
for each pair of vertices $u, v \in V$.

The residual network (剩余网络) for f is the direct graph $G_f = (V, E_f)$, with capacities defined by c_f and edge set $E_f = \{(u, v) | c_f(u, v) > 0\}$. 剩余网络的边，是原图中还有容量的边，这个意味着这条边的剩余容量



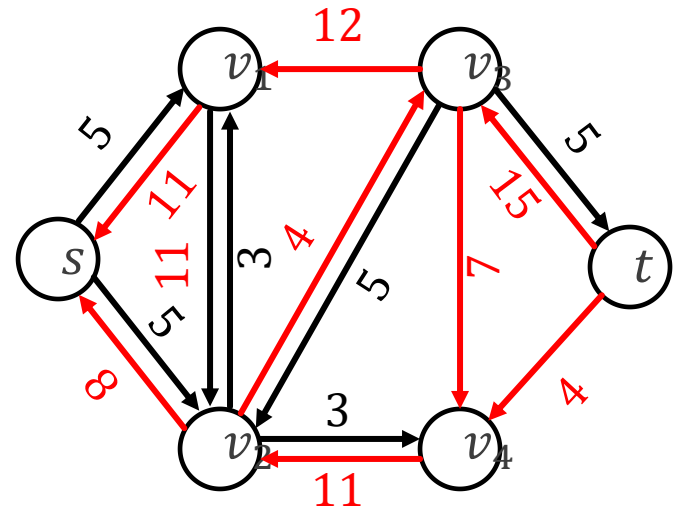
Residual Networks 剩余网络

$$c_f(u, v) = c(u, v) - f(u, v)$$



G

We don't draw negative flow and zero capacity on the graph!



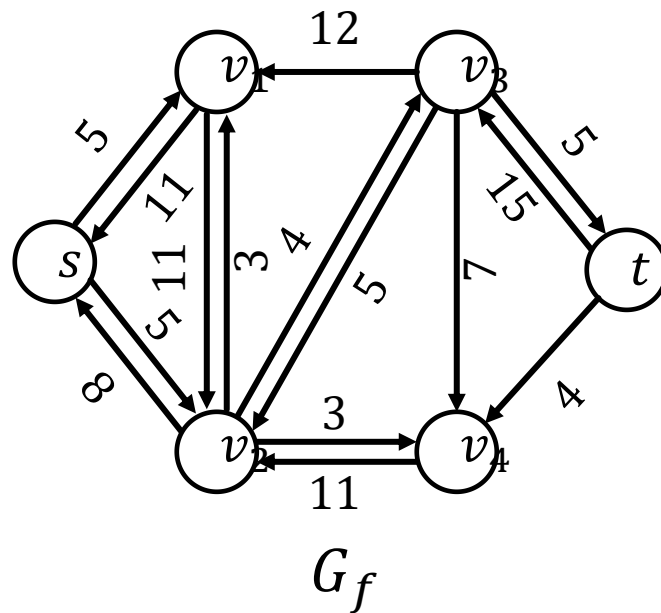
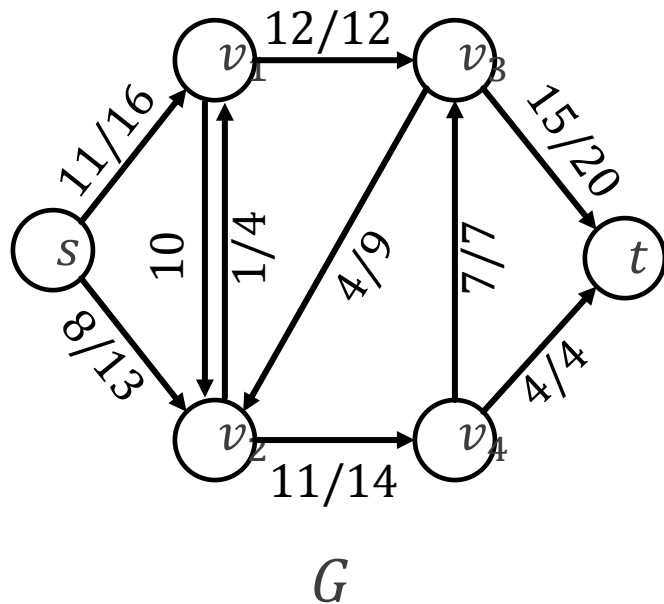
G_f

负的流和0的容量, 我们只是没有画, 实际也是存在的

Note: G_f is different with different flow f .
不同的流 f 会导致不同的 G_f



Residual Networks



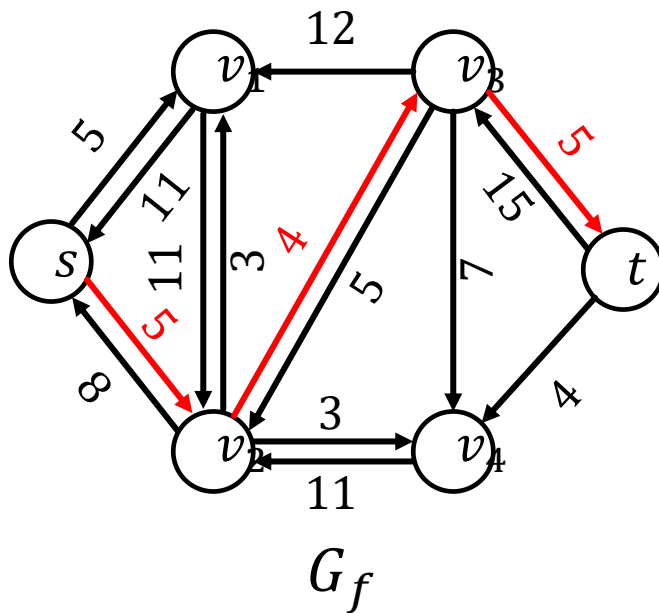
- Capacity in residual network simply means how much you can increase the flow at most.
- 在剩余网络中的容量意味着，这条边的这个方向还可以增加多少



Augmenting Paths

Definition 9.5

Given a residual network $G_f = (V, E_f)$, an **augmenting path** (增广路径) is a direct path from s to t . 从 s 到 t 的一条有向路径，在这条路径上可以再增加 f 的流量 $|f|$



If we can find such a path in the residual network, it means we can still increase the flow!



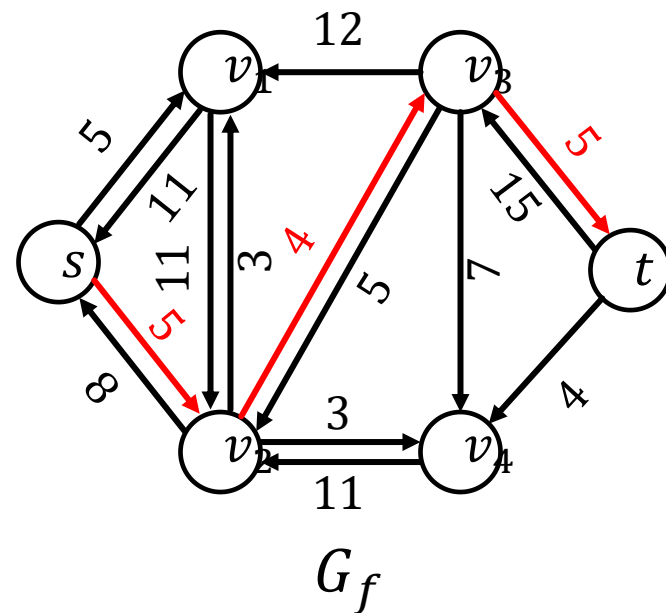
Bottleneck Capacity

Definition 9.5

Given a residual network G_f and an augmenting path p from s to t , we define $c_f(p)$ as the bottleneck capacity (瓶颈容量): 增广路径的瓶颈

$$c_f(p) = \min\{c_f(u, v): (u, v) \text{ is on } p\}.$$

- $c_f(p)$ is actually the maximum amount by which we can increase the flow on each edge in an augmenting path p . 增广路径能增加多少流量，取决于瓶颈



$$c_f(p) = 4$$



Augmenting Paths

这里我们定义了一个映射关系 f_p ，从边到实数的映射， f_p 是 G 的剩余网络 G_f 的流，并且这个流的流量等于其瓶颈容量

Lemma 9.2

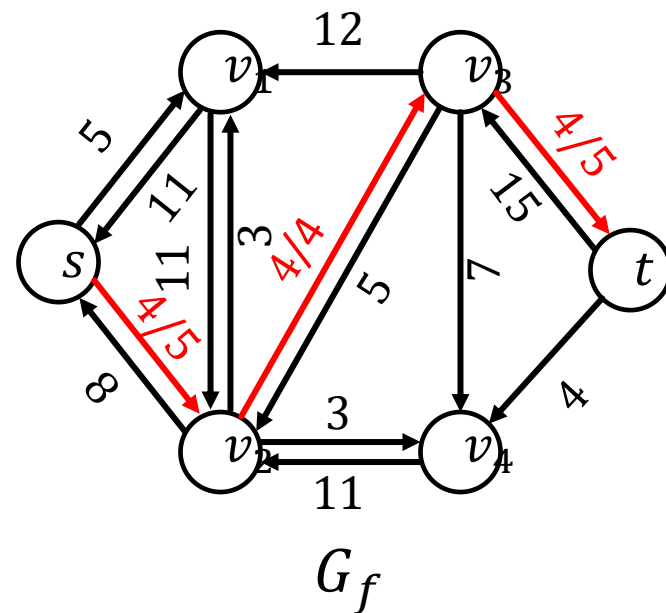
Let p be an augmenting path in G_f .

Define a function $f_p: V \times V \rightarrow R$ by

$$f_p(u, v) = \begin{cases} c_f(p) & \text{if } (u, v) \text{ is on } p \\ -c_f(p) & \text{if } (v, u) \text{ is on } p \\ 0 & \text{otherwise} \end{cases}$$

Then, f_p is a flow in G_f with value $|f_p| = c_f(p) > 0$.

- If we can find an augmenting path, there exists a flow of G_f .



$$\begin{aligned} f_p(s, v_2) &= f_p(v_2, v_3) \\ &= f_p(v_3, t) = c_f(p) = 4 \end{aligned}$$



Definitions So Far...

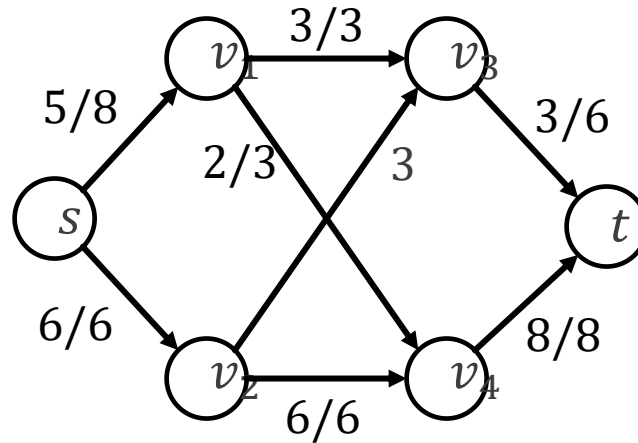
Given a network $G = (V, E)$, now we have a bunch of definitions:

- $c(u, v)$: capacity of $(u, v) \in E$. 边的容量
- $f(u, v)$: flow of $(u, v) \in E$. 边的流量
- f and $|f|$: a flow of G and its value. 图的流, 以及图的流量
- $G_f = (V, E_f)$: residual network of f . 图的剩余网络
- $c_f(u, v)$: residual capacity of $(u, v) \in E_f$. 边的剩余容量
- p : an augmenting path in G_f . 剩余网络的路径
- $c_f(p)$: bottleneck capacity. 路径p在剩余网络上的瓶颈容量
- f_p : a flow of G_f with augmenting path p . 剩余网络上增广路径的容量



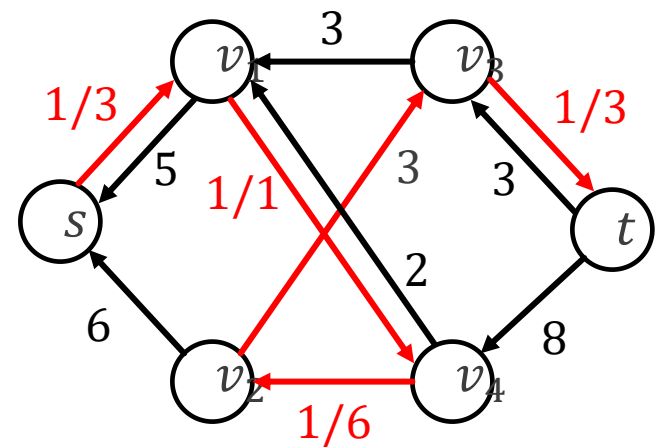
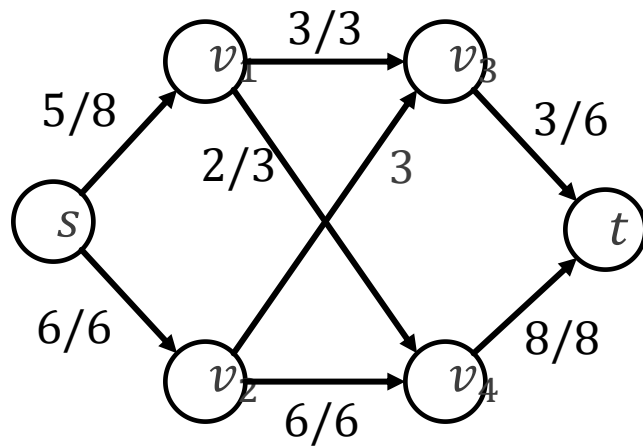
Classroom Exercise 课堂练习

Compute the residual network and find an augmenting path and its bottleneck capacity.





Classroom Exercise





Augmenting Paths 增广路径

- Once we obtain an augmenting path, we can improve f by f_p . We first define the sum of flows. 如果我们找到了 f 的增广路径 f_p ，那么我们就可以提高 f 的流量 $|f|$ 为 $|f| + |f_p|$:

Definition 9.4

Given two flows f_1 and f_2 on G , let $(f_1 + f_2)(u, v) = f_1(u, v) + f_2(u, v)$, $f_1 + f_2$ is called the sum of flows f_1 and f_2 .



Augmenting Paths

Lemma 9.1

Let $G = (V, E)$ be a flow network with source s and sink t , and let f be a flow in G . Let G_f be the residual network of G induced by f , and let f' be a flow in G_f . Then the flow sum $f + f'$ is a flow in G with value $|f + f'| = |f| + |f'|$. 给定一个连通的图 $G = (V, E)$ ，图上源点为 s ，汇点为 t ， f 为图 G 上的流。 G_f 为 G 关于流 f 的剩余网络， f' 是剩余网络 G_f 上的流，请两个流的和 $f + f'$ 仍然是图 G 上的流，并且流量 $|f + f'| = |f| + |f'|$

- **Lemma tells you:** if you find a flow f' of G_f , no worry and simply add it onto f , the resulting flow sum is also a flow in G .
- 在图 G 的流 f 上叠加剩余网络 G_f 的流 f' ，还是流，并且新的流的流量 $|f + f'|$ 等于 $|f| + |f'|$
- Adding f' to f will not explode. Why?



Augmenting Paths

首先证明，叠加之后还是流，就要证明符合流的3个性质

- 容量约束
- 反对称
- 流守恒

To show that $f + f'$ is also a flow in G , we need to show that $f + f'$ satisfies the three properties of flow, and $|f + f'| = |f| + |f'|$.

Proof:

(1) Capacity constraint: $\forall u, v \in V, f(u, v) \leq c(u, v)$.

$$\begin{aligned}(f + f')(u, v) &= f(u, v) + f'(u, v) \\ &\leq f(u, v) + c_f(u, v) \\ &= f(u, v) + (c(u, v) - f(u, v)) \\ &= c(u, v)\end{aligned}$$

Capacity
constraint
of f' .

Definition of c_f .



Augmenting Paths

首先证明，叠加之后还是流，就要证明符合流的3个性质

- 容量约束
- 反对称
- 流守恒

Proof (cont'd):

(2) Skew symmetry: $\forall u, v \in V, f(u, v) = -f(v, u)$.

$$\begin{aligned}(f + f')(u, v) &= f(u, v) + f'(u, v) \\ &= -f(v, u) - f'(v, u) \\ &= -(f(v, u) + f'(v, u)) \\ &= -(f + f')(v, u)\end{aligned}$$

Skew
symmetry
of f and f' .

Definition
of flow sum



Augmenting Paths

首先证明，叠加之后还是流，就要证明符合流的3个性质

- 容量约束
- 反对称
- 流守恒

Proof (cont'd):

(3) Flow conservation: $\forall u \in V - \{s, t\}, \sum_{v \in V} f(u, v) = 0$.

$$\begin{aligned} \sum_{v \in V} (f + f')(u, v) &= \sum_{v \in V} (f(u, v) + f'(u, v)) \\ &= \sum_{v \in V} f(u, v) + \sum_{v \in V} f'(u, v) \\ &= 0 + 0 = 0 \end{aligned}$$



Augmenting Paths

证明叠加之后是流，再证明 $|f + f'| = |f| + |f'|$.

Proof (cont'd):

Finally, we prove $|f + f'| = |f| + |f'|$.

$$\begin{aligned} |f + f'| &= \sum_{v \in V} (f + f')(s, v) \\ &= \sum_{v \in V} f(s, v) + \sum_{v \in V} f'(s, v) \\ &= |f| + |f'| \end{aligned}$$



Augmenting Paths

- Everything is prepared for **increase the value of flow** to reach max-flow.

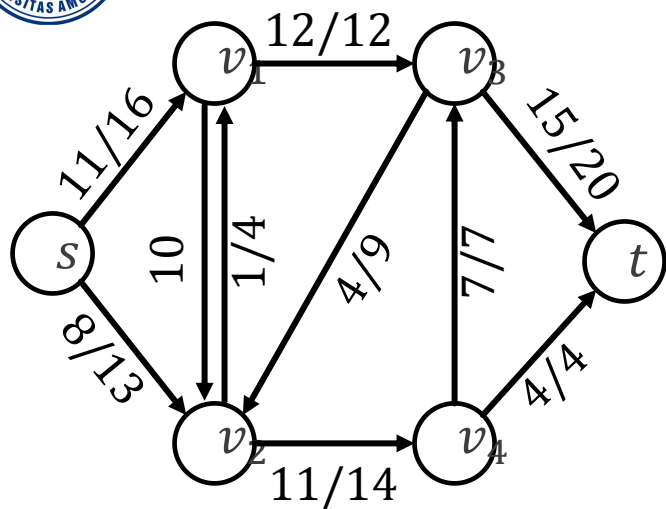
Proposition 9.1 推论9.1

Let $G = (V, E)$ be a flow network, let f be a flow in G , p be an augmenting path in G_f , f_p be a flow of G_f with augmenting path p .

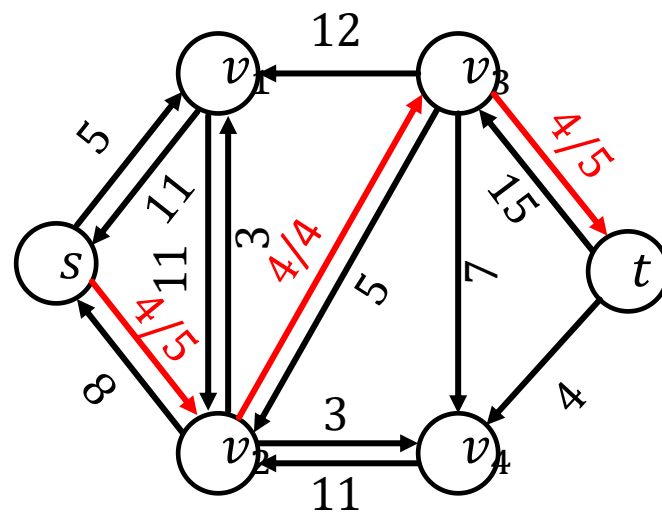
Then $f + f_p$ is a flow in G with value $|f| + |f_p| > |f|$.

- This Proposition tells you: Adding f_p must result in larger flow value.

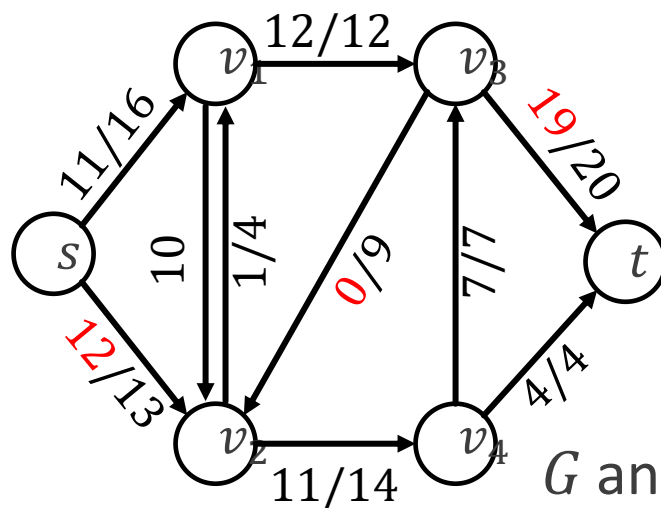
如果找到一条增广路径，那么就可以把这个增广路径的流量，加到 f 上



G and f
 $|f| = 19$



G_f and f_p
 $|f_p| = 4$



G and $f + f_p$
 $|f + f_p| = 23$



Augmenting Paths

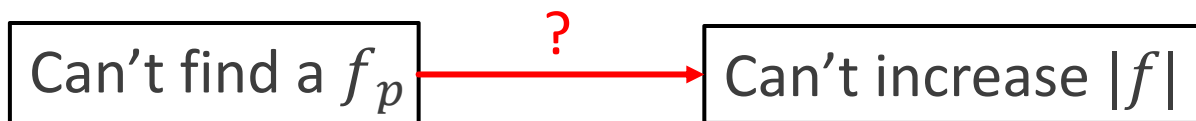
- Proposition 9.1 indicates 推论9.1表明:



- By contrapositive law (假言易位), we know:



- However, we don't know:



从逻辑推理关系看充分条件、必要条件:

- 1) $p \rightarrow q$ 且 $q \not\rightarrow p$, 则 p 是 q 的
充分非必要条件
- 2) $p \not\rightarrow q$ 且 $q \rightarrow p$, 则 p 是 q 的
必要非充分条件
- 3) $p \not\rightarrow q$ 且 $q \not\rightarrow p$, 则 p 是 q 的
既不充分也不必要条件
- 4) $p \rightarrow q$ 且 $q \rightarrow p$, 则 p 是 q 的
充分且必要条件

contrapositive law $A \rightarrow B \Leftrightarrow \neg B \rightarrow \neg A$



Augmenting Paths

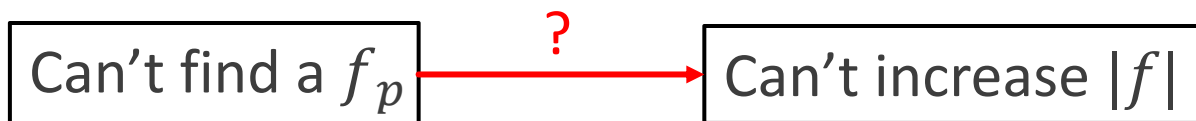
- Proposition 9.1 indicates:



- By contrapositive law (假言易位), we know:



- However, we don't know:



如果再也找不到增广路径了，那么得到的流是最大流么？
后续我们围绕这个问题，研究最小割，最大流，增广路径之间的关系。



Cuts of Flow Networks

Definition 9.7

A **cut (割)** (S, T) of a flow network $G = (V, E)$ is a partition of V into S and $T = V - S$ such that $s \in S$ and $t \in T$.

The capacity $c(S, T)$ of the cut $\{S, T\}$ is denoted as:

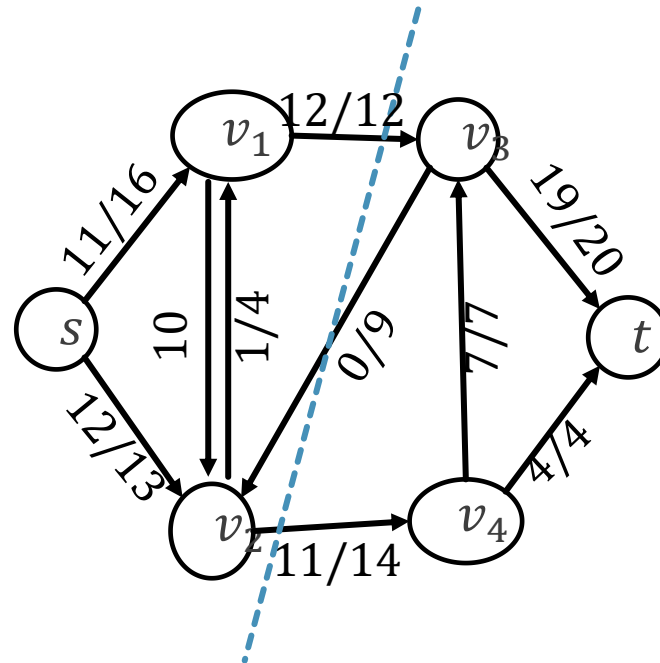
$$c(S, T) = \sum_{u \in S, v \in T} c(u, v).$$

The flow $f(S, T)$ across the cut $\{S, T\}$ is denoted as:

$$f(S, T) = \sum_{u \in S, v \in T} f(u, v).$$



Cuts of Flow Networks



$$S = \{s, v_1, v_2\}, \quad T = \{v_3, v_4, t\}$$

$$c(S, T) = c(v_1, v_3) + c(v_2, v_4) = 12 + 14 = 26$$

$$f(S, T) = f(v_1, v_3) + f(v_2, v_4) + f(v_2, v_3) = 12 + 11 - 0 = 23$$



$s \in S$ and $t \in T$

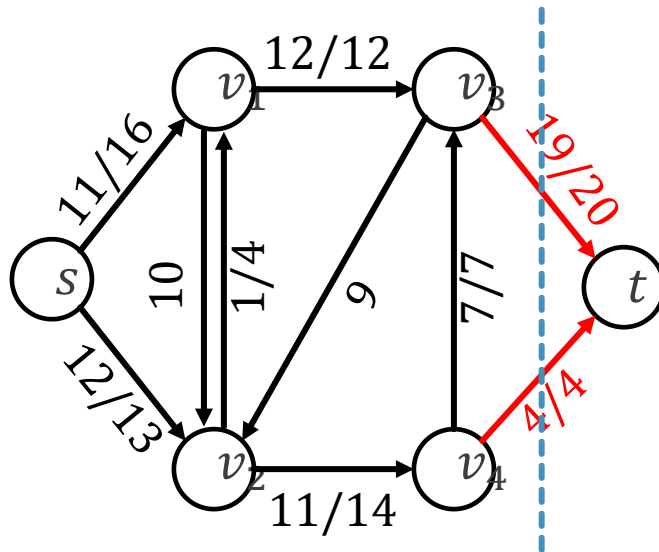
Lemma 9.3

Given a flow network $G = (V, E)$, for any cut $\{S, T\}$ and a flow f , $f(S, T) = |f|$. 对于任意的切割 $\{S, T\}$ ， $f(S, T) = |f|$.

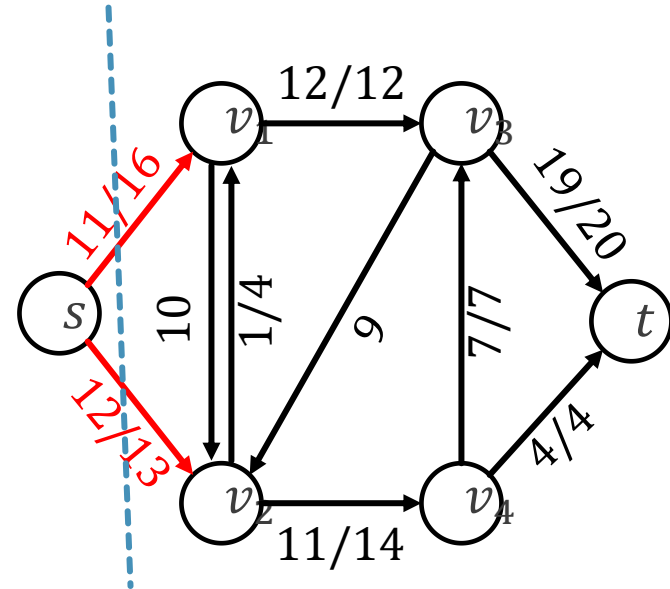
- This Lemma tells you: No matter how you cut, the flow across the cut is same and equals to the value of the flow. 一个图的流一旦定义好了，不管怎么切，切割的流量都是一样的。



Property of Cuts



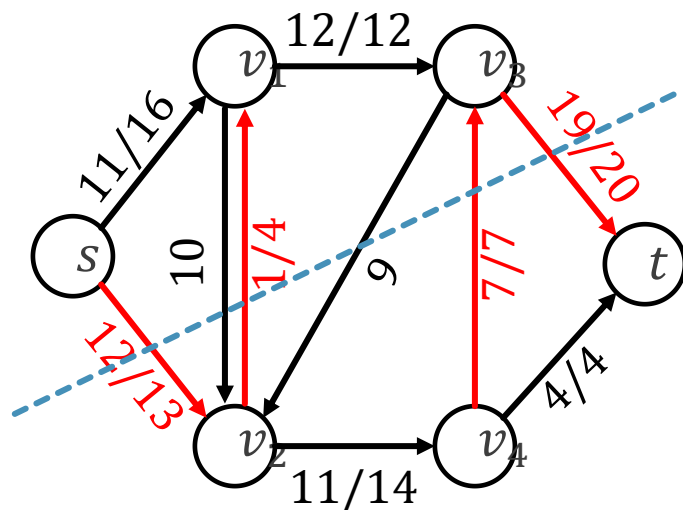
$$\begin{aligned}
 f(S, T) &= f(v_3, t) + f(v_4, t) \\
 &= 19 + 4 = 23
 \end{aligned}$$



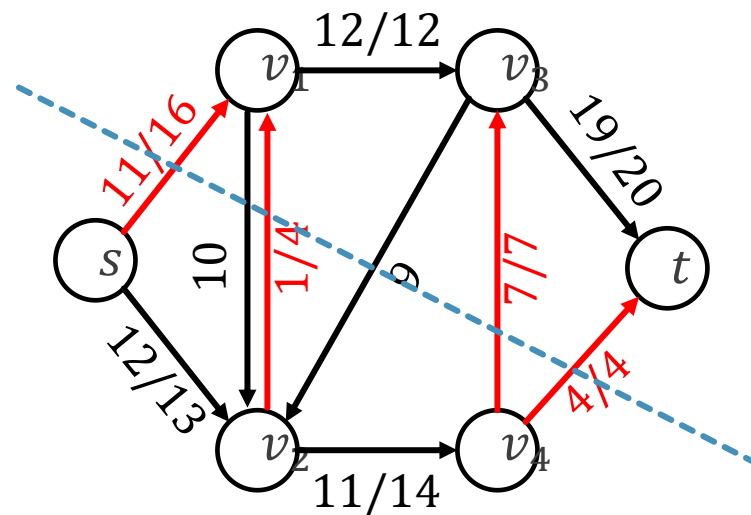
$$\begin{aligned}
 f(S, T) &= f(s, v_1) + f(s, v_2) \\
 &= 11 + 12 = 23
 \end{aligned}$$



Property of Cuts



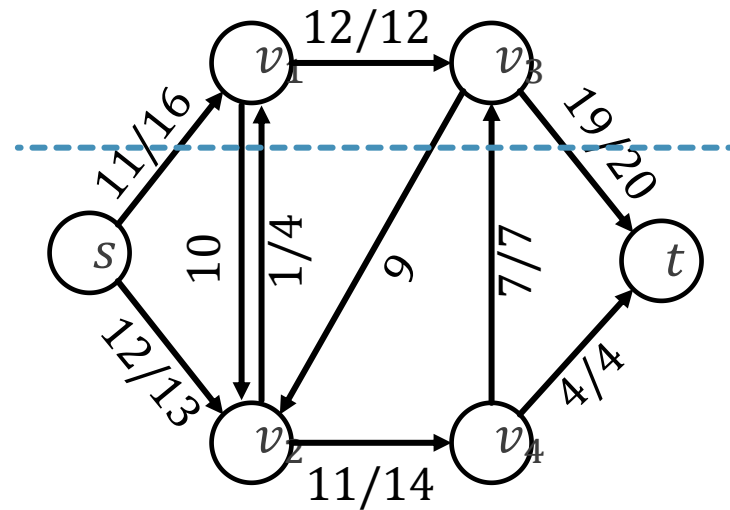
$$\begin{aligned}
 f(S, T) &= f(s, v_2) + f(v_1, v_2) + f(v_3, v_4) \\
 &\quad + f(v_3, t) \\
 &= 12 - 1 - 7 + 19 = 23
 \end{aligned}$$



$$\begin{aligned}
 f(S, T) &= f(s, v_1) + f(v_2, v_1) + f(v_4, v_3) \\
 &\quad + f(v_4, t) \\
 &= 11 + 1 + 7 + 4 = 23
 \end{aligned}$$



Property of Cuts



Not a valid cut! s and t are not separated.



Property of Cuts

Lemma 9.3

Given a flow network $G = (V, E)$, for any cut $\{S, T\}$ and a flow f , $f(S, T) = |f|$. 对于任意的切割 $\{S, T\}$ ， $f(S, T) = |f|$ 。

Proof:

By **induction** on the number of **vertices in S** .

If $S = \{s\}$, it is true by the definition of flow: $|f| = \sum_{v \in V} f(s, v)$.

Assume it is true for the cut $\{S, T\}$, namely $f(S, T) = |f|$. We show that it also holds for the cut $\{S \cup \{w\}, T - \{w\}\}$ for $w \in T - \{s, t\}$:

$$\begin{aligned} f(S \cup \{w\}, T - \{w\}) &= f(S, T - \{w\}) + f(w, T - \{w\}) \\ &= f(S, T) - f(S, w) + f(w, T) - f(w, w) \\ &= f(S, T) + f(w, S) + f(w, T) \\ &= f(S, T) + f(w, V) \\ &= f(S, T) + 0 = |f| \end{aligned}$$

By definition of cut

Flow conservation property



Corollary 9.2

The value of **any flow f** in a flow network G is bounded from above by **the capacity of any cut of G** , namely $|f| \leq c(S, T)$.

Proof:

$$\begin{aligned} |f| &= f(S, T) \\ &= \sum_{u \in S} \sum_{v \in T} f(u, v) \leq \sum_{u \in S} \sum_{v \in T} c(u, v) = c(S, T) \end{aligned}$$

- It means that max-flow can't exceed min-cut.

图的流量不能超过最小割的容量



Max-Flow Min-Cut Theorem

证明，只要找不到增广路径，那么就找到最大流了

Theorem 9.1 (max-flow min-cut theorem) 最大流最小割定理

If f is a flow in a flow network $G = (V, E)$ with source s and sink t , then the following conditions are equivalent:

- (1) f is a maximum flow in G .
- (2) The residual network G_f contains no augmenting paths.
- (3) $|f| = c(S, T)$ for some cut (S, T) of G .

- This Theorem tells you: When you can't find an augmenting path, you have obtained the max-flow.
- We prove (1)->(2), (2)->(3), and (3)->(1).



Max-Flow Min-Cut Theorem

- (1) f is a maximum flow in G .
- (2) The residual network G_f contains no augmenting paths.

Proof of (1) $\rightarrow$ (2):

- We prove by **contradiction** that f is a maximum flow in G but there **still exists** an augmenting path p in G_f .

- Then by Corollary 9.1, we can augment the flow in G :

$$|f'| = |f + f_p| > |f|$$

- The flow f' is strictly greater than f which is in **contradiction** to our assumption that f is a **maximum flow**.



Max-Flow Min-Cut Theorem

(2) The residual network G_f contains no augmenting paths.

(3) $|f| = c(S, T)$ for some cut (S, T) of G .

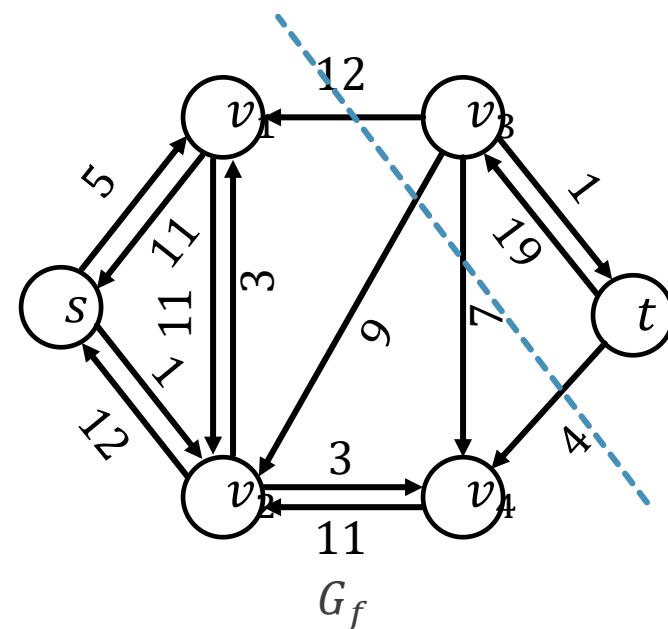
Proof of (2)->(3):

■ Define 定义一个割

$$S = \{v \in V \mid \exists \text{ path } p \text{ from } s \text{ to } v \text{ in } G_f\}$$

$$T = V - S$$

- We have $s \in S$ and $t \notin S$, otherwise there exists an augmenting path from s to t .
- Therefore, (S, T) is a cut of G by definition.



$$S = \{s, v_1, v_2, v_4\}$$

$$T = \{v_3, v_4, t\}$$



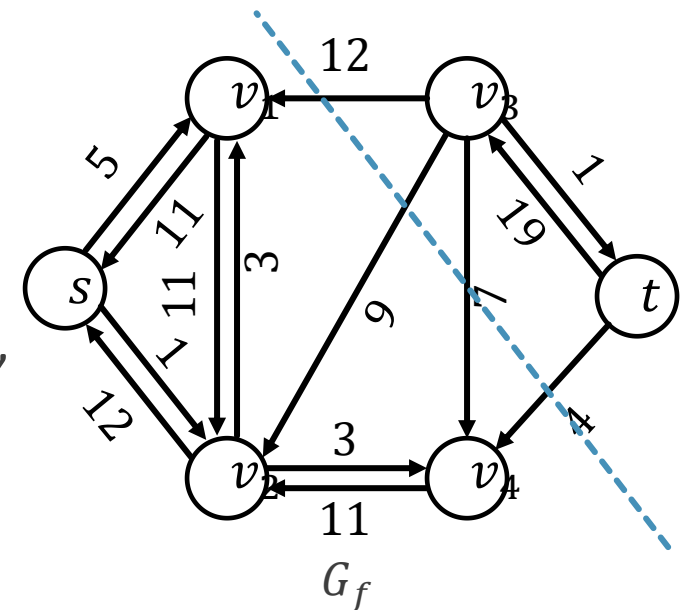
Max-Flow Min-Cut Theorem

(2) The residual network G_f contains no augmenting paths.

(3) $|f| = c(S, T)$ for some cut (S, T) of G .

Proof of (2) $\rightarrow$ (3) (cont'd):

- All edges (u, v) from S to T are saturated, i.e. $f(u, v) = c(u, v)$.
- Otherwise, there exists a path from u to v , i.e. $(u, v) \in E_f$, which means $v \in S$ contradicting to $v \in T$.
- Therefore, $f(S, T) = c(S, T)$ and $|f| = f(S, T)$ by Lemma 9.3, we have $|f| = c(S, T)$.



$$S = \{s, v_1, v_2, v_4\}$$
$$T = \{v_3, v_4, t\}$$



Max-Flow Min-Cut Theorem

(3) $|f| = c(S, T)$ for some cut (S, T) of G .

(1) f is a maximum flow in G .

Proof of (3) $\rightarrow$ (1):

- By Lemma 9.2, $|f| \leq c(S, T)$ for any cut (S, T) .
- Now we find a cut (S, T) such that $|f| = c(S, T)$. It implies that f is a maximum flow.
 - $|f|$ must be maximum, otherwise some other $|f|$ will be greater than $c(S, T)$.
 - Cut (S, T) must be the minimum cut, otherwise $|f|$ will be greater than some other $c(S, T)$. 这个割肯定也是最小割

证明了，只要找不到增广路径，那么就找到最大流了



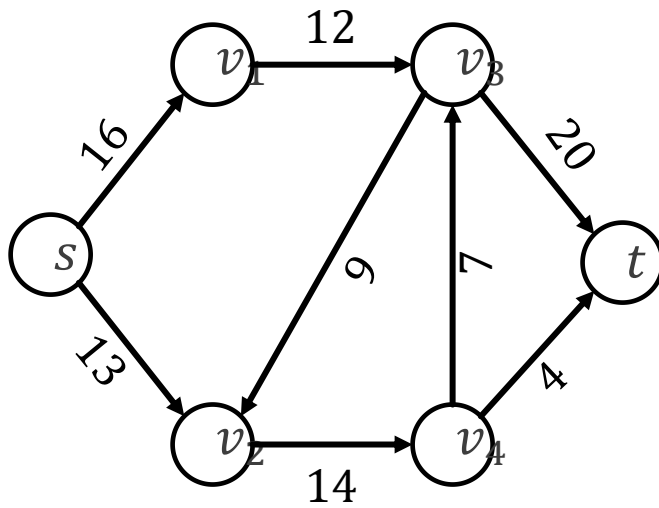
Ford-Fulkerson Method (FF方法)

```
FordFulkerson( $G, s, t, c$ )  
1 for each edge  $(u, v) \in E$  do  
2    $f(u, v) \leftarrow 0$   
3  $G_f \leftarrow G$   
4 while there is an augmenting path  $p$  in  $G_f$  do  
5   Let  $c_f(p)$  be the bottleneck capacity of  $p$   
6   for each edge  $(u, v)$  in  $p$  do  
7      $f(u, v) \leftarrow f(u, v) + c_f(p)$   
8   Update the residual graph  $G_f$ 
```

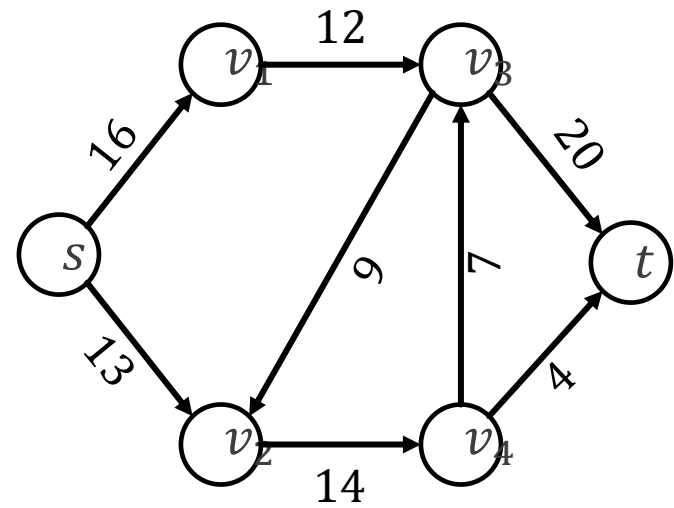


Example

Initialization



G

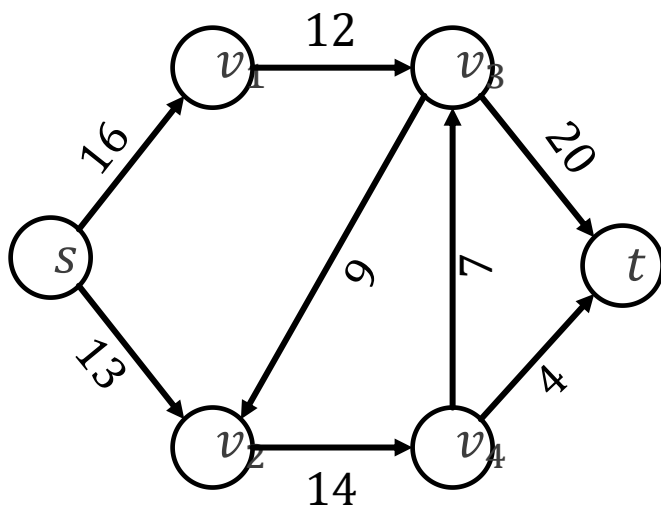


G_f

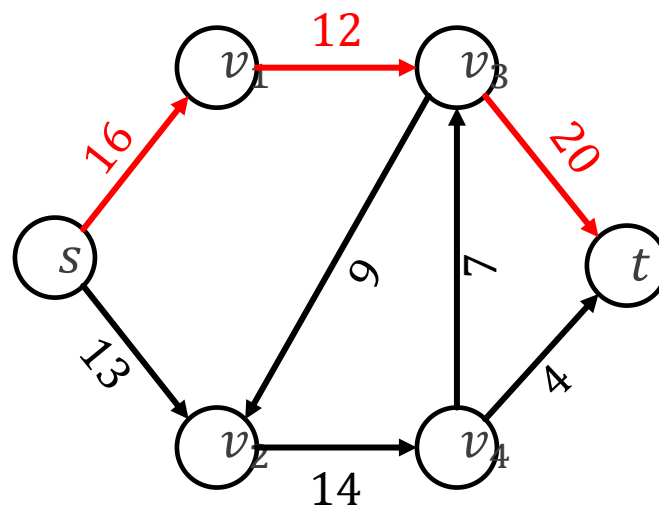


Example

Find an augmenting path p



G

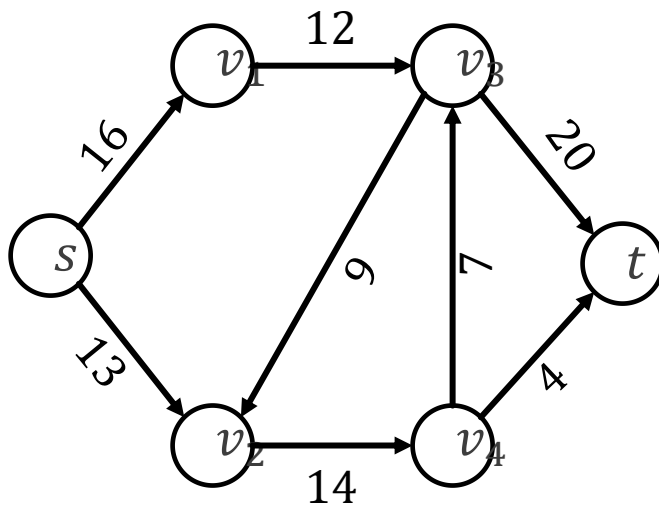


G_f

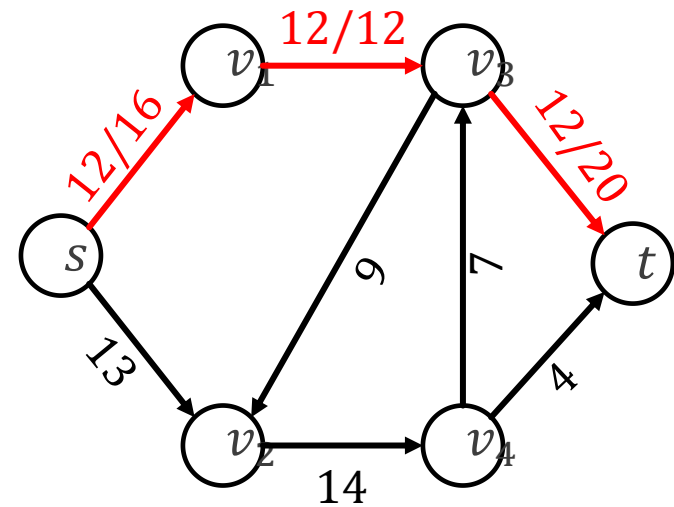


Example

Determine the bottleneck capacity $c_f(p) = 12$



G

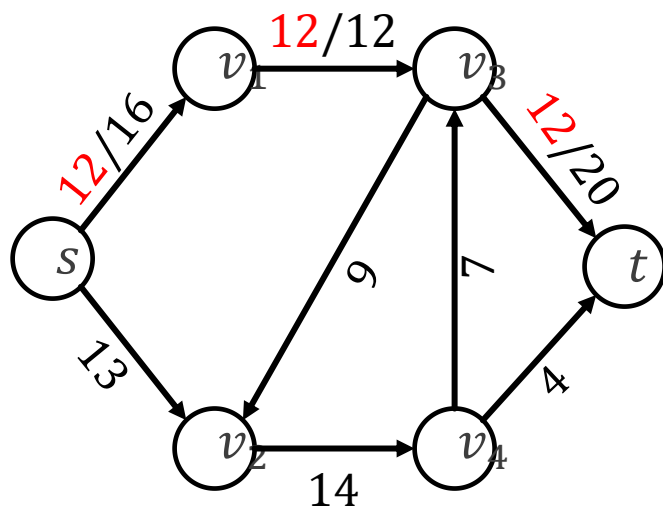


G_f

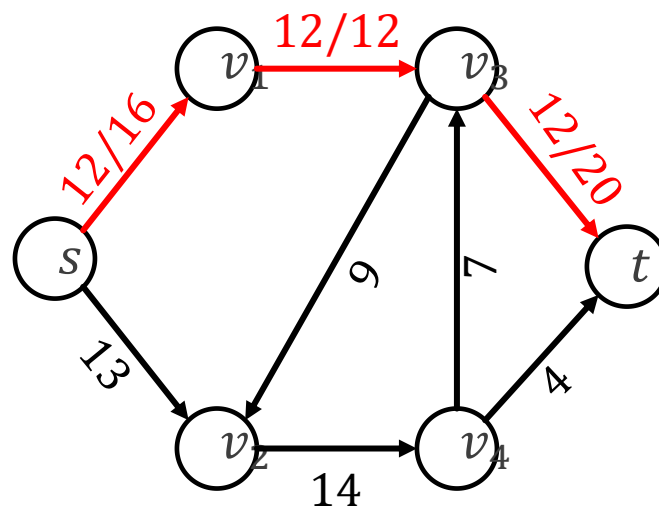


Example

Add the flow f_p to f



G

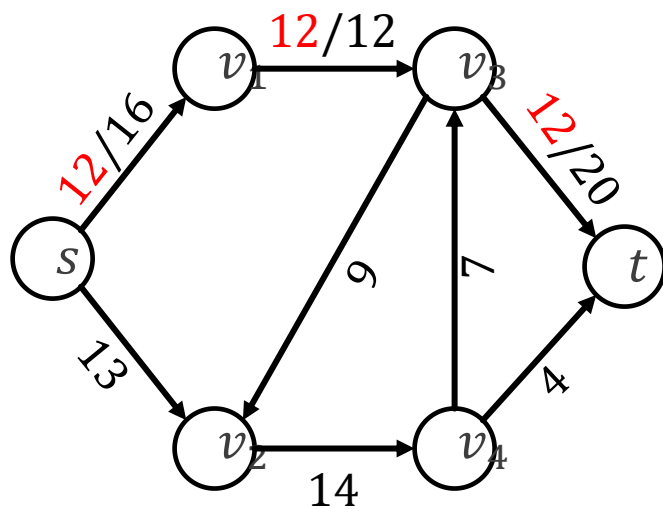


G_f

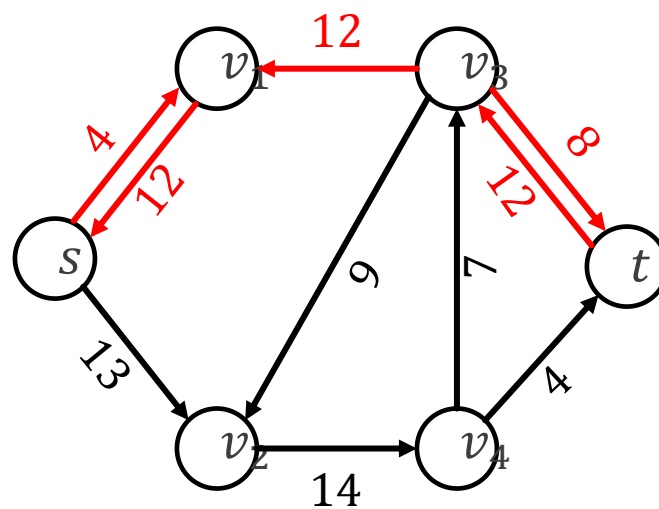


Example

Update G_f



G

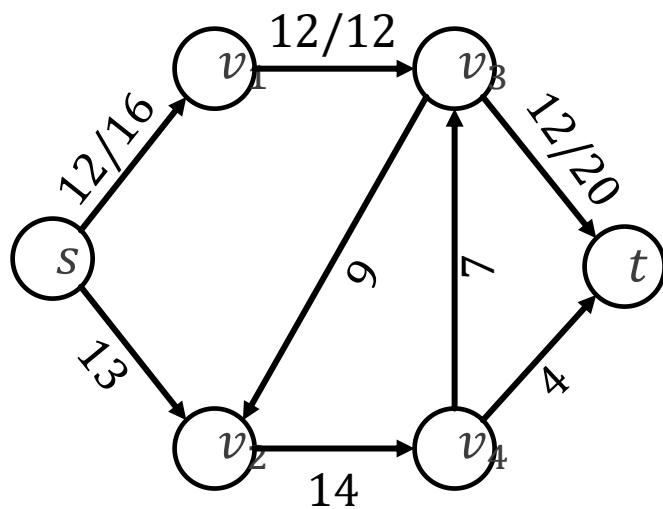


G_f

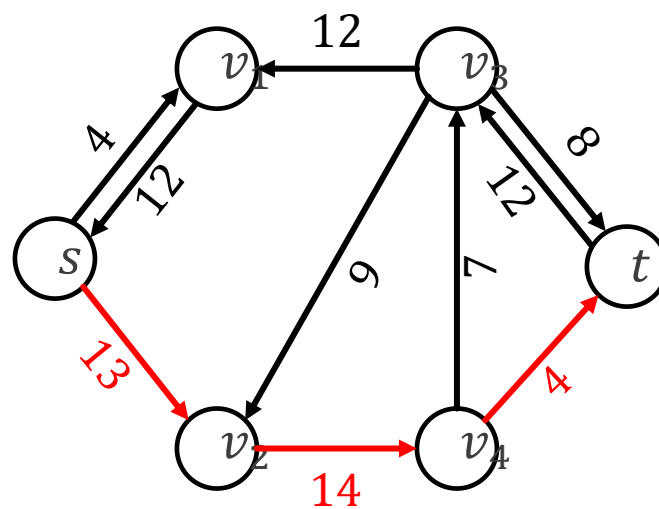


Example

Find an augmenting path p



G

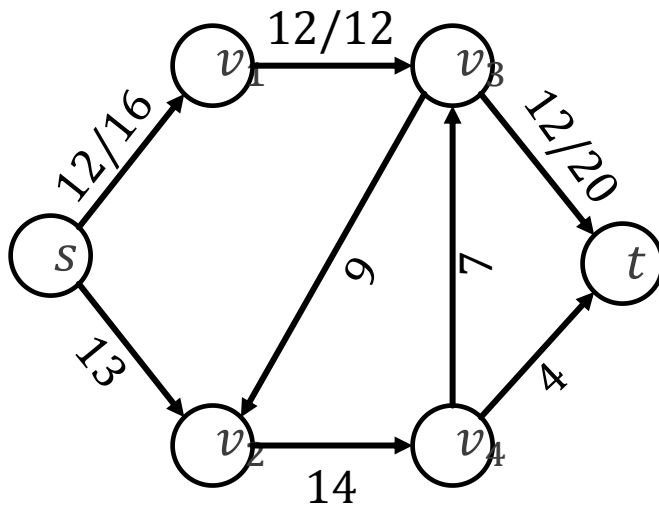


G_f

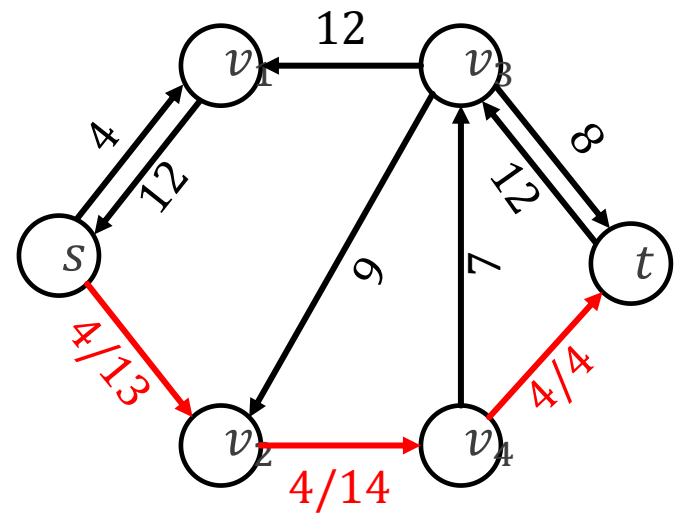


Example

Determine the bottleneck capacity $c_f(p) = 4$



G

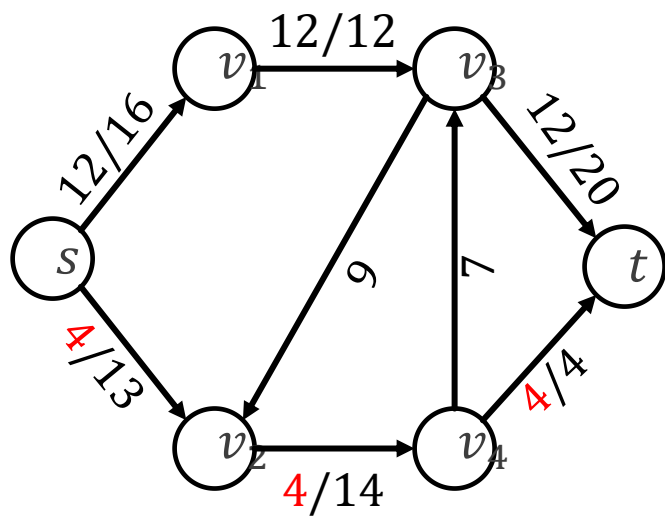


G_f

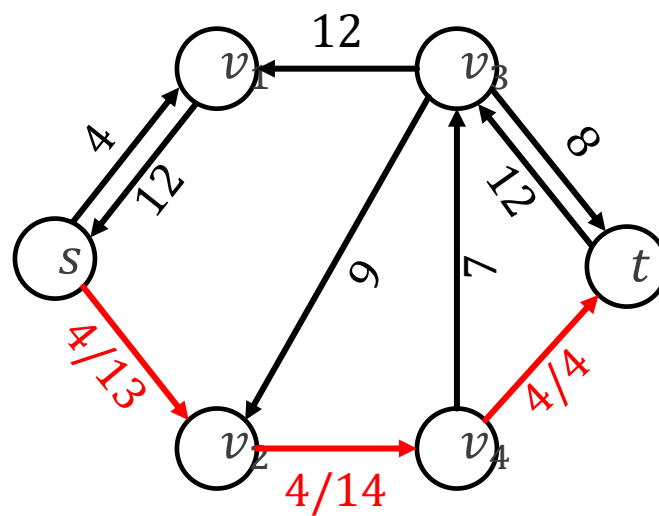


Example

Add the flow f_p to f



G

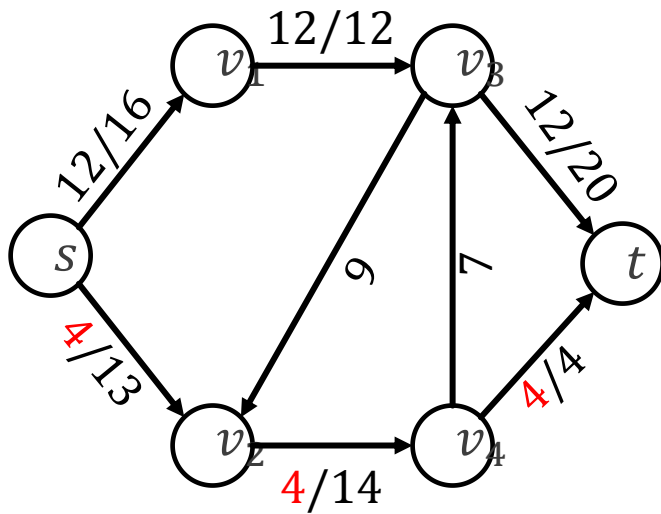


G_f

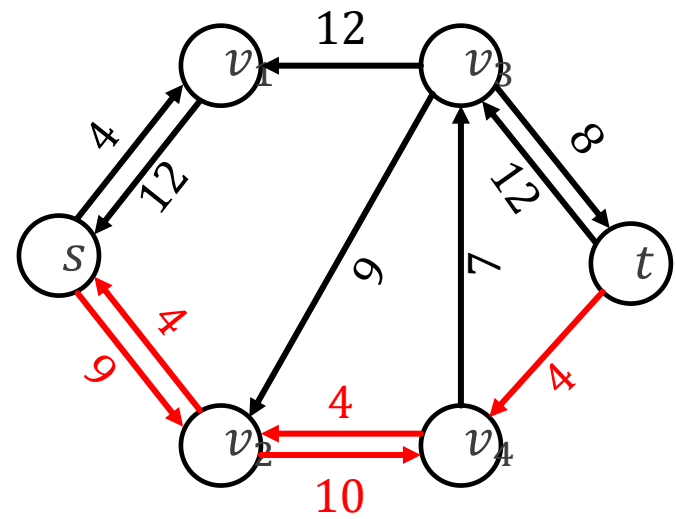


Example

Update G_f



G

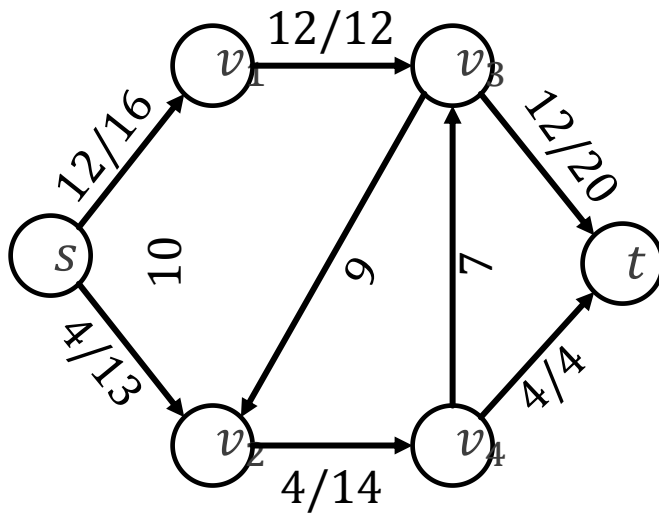


G_f

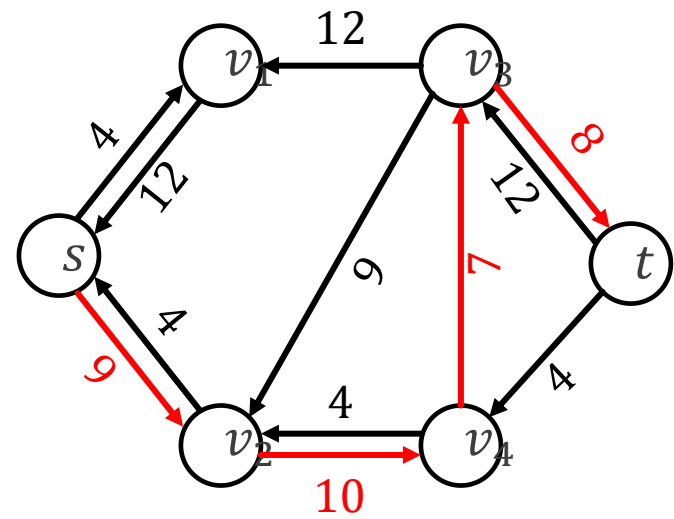


Example

Find an augmenting path p



G

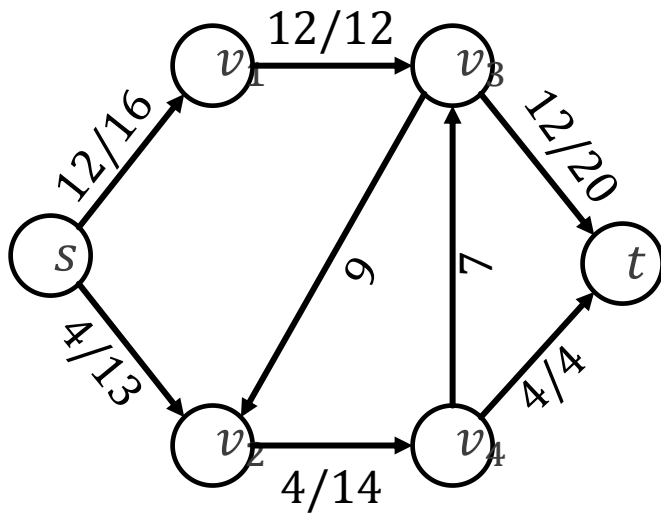


G_f

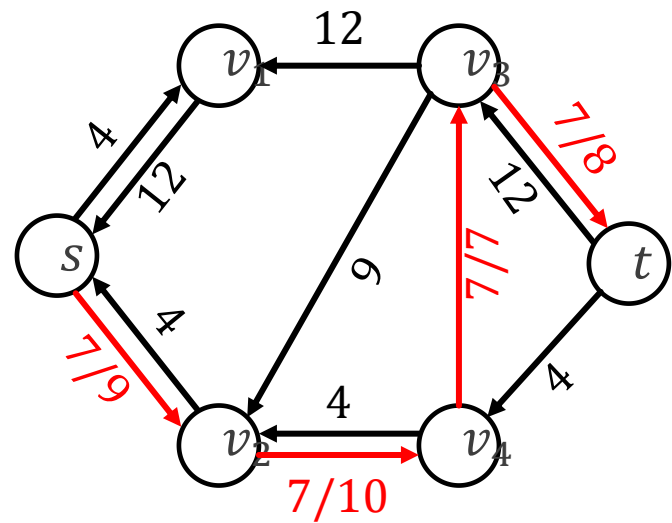


Example

Determine the bottleneck capacity $c_f(p) = 7$



G

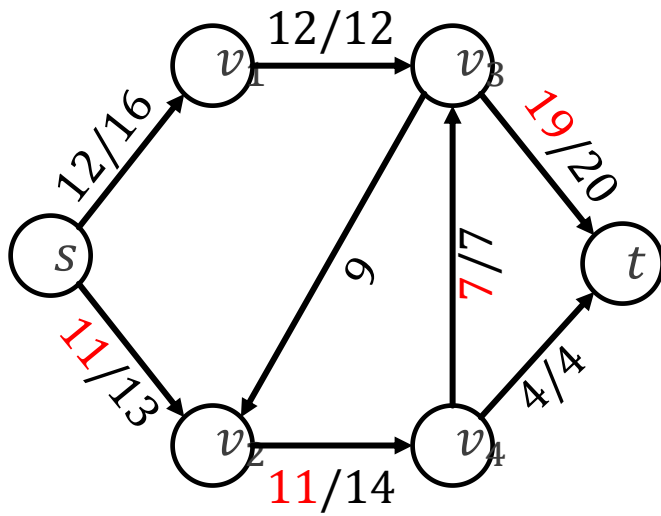


G_f

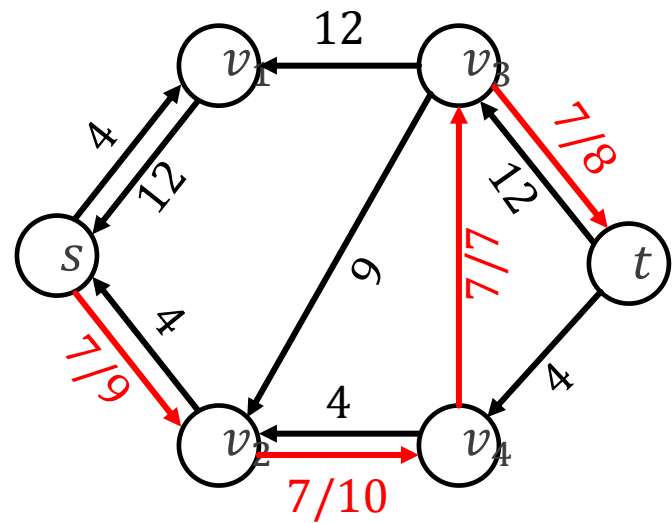


Example

Add the flow f_p to f



G

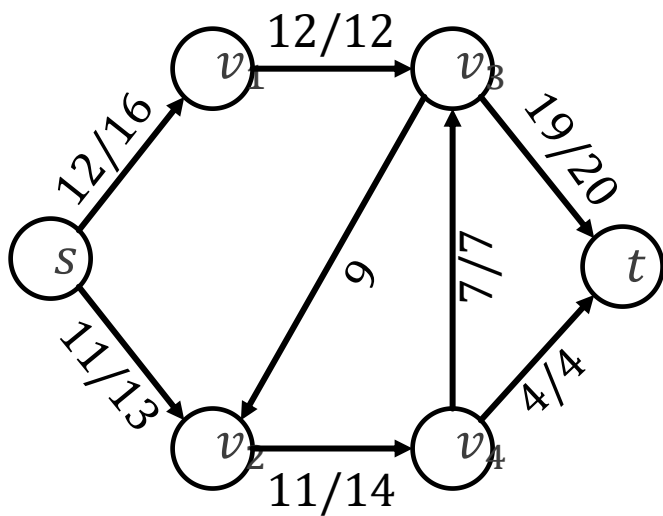


G_f

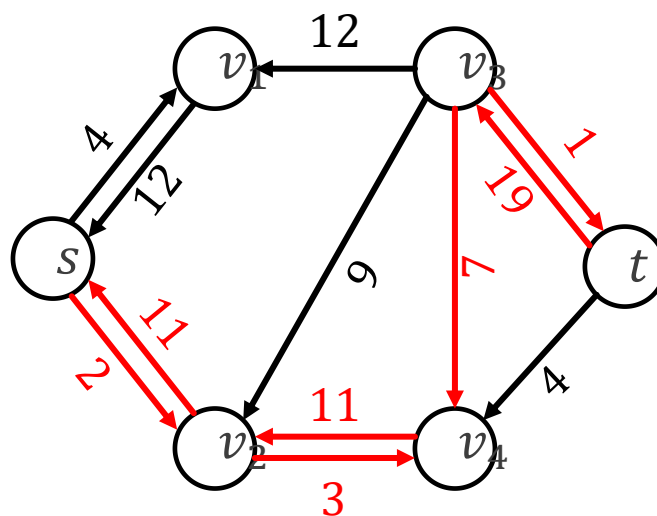


Example

Update G_f



G

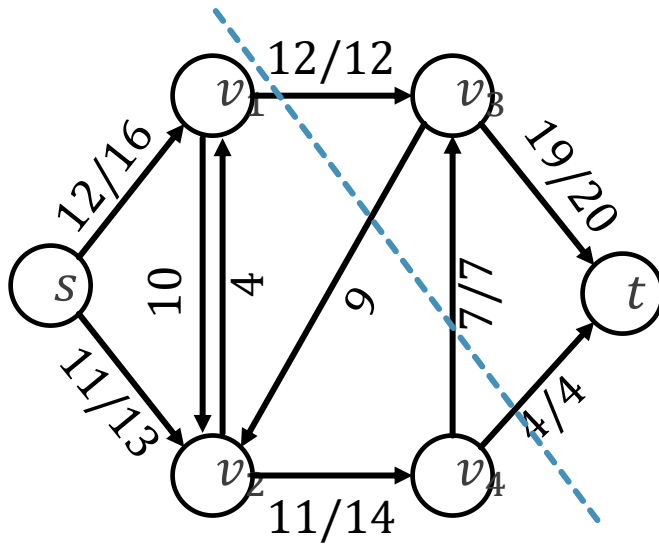


G_f

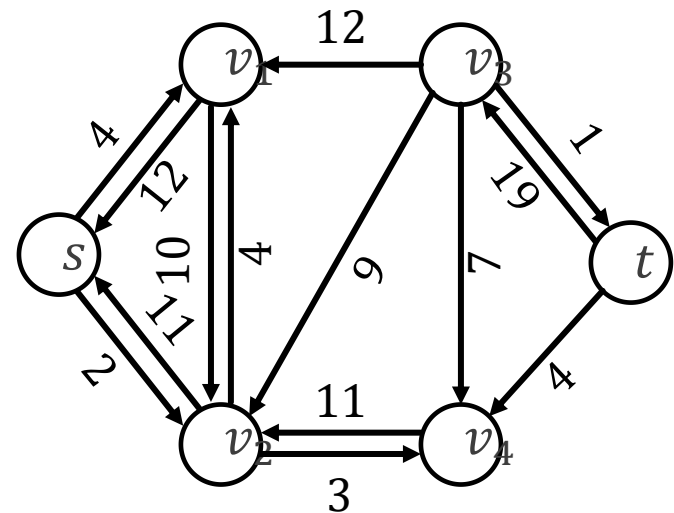


Example

Can't find an augmenting path and $|f| = c(S, T) = 23$ for some cut (S, T) .



G



G_f



Ford-Fulkerson Method

FordFulkerson(G, s, t, c)

1 **for** each edge $(u, v) \in E$ **do**

2 $f(u, v) \leftarrow 0$

3 $G_f \leftarrow G$

4 **while** there is an augmenting path p in G_f **do**

5 Let $c_f(p)$ be the bottleneck capacity of p

6 **for** each edge (u, v) in p **do**

7 $f(u, v) \leftarrow f(u, v) + c_f(p)$

8 Update the residual graph G_f

Total:

$O(|E||f^*|)$

$O(|E|)$

$O(|f^*|)$

$O(|E|)$

$O(|E|)$

$O(|E|)$

- Each time we can find a augmenting path which can at least increase the flow value by 1. Therefore, at most f^* iterations in while loop.
 - f^* is the maximum flow value that the method can find.



Method vs. Algorithm

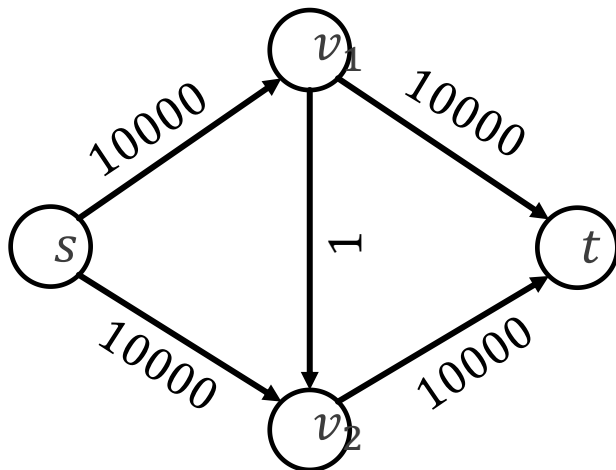
- We call it a “method” rather than an “algorithm” because the way to find the augmented path is not identified.
- Different implementations have differing running times.

```
FordFulkerson( $G, s, t, c$ )
1  for each edge  $(u, v) \in E$  do
2     $f(u, v) \leftarrow 0$ 
3   $G_f \leftarrow G$ 
4  while there is an augmenting path  $p$  in  $G_f$  do
5    Let  $c_f(p)$  be the bottleneck capacity of  $p$ 
6    for each edge  $(u, v)$  in  $p$  do
7       $f(u, v) \leftarrow f(u, v) + c_f(p)$ 
8    Update the residual graph  $G_f$ 
```

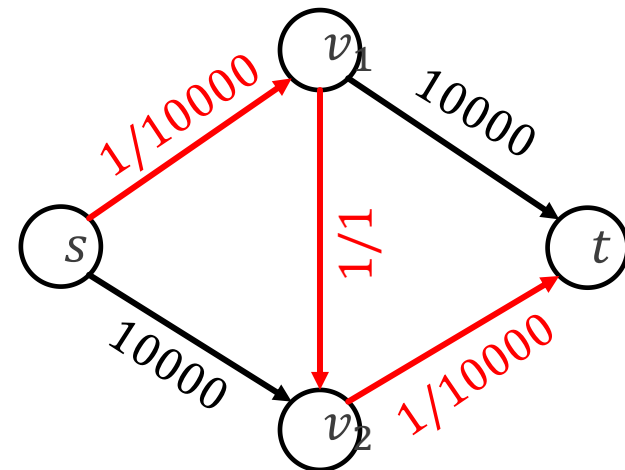


Example

Find an augmenting path p and bottleneck capacity



G

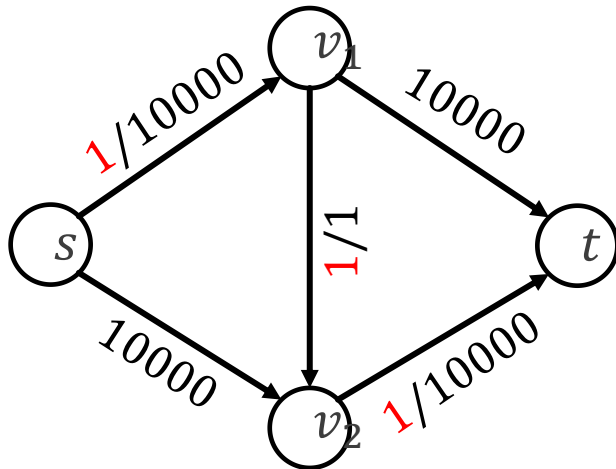


G_f

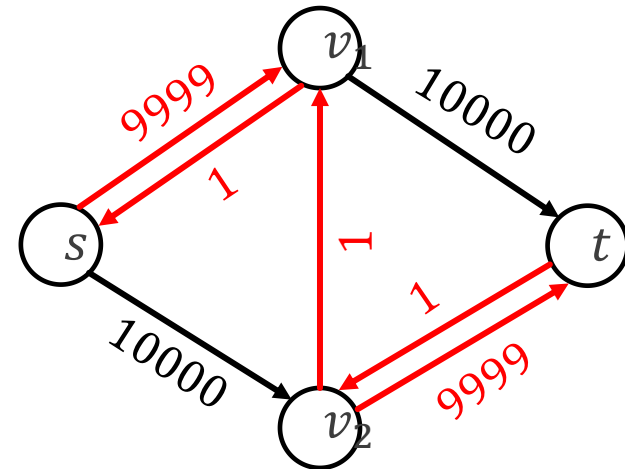


Example

Add the flow f_p to f and update G_f



G

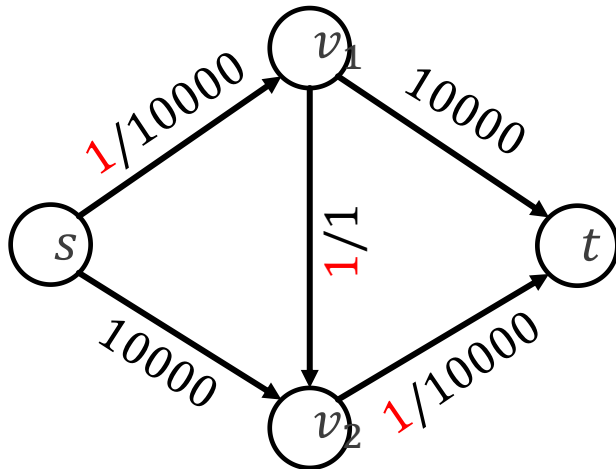


G_f

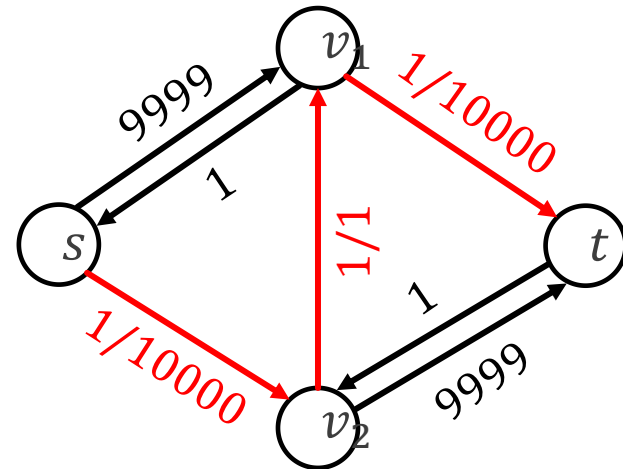


Example

Find an augmenting path p and bottleneck capacity



G

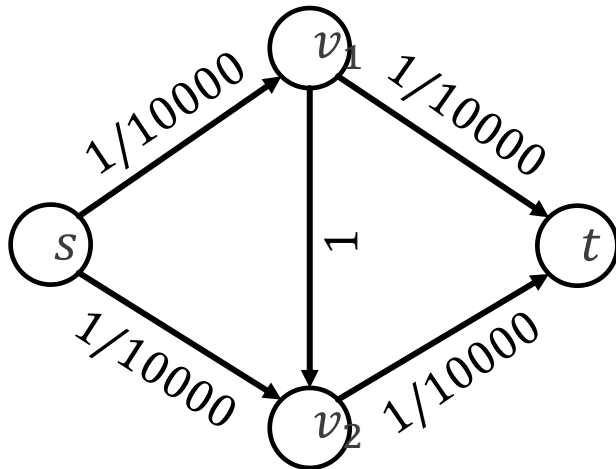


G_f

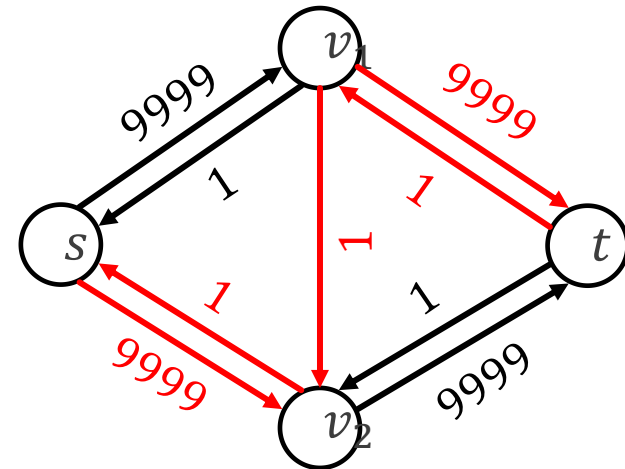


Example

Add the flow f_p to f and update G_f



G



G_f

Running time: Execute 20000 times...



MAX-FLOW PROBLEM

SHORTEST PATH AUGMENTATION ALGORITHM

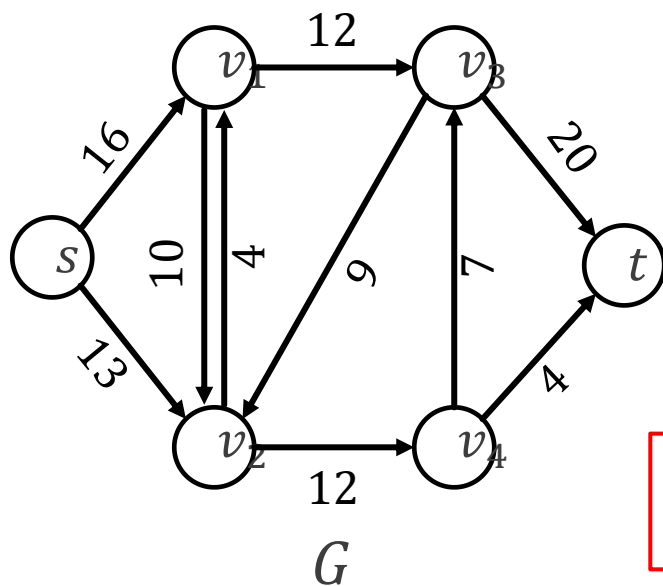


Shortest Path Augmentation Algorithm

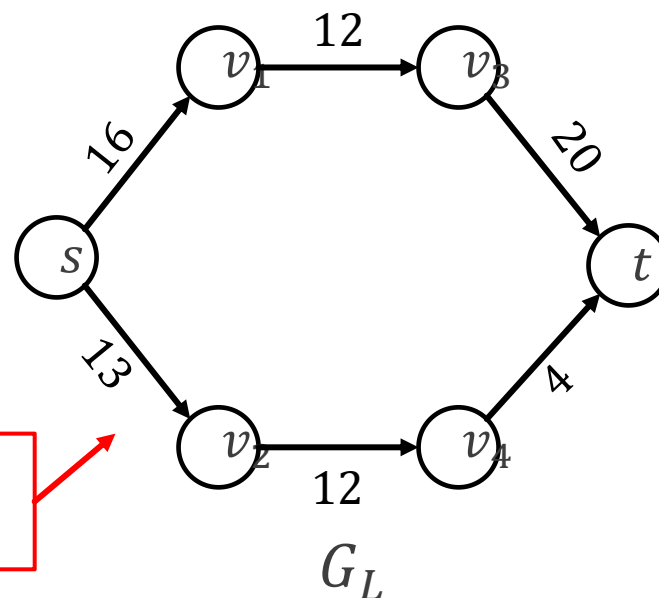
Definition 9.8

The **level (层次)** of a vertex v , denoted by $d(v)$, is **the least number of edges** in a path from s to v .

Given a directed graph $G = (V, E)$, the **level graph (层次图)** is defined as $G_L = (V, E')$, where $E' = \{(u, v) : d(v) = d(u) + 1\}$.



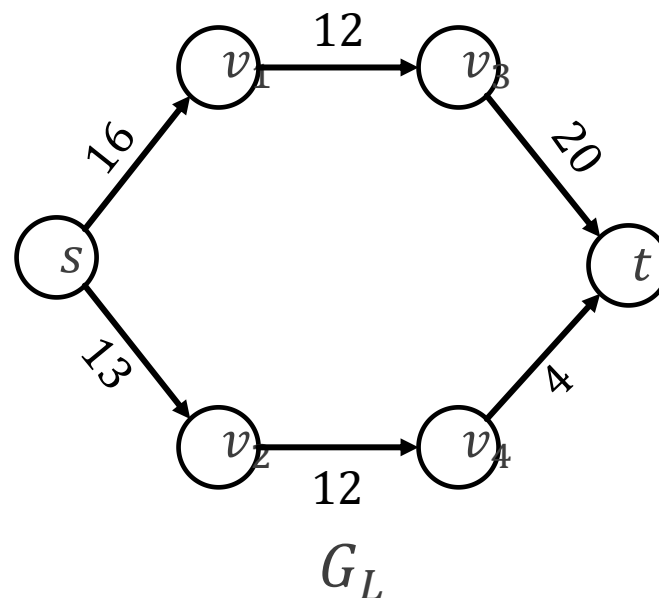
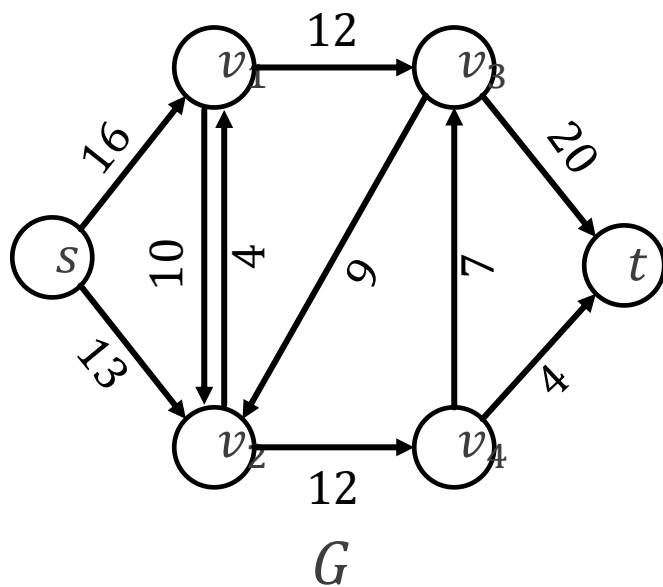
Only contain
shortest paths





Shortest Path Augmentation Algorithm

- The **level graph** can be easily obtained by **BFS**.
- Can DFS do this?
 - No, DFS doesn't give the shortest unless each time we compare $d[v]$.





Shortest Path Augmentation Algorithm

- This algorithm is also called **Edmonds-Karp algorithm**.
- Idea: Select an augmenting path of minimum length and increases the current flow by its bottleneck capacity.
- Generally there are two steps.
 1. **Compute** the level graph G_L from the residual graph G_f . If t is not in G_L , then halt: otherwise continue.
 - t not in G_L means there's no path from s to t in G_f .
 2. As long as there is **a path p** from s to t in G_L , **augment the current flow** by p , and **update G_L and G_f** accordingly.



Shortest Path Augmentation Algorithm

ShortestPathAugmentation(G, s, t)

- 1 **for** each edge $(u, v) \in E$ **do**
- 2 $f(u, v) \leftarrow 0$
- 3 $G_f \leftarrow G$
- 4 find the level graph G_L of G_f
- 5 **while** t is a vertex in G_L **do**
- 6 **while** there is a path p from s to t in G_L **do**
- 7 let $c_f(p)$ be the bottleneck capacity on p
- 8 augment the current flow f by $c_f(p)$
- 9 update G_L and G_f along the path p
- 10 use G_f to compute a new level graph G_L

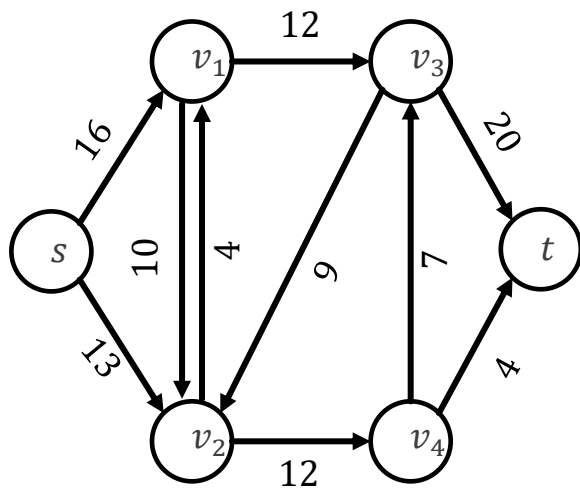
There may be multiple shortest paths in G_L from s to t .



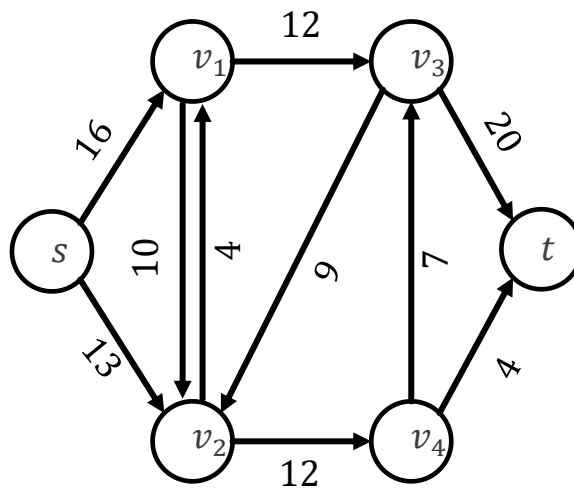


Example

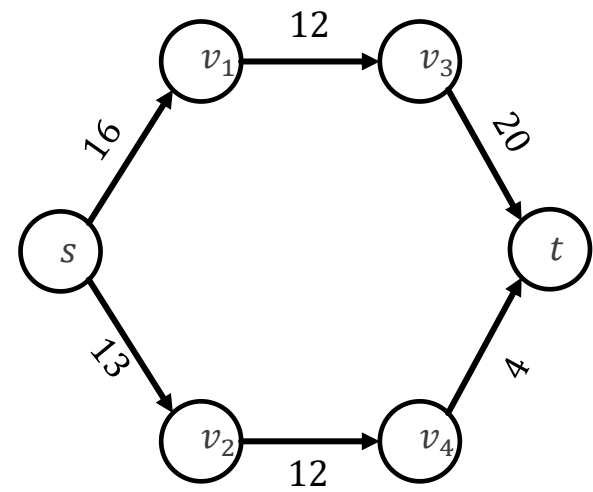
Initialization



G



G_f

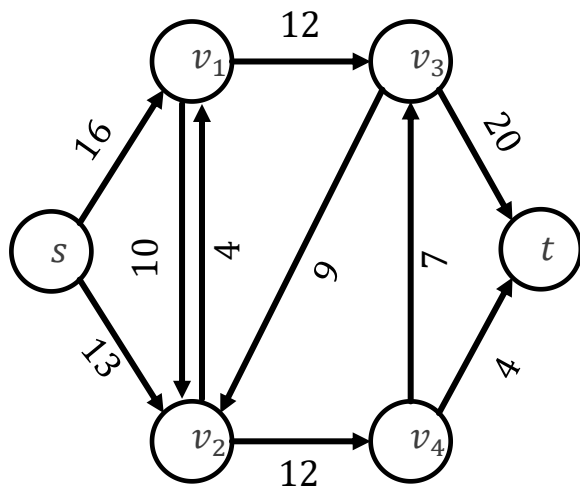


G_L

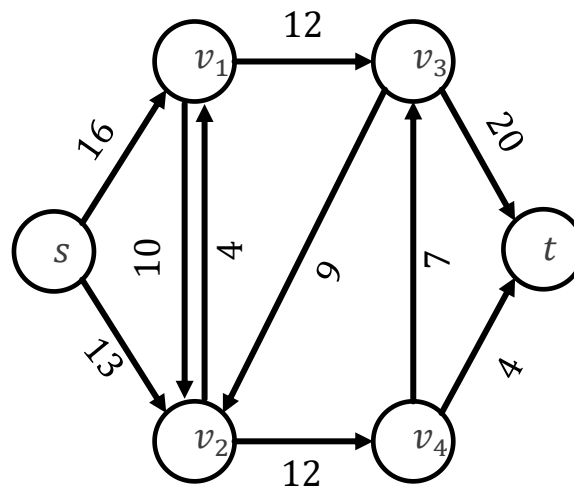


Example

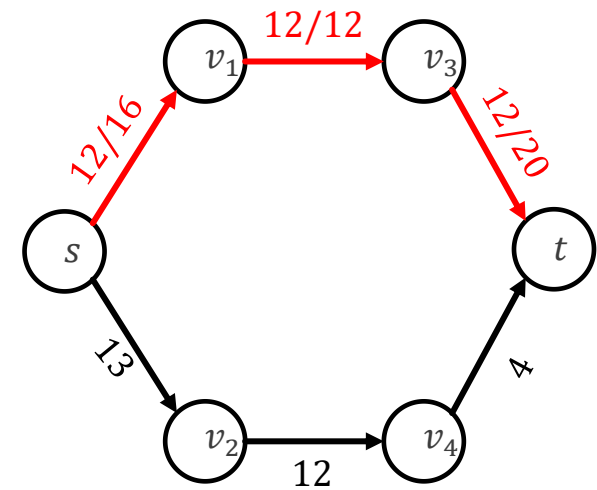
Find an augmenting path p and bottleneck capacity in G_L



G



G_f

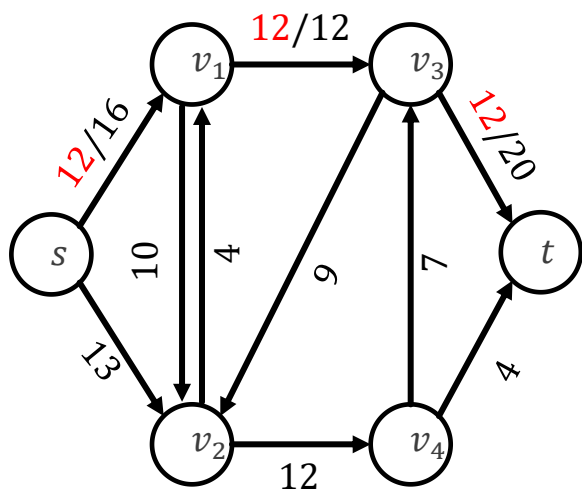


G_L

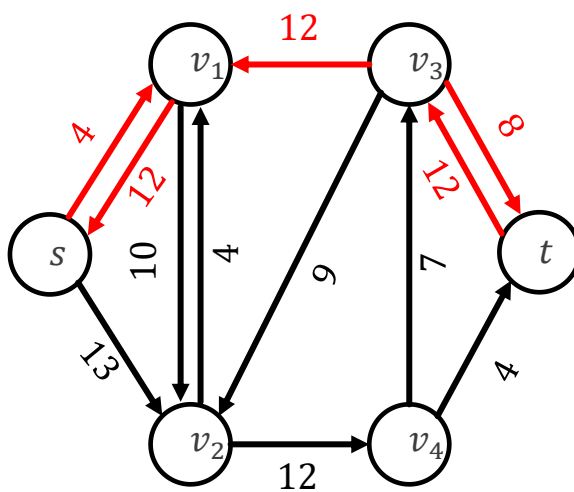


Example

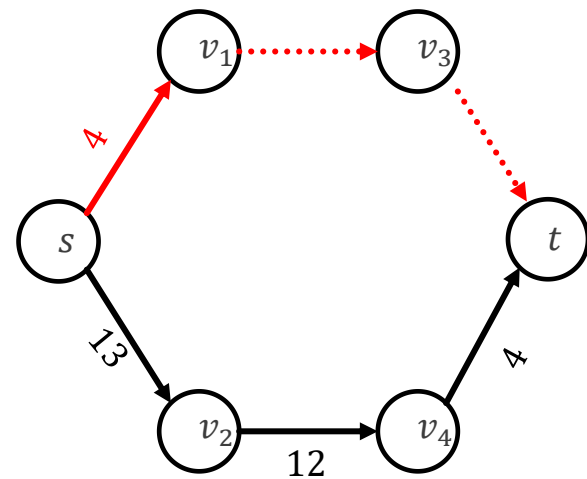
Add the flow f_p to f and update G_f and G_L



G



G_f

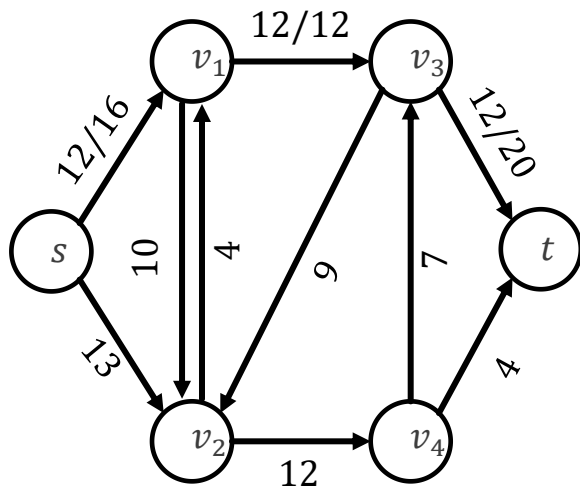


G_L

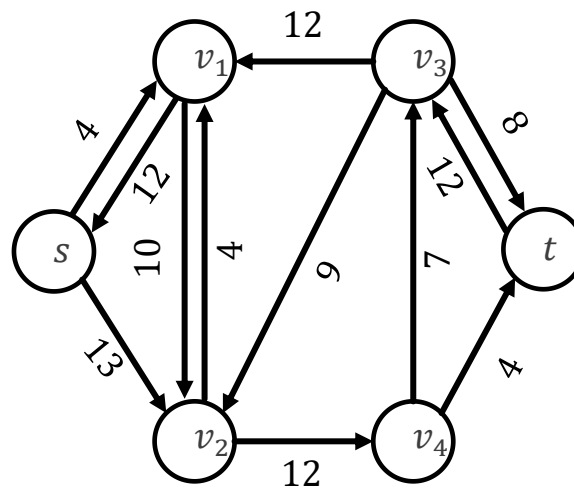


Example

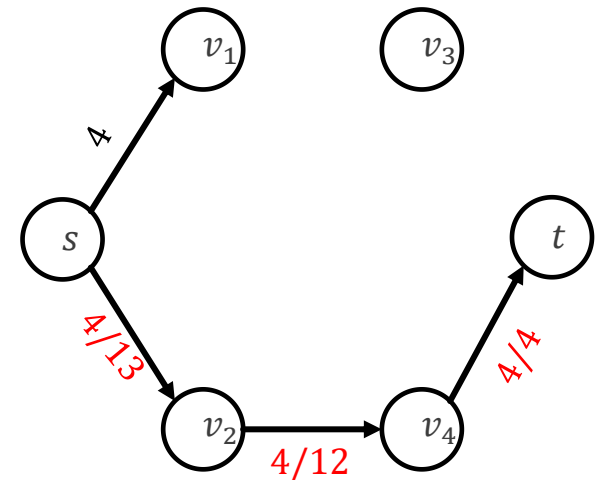
Find an augmenting path p and bottleneck capacity in G_L



G



G_f

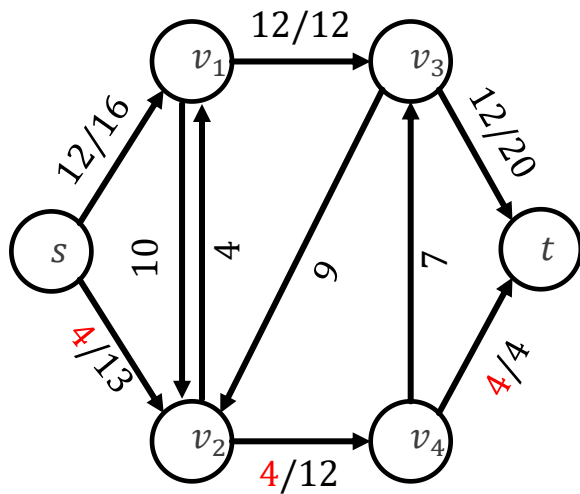


G_L

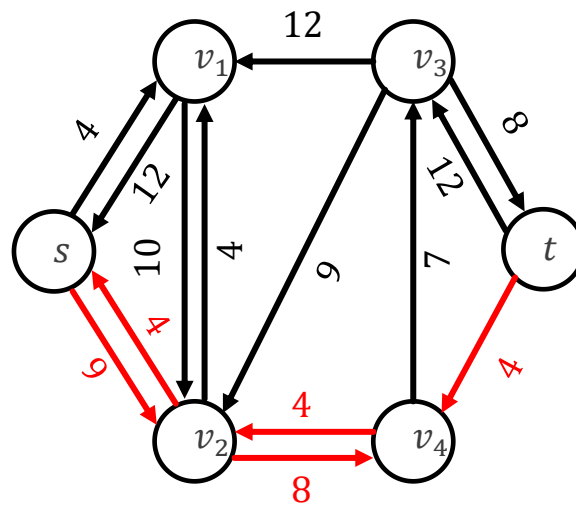


Example

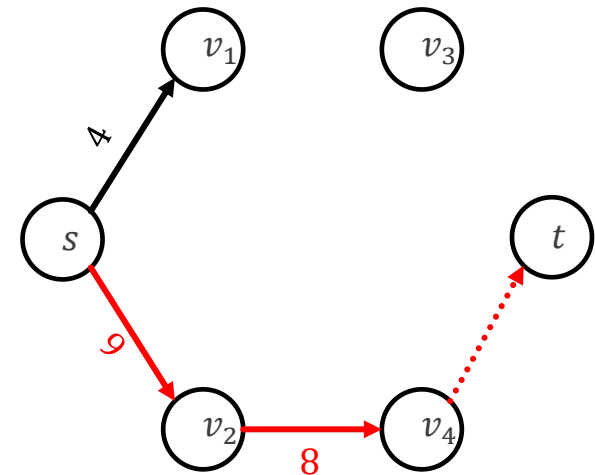
Add the flow f_p to f and update G_f and G_L



G



G_f

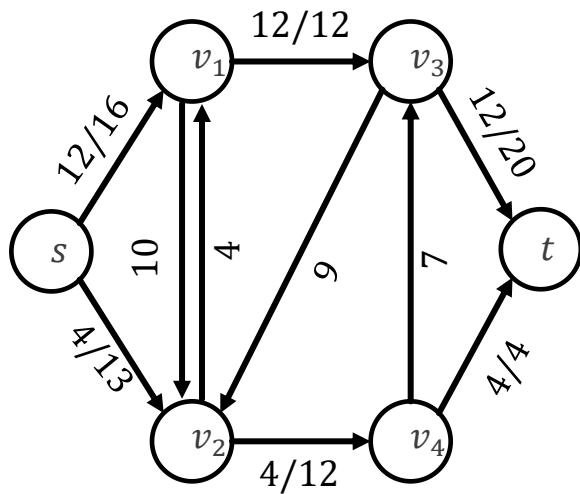


G_L

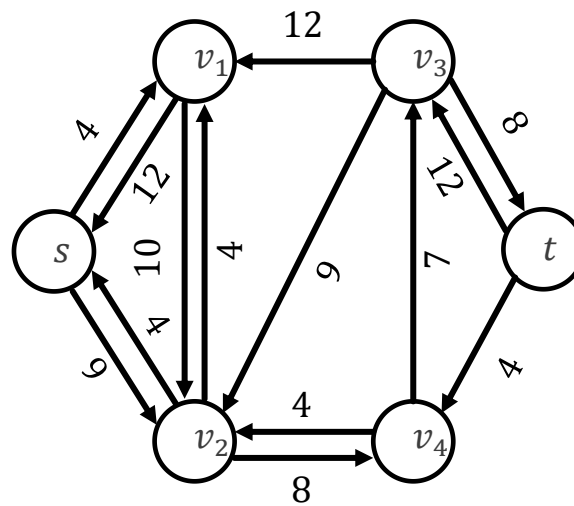


Example

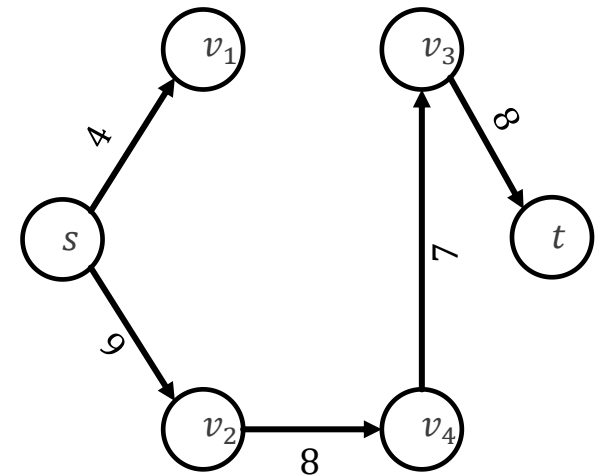
No path from s to t in G_L , use G_f to compute a new G_L



G



G_f

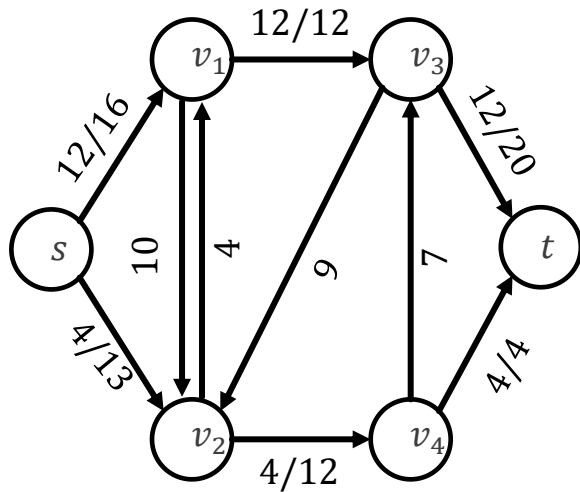


G_L

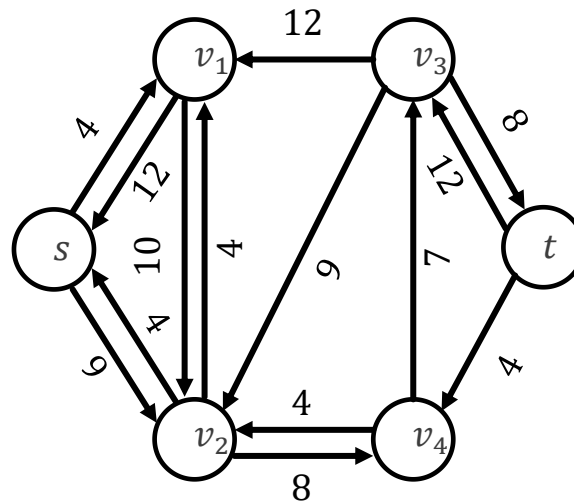


Example

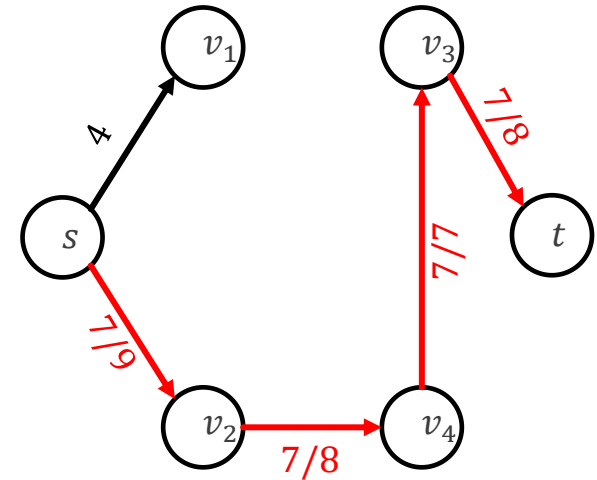
Find an augmenting path p and bottleneck capacity in G_L



G



G_f

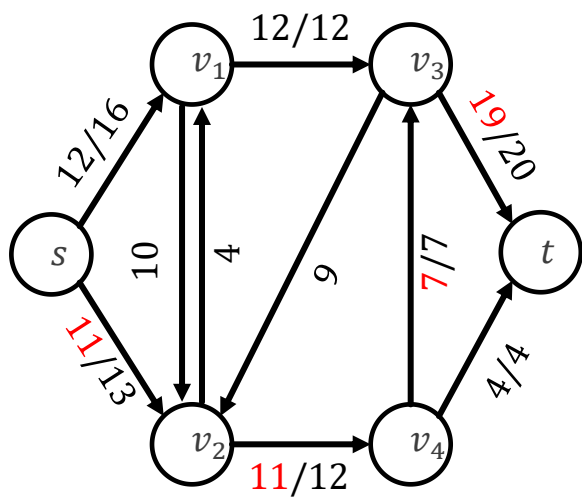


G_L

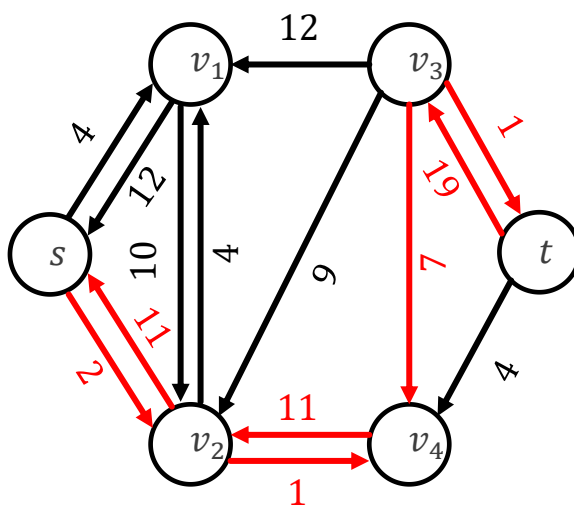


Example

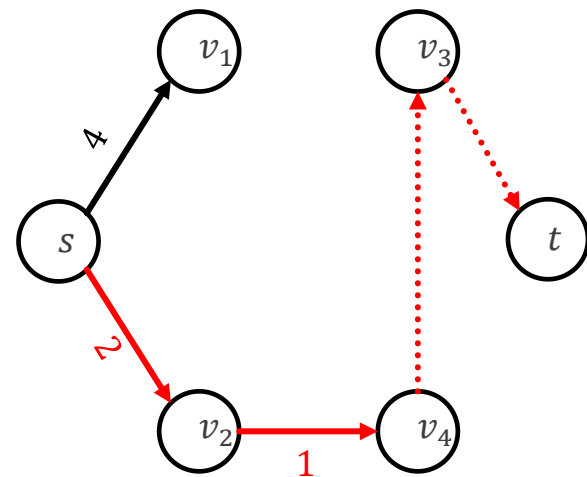
Add the flow f_p to f and update G_f and G_L



G



G_f

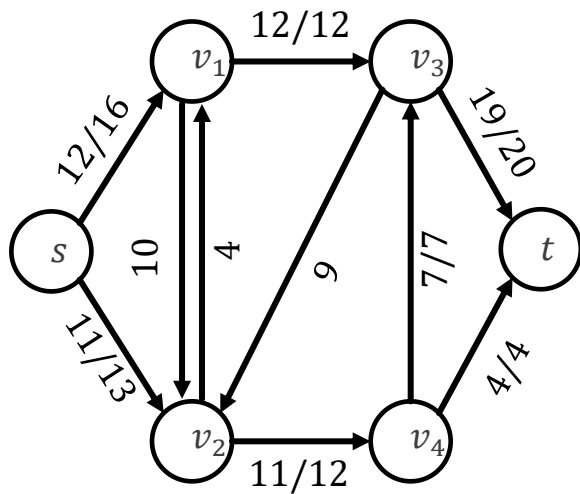


G_L

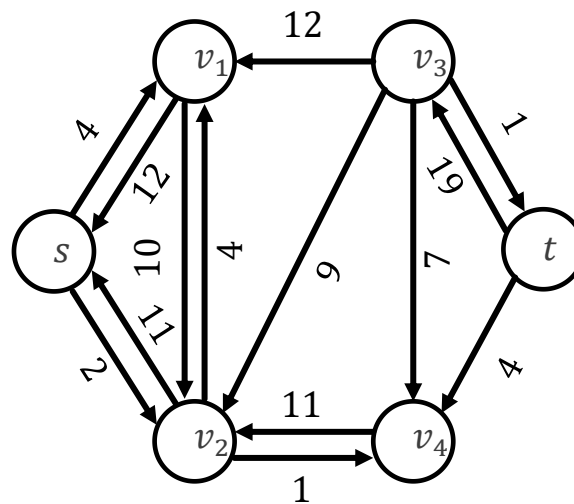


Example

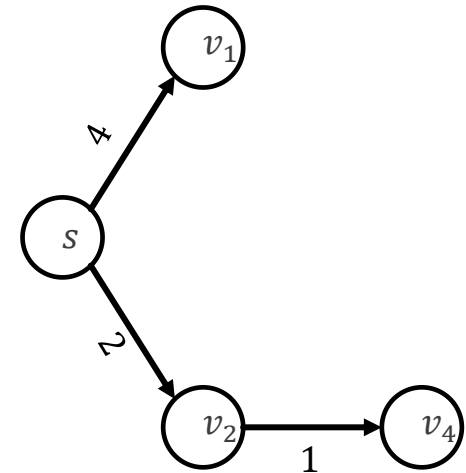
No path from s to t in G_L , use G_f to compute a new G_L



G



G_f



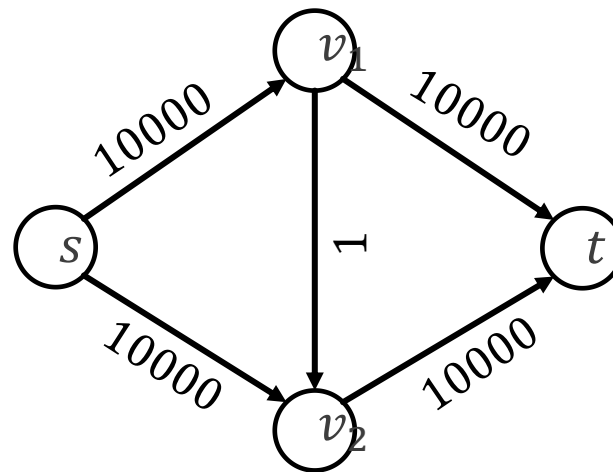
G_L

t is not a vertex in G_L . Algorithm terminates.



Example

- Now, look back at this example again...



G



Analysis of Shortest Path Augmentation Algorithm

Lemma 9.4

The while loop in Line 5 in ShortestPathAugmentation is executed at most $|V|$ times.

Proof:

- We show that **the number of level graph** computed using the algorithm is **at most $|V|$** .

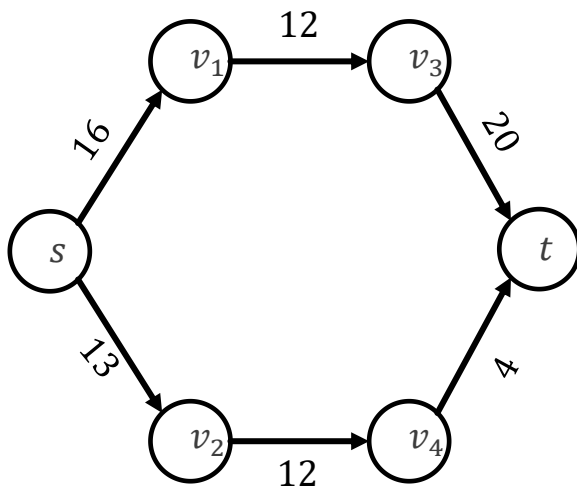
```
5 while  $t$  is a vertex in  $G_L$  do  
6   while there is a path  $p$  from  $s$  to  $t$  in  $G_L$  do  
7     let  $c_f(p)$  be the bottleneck capacity on  $p$   
8     augment the current flow  $f$  by  $c_f(p)$   
9     update  $G_L$  and  $G_f$  along the path  $p$   
10    use  $G_f$  to compute a new level graph  $G_L$ 
```



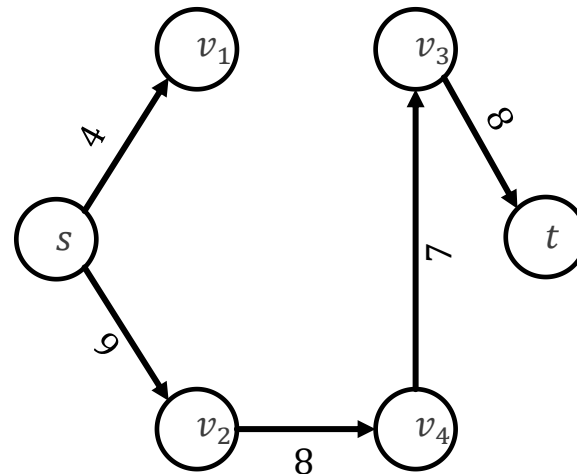
Analysis of Shortest Path Augmentation Algorithm

Proof (cont'd):

- First, we show that the sequence of **lengths** of augmenting paths using ShortestPathAugmentation is **strictly increasing**.



Length of augmenting path of the first G_L : 3



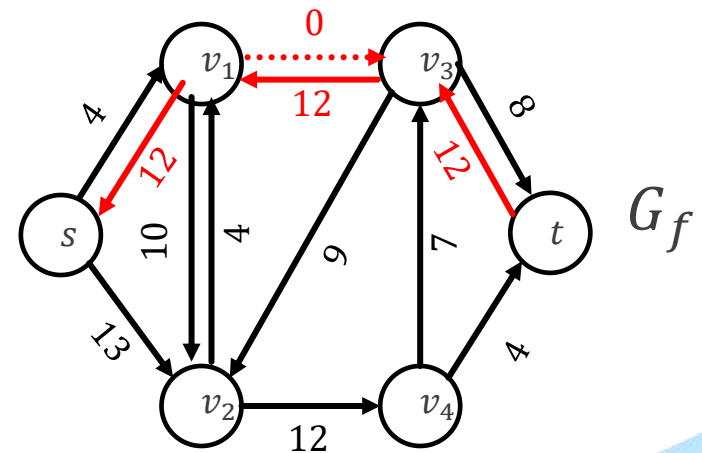
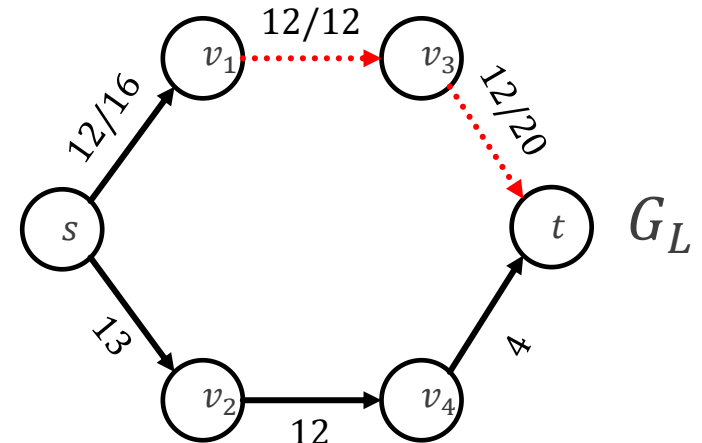
Length of augmenting path of the second G_L : 4



Analysis of Shortest Path Augmentation Algorithm

Proof (cont'd):

- Let p be any augmenting path in the current G_L .
- After augmenting using p , **at least one edge** will be saturated and will **disappear** in G_f .
- At most $|p|$ new edges will appear in G_f , but they are back edges, and hence will not contribute to a shortest path from s to t .
- When all shortest paths are cut in the current G_L , BFS will find **strict longer shortest paths** for the next G_L . 同一个 G_L 下, 同样长度的最短路径可能有很多条。将这些最短路径全部去掉之后, 汇点 t 就没有可以到达的路径了, 那么就重新基于 G_f 生成一个新的 G_L 。新的 G_L 里面最短路径会更长





Proof (cont'd):

- When t is no longer in G_L , we can't find an augmenting path any more. 当 t 不在 G_L 中，我们不可能找到增广路径
- Since the length of any augmenting path is between 1 and $|V| - 1$, the number of level graphs use for augmentations is at most $|V| - 1$.
- Since one more level graph is computed in which t does not appear, the total number of level graphs computed is at most $|V|$. 总的计算层次图的次数不超过 $|V|$



Analysis of Shortest Path Augmentation Algorithm

Theorem 9.2

Given a flow network $G = (V, E)$, the algorithm finds a maximum flow in $O(|V||E|^2)$.

Proof:

- Given a G_L , there are **at most $|E|$ augmenting paths** of the same length.
- By Lemma 9.4, the number of augmenting steps is at most $|V||E|$.
- Computing each level graph takes $O(|E|)$ using BFS.
- Therefore, the total time required to compute all level graphs is $O((|V||E|^2))$.

ShortestPathAugmentation(G, s, t)

```
1 for each edge  $(u, v) \in E$  do
2    $f(u, v) \leftarrow 0$ 
3  $G_f \leftarrow G$ 
4 find the level graph  $G_L$  of  $G_f$ 
5 while  $t$  is a vertex in  $G_L$  do
6   while there is a path  $p$  from  $s$  to  $t$  in  $G_L$  do
7     let  $c_f(p)$  be the bottleneck capacity on  $p$ 
8     augment the current flow  $f$  by  $c_f(p)$ 
9     update  $G_L$  and  $G_f$  along the path  $p$ 
10  use  $G_f$  to compute a new level graph  $G_L$ 
```



Published in 1956

不断有人提出更快的最大流算法

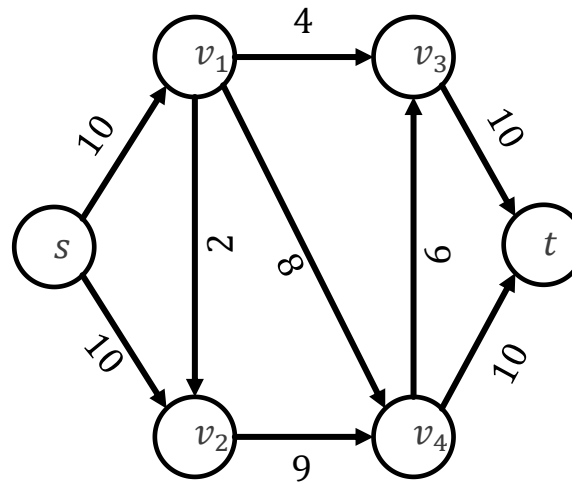
Method	Complexity	Description
Linear programming		Constraints given by the definition of a legal flow . See the linear program here.
Ford–Fulkerson algorithm	$O(E f_{\max})$	As long as there is an open path through the residual graph, send the minimum of the residual capacities on the path. The algorithm is only guaranteed to terminate if all weights are rational , in which case the amount added to the flow in each step is at least the greatest common divisor of the weights. Otherwise it is possible that the algorithm will not converge to the maximum value. However, if the algorithm terminates, it is guaranteed to find the maximum value.
Edmonds–Karp algorithm	$O(VE^2)$	A specialization of Ford–Fulkerson, finding augmenting paths with breadth-first search .
Dinic's algorithm	$O(V^2E)$	In each phase the algorithms builds a layered graph with breadth-first search on the residual graph . The maximum flow in a layered graph can be calculated in $O(VE)$ time, and the maximum number of phases is $V - 1$. In networks with unit capacities, Dinic's algorithm terminates in $O(\min\{V^{2/3}, E^{1/2}\}E)$ time. ^[citation needed]
MKM (Malhotra, Kumar, Maheshwari) algorithm ^[10]	$O(V^3)$	A modification of Dinic's algorithm with a different approach to constructing blocking flows. Refer to the original paper .
Dinic's algorithm with dynamic trees	$O(VE \log V)$	The dynamic trees data structure speeds up the maximum flow computation in the layered graph to $O(VE \log V)$.
General push–relabel algorithm ^[11]	$O(V^2E)$	The push relabel algorithm maintains a preflow, i.e. a flow function with the possibility of excess in the vertices. The algorithm runs while there is a vertex with positive excess, i.e. an active vertex in the graph. The push operation increases the flow on a residual edge, and a height function on the vertices controls through which residual edges can flow be pushed. The height function is changed by the relabel operation. The proper definitions of these operations guarantee that the resulting flow function is a maximum flow.
Push–relabel algorithm with FIFO vertex selection rule ^[11]	$O(V^3)$	Push-relabel algorithm variant which always selects the most recently active vertex, and performs push operations while the excess is positive and there are admissible residual edges from this vertex.
Push-relabel algorithm with dynamic trees ^[11]	$O\left(VE \log \frac{V^2}{E}\right)$	The algorithm builds limited size trees on the residual graph regarding to the height function. These trees provide multilevel push operations, i.e. pushing along an entire saturating <i>path</i> instead of a single edge.
KRT (King, Rao, Tarjan)'s algorithm ^[12]	$O\left(VE \log \frac{E}{V \log V} V\right)$	
Binary blocking flow algorithm ^[13]	$O\left(E \cdot \min\{V^{2/3}, E^{1/2}\} \cdot \log \frac{V^2}{E} \cdot \log U\right)$	The value U corresponds to the maximum capacity of the network.
James B Orlin's + KRT (King, Rao, Tarjan)'s algorithm ^[9]	$O(VE)$	Orlin's algorithm solves max-flow in $O(VE)$ time for $E \leq O(V^{\frac{16}{15}-\epsilon})$ while KRT solves it in $O(VE)$ for $E > V^{1+\epsilon}$.
Kathuria-Liu-Sidford algorithm ^[14]	$E^{4/3+o(1)} U^{1/3}$	Interior point methods and edge boosting using ℓ_p -norm flows. Builds on earlier algorithm of Madry, which achieved runtime $\tilde{O}(m^{10/7} U^{1/7})$. ^[15]
BLNPSSW / BLLSSSW algorithm ^[16] [17]	$\tilde{O}((E + V^{3/2}) \log U)$	Interior point methods and dynamic maintenance of electric flows with expander decompositions.
Gao-Li-Peng algorithm ^[18]	$\tilde{O}(E^{\frac{3}{2} - \frac{1}{328}} \log U)$	Gao, Liu, and Peng's algorithm revolves around dynamically maintaining the augmenting electrical flows at the core of the interior point method based algorithm from [Madry JA 2014, '16]. This entails designing data structures that, in limited settings, return edges with large electric energy in a graph undergoing resistance updates.

Jan 2021



Classroom Exercise

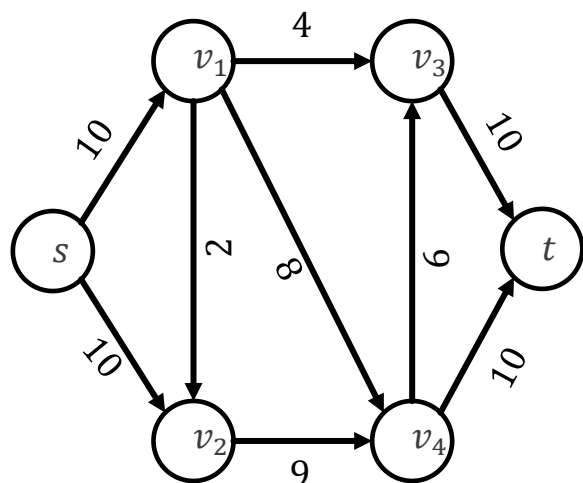
- Use ShortestPathAugmentation to find the max-flow of the following graph.



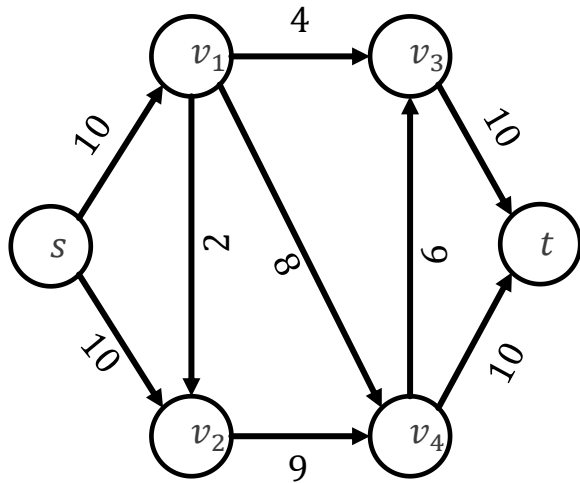


Classroom Exercise (课堂练习)

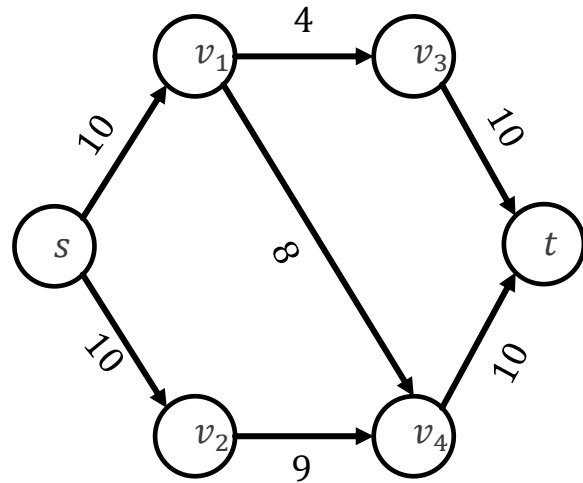
$$|f| = 0$$



G



G_f



G_L

当前 G_L 有3条最短路径 (增广路径)

S, v_1, v_3, t 瓶颈容量4, 并更改对应的 G_f 和 G_L

S, v_2, v_4, t 瓶颈容量9, 并更改对应的 G_f 和 G_L

S, v_1, v_4, t 瓶颈容量1, 并更改对应的 G_f 和 G_L , 此时 t 已经没有路径相连了,

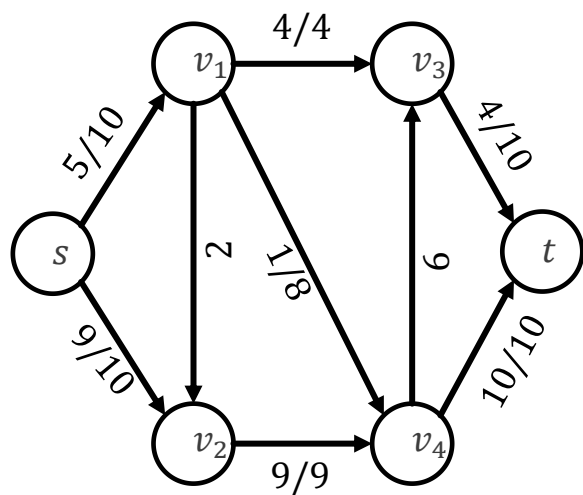
必须重新计算一个新的 G_L

选择路径的顺序不同, 会导致不同的流

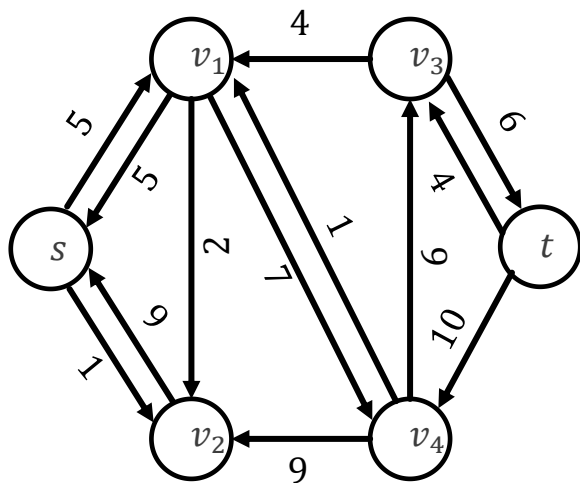


Classroom Exercise

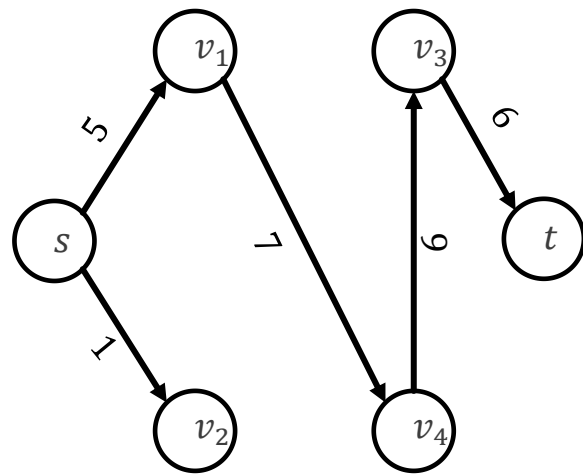
$$|f| = 14$$



G



G_f



G_L

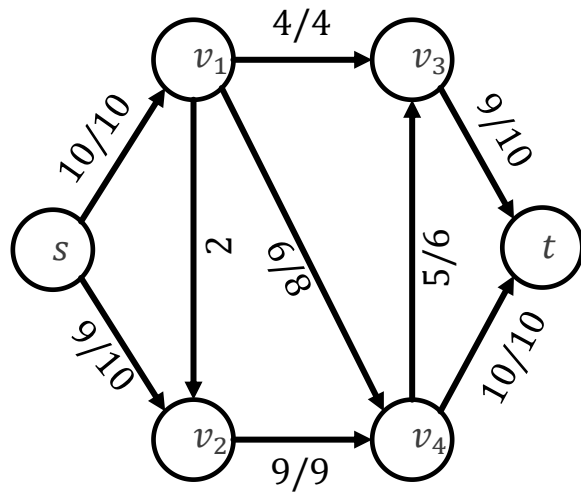
增广路径

s, v_1, v_4, v_3, t 瓶颈容量5

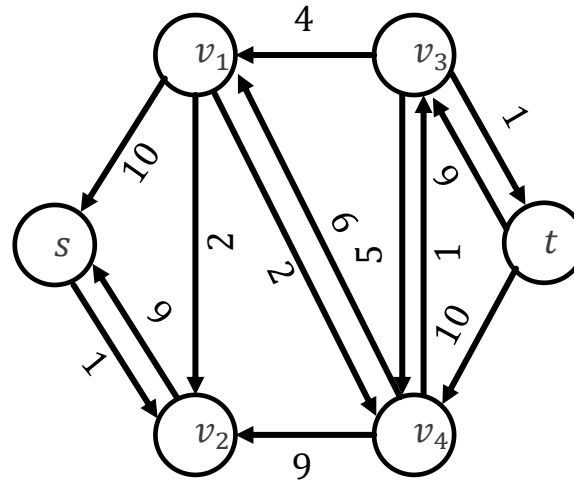


Classroom Exercise

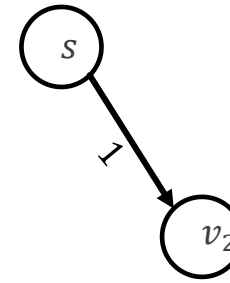
$$|f| = 19$$



G



G_f



G_L



MATCHING PROBLEM

最小费用流问题



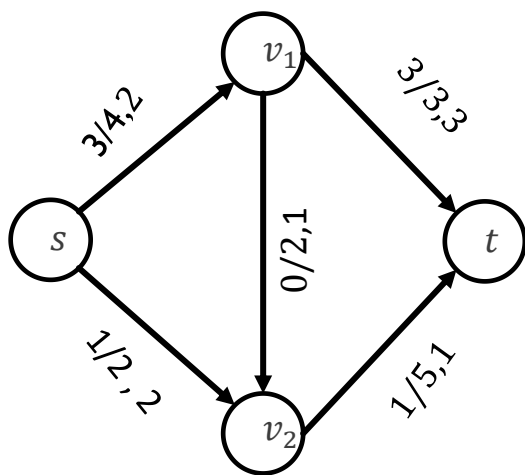
最小费用流

- Given network $G = (V, E)$, $w: V \times V \rightarrow R$ is the **weight**, find the minimal cost for flow F

- 费用 $w(f) = \sum_{(u,v) \in f} w(u,v)f(u,v)$

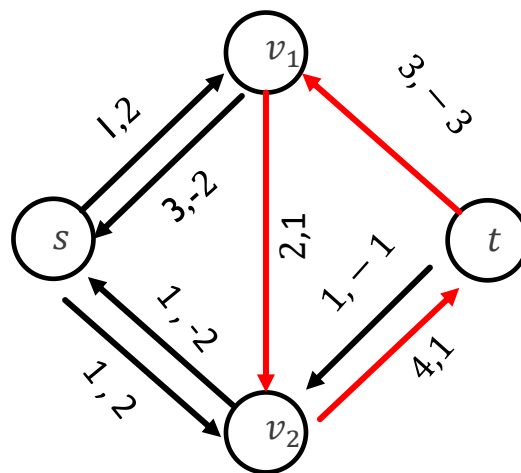
最大流问题，最短路径问题
是最小费用流问题的特例

费用流网络 G



我们这里只研究在流量固定的情况下，费用最小的流 $|F|$

费用流剩余网络 G_f



$$c_f(u, v) = c(u, v) - f(u, v)$$
$$w(v, u) = -w(u, v)$$



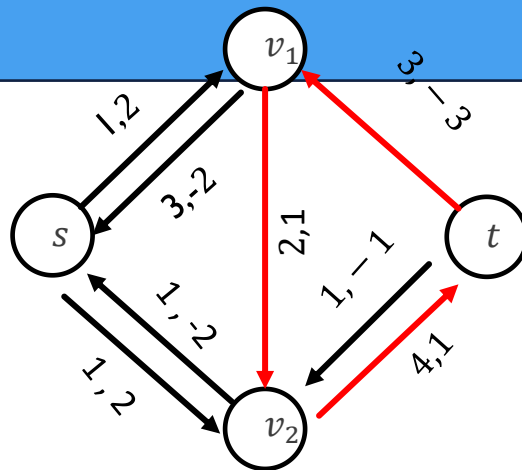
消除回路算法

- 通过不停的在剩余网络中寻找增广回路，达到在保持流量不变的情况下，降低费用
- **定义9.12** 给定一个网络 $G = (V, E)$, 对应剩余网络的增广回路(Cycle)是一个增广路径 p ， p 的起点和终点相同
 - 回路中每条边的剩余容量大于0
 - 增广回路 p 的费用为 $w(p) = \sum_{(u,v) \in p} w(u, v)$



消除回路算法

- 费用为负的增广回路见右图



- 引理 9.5 设 p 为剩余网络 G_f 的一条增广回路，则存在流量为 $|f|$ 的流 f' ，且其费用为

$$w(f') = w(f) + w(p)c_f(p)$$

- 定理 9.5 流 f 为最小费用流当且仅当剩余网络 G_f 中不存在费用为负的增广回路
 - 这个意味着我们如果找不到费用为负的增广回路那么就找到最小费用流



CancelingCycle 消除回路算法

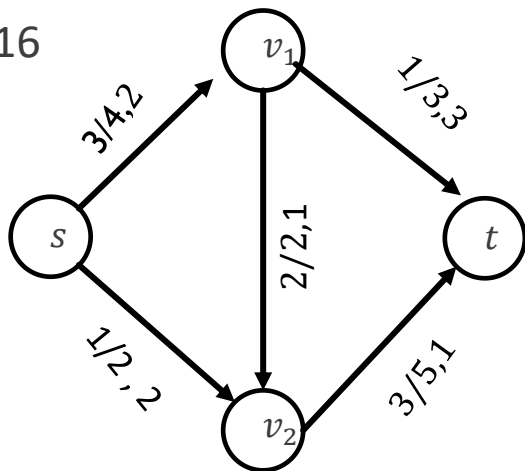
CancelingCycle(G, s, t, c, w)

1. Establish a **feasible flow** f in the network 通过某种方式找到一个流
2. **While** G_f contains a negative cycle **do** 如果剩余网络中有负回路
3. use some algorithm to **identify** a negative cycle p
4. **compute** $c_f(p)$
5. **augment** $c_f(p)$ units of flow **in the cycle** p and **update** G and G_f

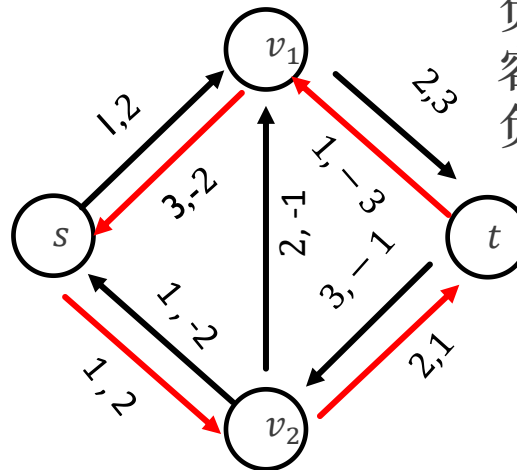


消除回路算法

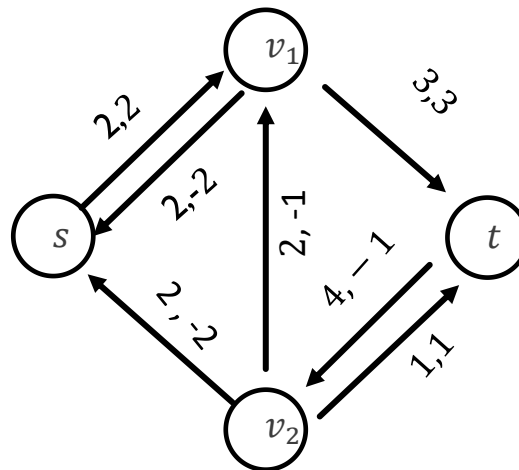
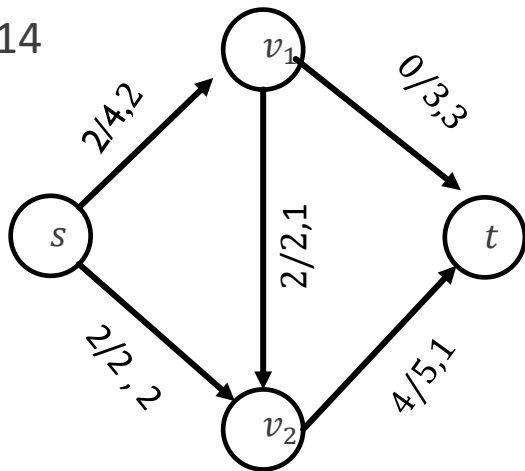
流量4
费用16



负回路
容量1
负费用-2



流量4
费用14





最小费用路算法

- 对于剩余网络 G_f 的一条 s 到 t 的有向路径 p ，它一定是原网络的增广路径，沿着 p 对流 f 进行增广，得到流量值更大的可行流 f'
 - 增广回路 p 的费用为 $w(p) = \sum_{(u,v) \in p} w(u, v)$
 - $w(f') = w(f) + w(p)c_f(p)$
 - 沿着费用最小的增广路径对当前流进行增广，即可得到最小费用流
 - 贪心算法
 - 不断算费用最小增广路径（最短路径算法），沿着费用最小增广路径增广，直到找不到增广路径为止



最小费用路算法

MinimumWeightPath(G, s, t, c, w)

1. **For** each edge (u, v) **do**
2. $f(u, v) \leftarrow 0$
3. $G_f \leftarrow G$
4. **While** there is a **minimum weight path** p in G_f **do**
5. let $c_f(p)$ be the **bottleneck capacity** on p
6. **augment** $c_f(p)$ units of flow for f
7. update G_f along the path p

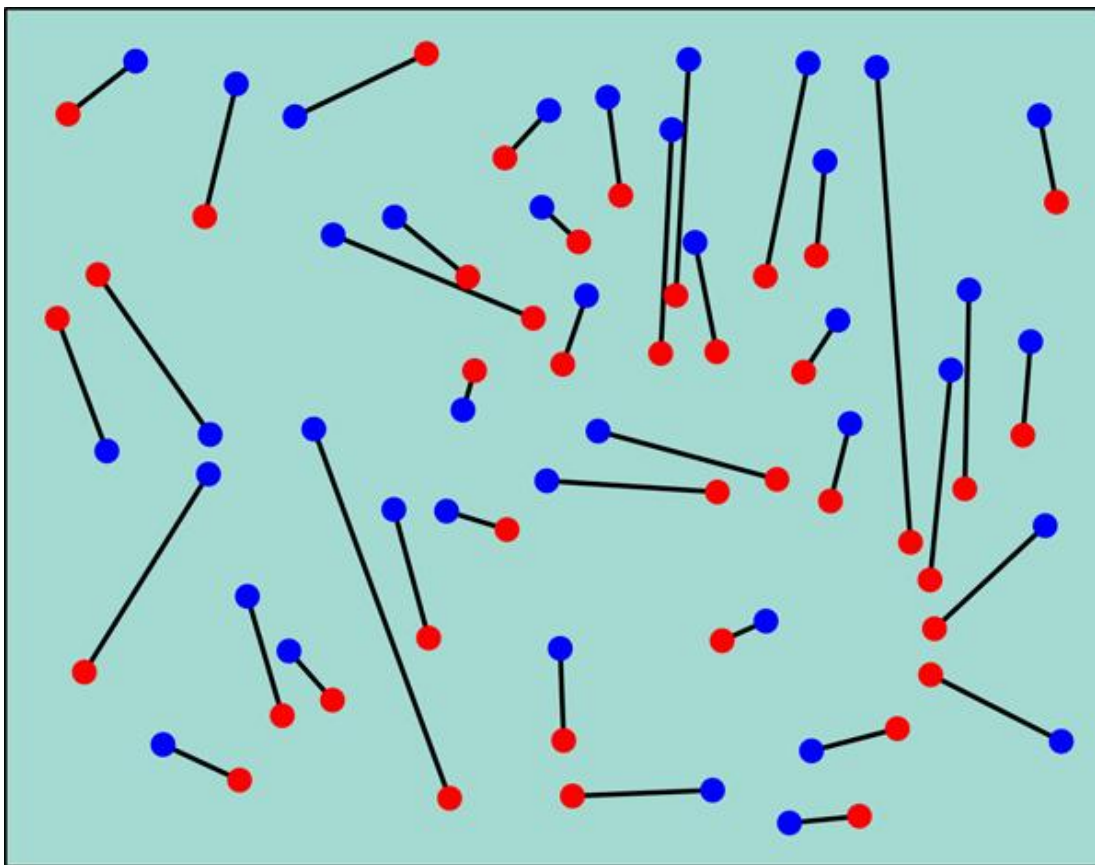


MATCHING PROBLEM

匹配问题



Matching Problem



假设图中的每个
顶点代表一个人

为顶点找到另外
一个顶点，进行
配对

保证一个点不被
两条边连起来



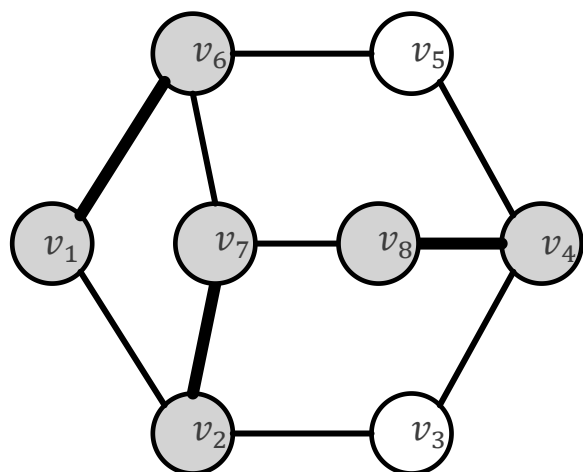
Matching Problem

Definition 9.14

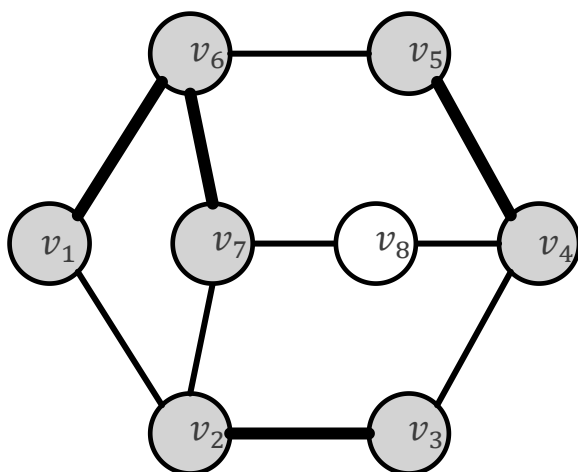
Given an **undirected** graph $G = (V, E)$, a **matching** (匹配) is a subset of edges $M \subseteq E$ such that for all vertices $v \in V$, **at most one edge of M is connected on v** .

We say that a **vertex $v \in V$ is matched by matching M** if some edge in M is **connected on v** ; otherwise, v is unmatched.

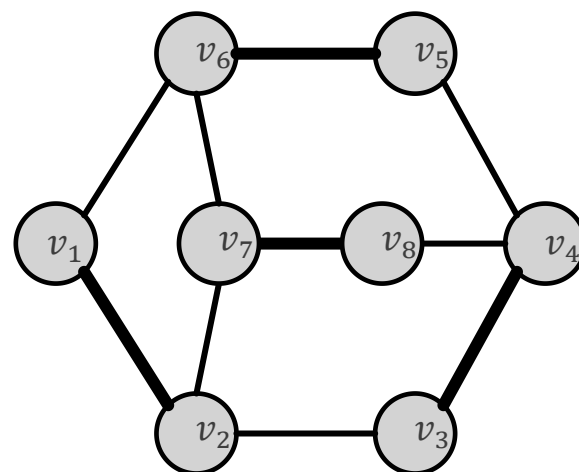
A **maximum matching** (最大匹配) is a matching of maximum cardinality. 最多的顶点匹配



A match



Not a match



A maximum match

Dark vertex
means matched

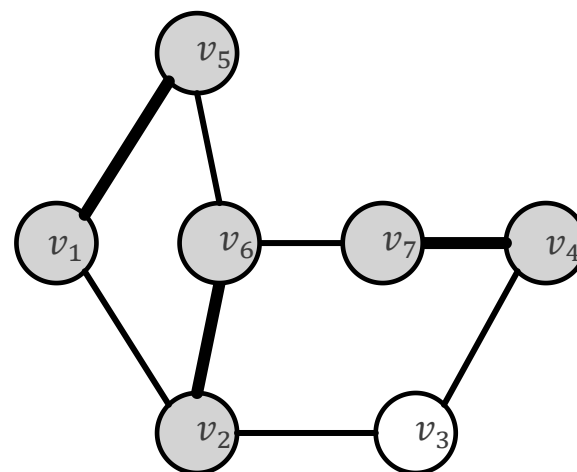


Matching Problem

Definition 9.15

Perfect matching (完美匹配) is one in which **every vertex** in V is **matched**.

- Not every undirected graph has perfect matching, e.g. graphs with odd number of vertices. 一个图有奇数个顶点没有完美匹配
- The maximum matching may not be perfect, but perfect matching must be maximum.



A maximum but not perfect matching



Matching Problem

Definition 9.16

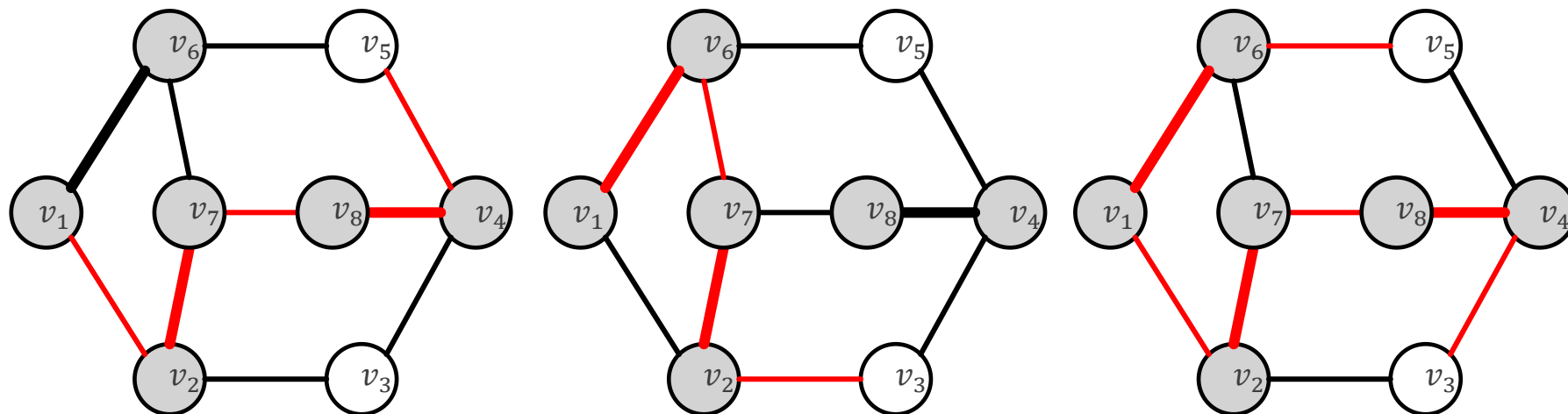
Given an **undirected graph** $G = (V, E)$ and M is a matching of G . We define:

Alternating path (交错路径): A simple path alternating between matching and non-matching edges. 匹配边和未匹配边交错的路径

Augmenting path (增广路径): A nontrivial alternating path that begins and ends with unmatched vertices. 开始和终止节点都是未匹配节点



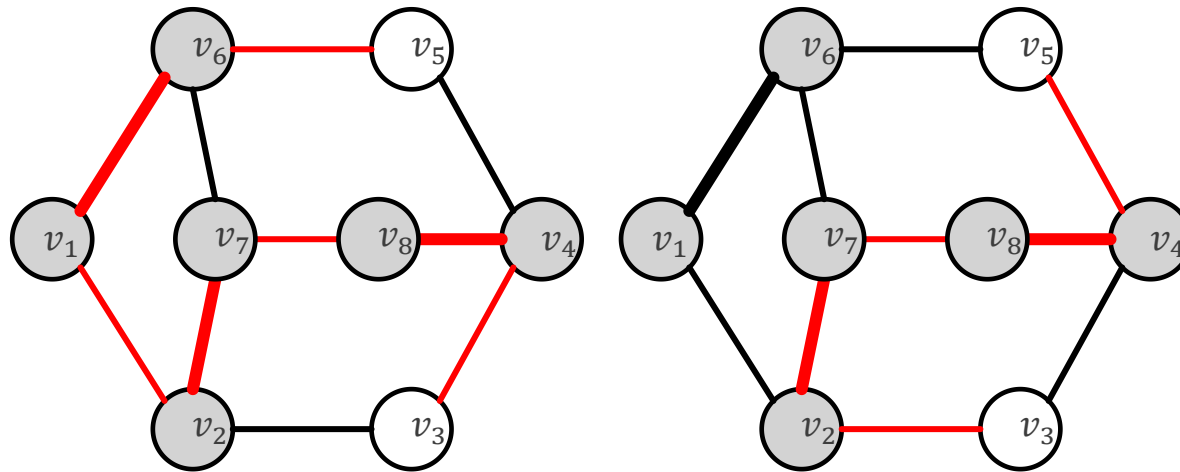
Matching Problem



Alternating path



Matching Problem



Augmenting path



Matching Problem

Definition 9.17

Let M_1 and M_2 be two matchings in a graph G , define

$$M_1 \oplus M_2 = (M_1 \cup M_2) - (M_1 \cap M_2)$$

Lemma 9.6

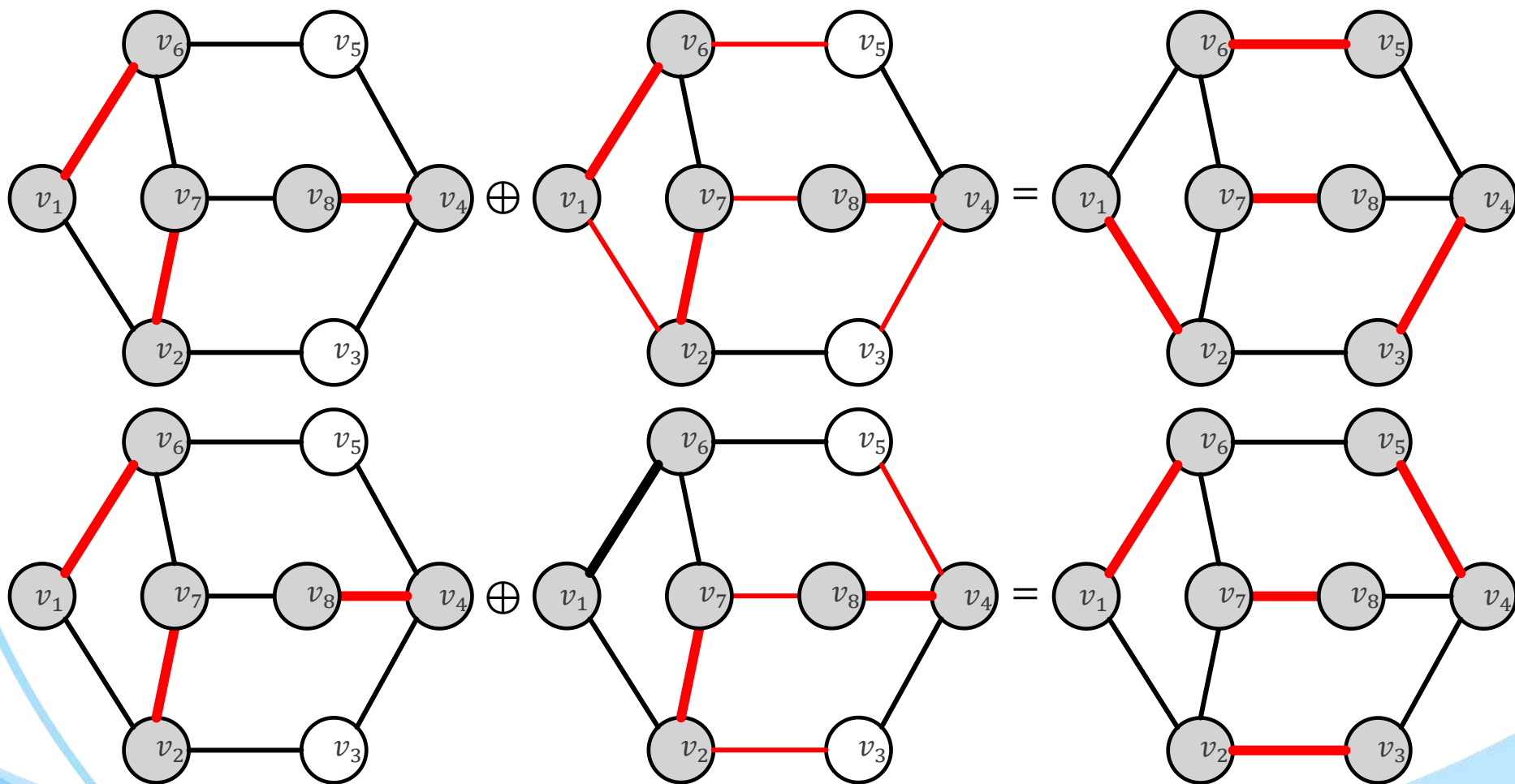
Suppose M is a matching and p is an augmenting path, Then $M \oplus p$ is also a matching, and $|M \oplus p| = |M| + 1$.

Theorem 9.9

A matching is maximum if and only if there is no augmenting path.

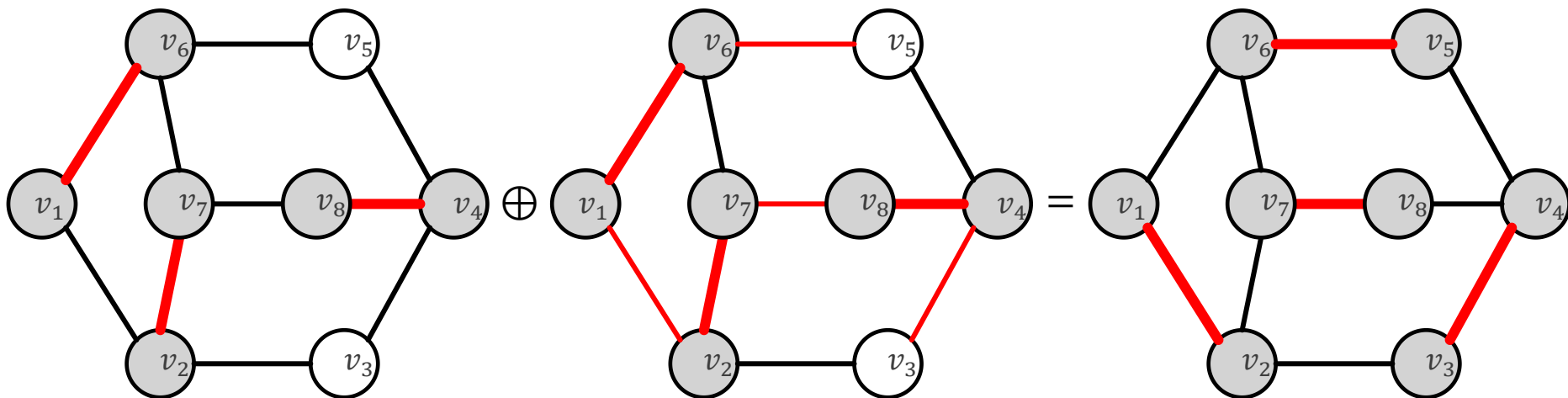
- All these things tell you: once you can find a augmenting path, adding it to the current matching results in a larger matching.

Example





Example



- What $\oplus$ does is **actually flipping** matching and non-matching edges in p . 将 p 上的匹配边变成非匹配边，将原来非匹配边变成匹配边
- There are always one more non-matching edge in p , by its definition.
 - Begins and ends with unmatched vertices



Max Matching Algorithm

MaximumMatching(G)

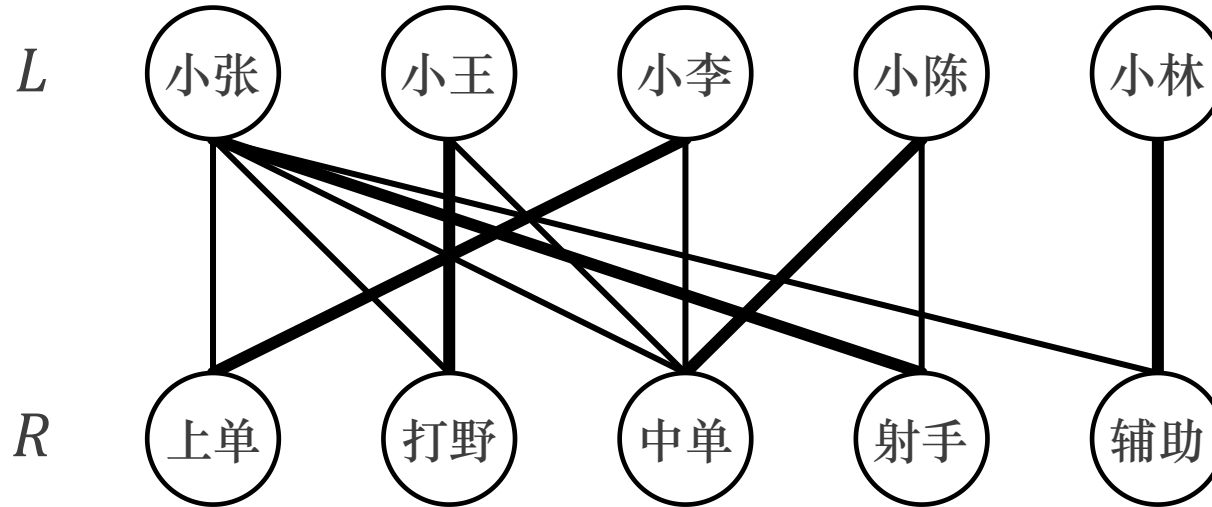
```
1   $M \leftarrow 0$ 
2  repeat
3    find an augmenting path  $p$ 
4     $M \leftarrow M \oplus p$ 
5  until no augmenting path in  $G$ 
6  return  $M$ 
```



Bipartite Graph

Definition

A bipartite graph (二分图, 二部图) $G = (V, E)$ is defined if V can be partitioned into two subsets L and R with $V = L \cup R$ and $L \cap R = \emptyset$, and $\forall e \in E$ has one endpoint in L and the other endpoint in R .





找二分图最大匹配算法

Solution 1: Use the max-flow algorithm. 使用最大流算法

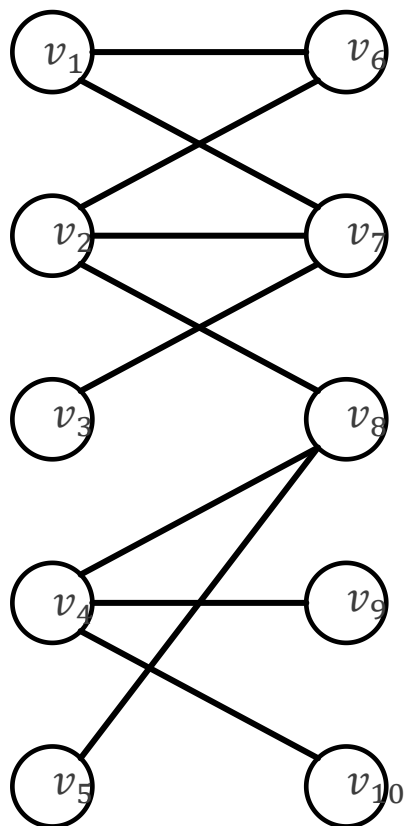
- We define the corresponding **flow network** $G' = (V', E')$ for the bipartite graph G as follows.
 - G' is **directed** but G is undirected.
- We let the source s and sink t be new vertices: $V' = V \cup \{s, t\}$. The directed edges of G' contain **three parts**: edges from s to L , from L to R , and from R to t :

$$\begin{aligned} E' = & \{(s, u): u \in L\} \\ & \cup \{(u, v): u \in L, v \in R, (u, v) \in E\} \\ & \cup \{(v, t): v \in R\}. \end{aligned}$$

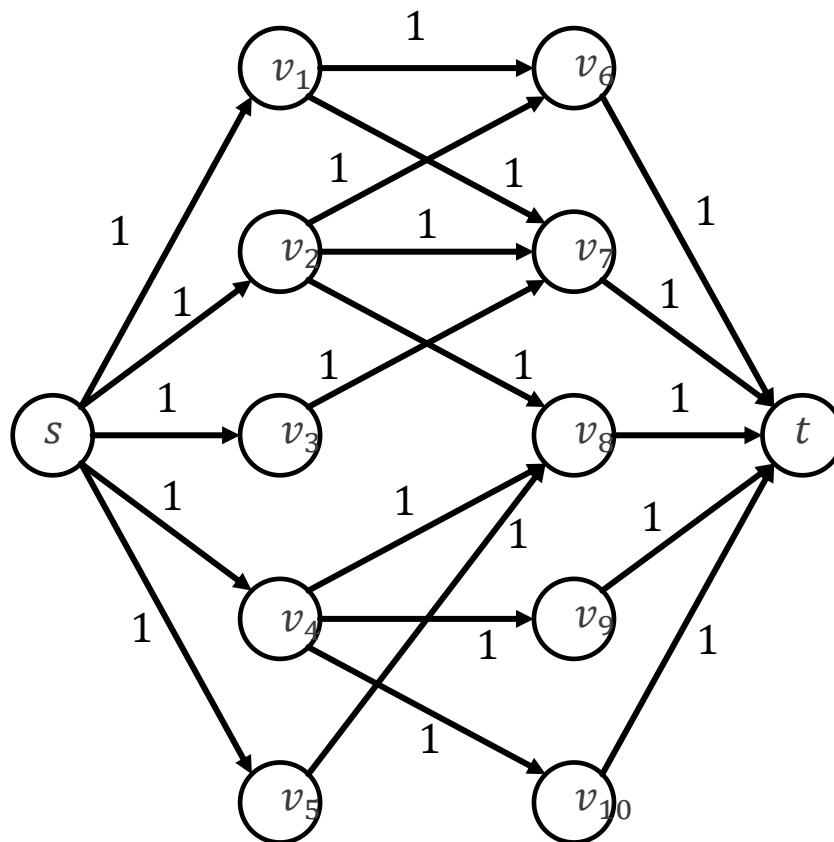
- All edges in E' have capacity 1.



Example



$$G = (V, E)$$



$$G' = (V', E')$$



Maximum Matching Algorithms for Bipartite Graph

Theorem 9.11

Let $G = (V, E)$ be a bipartite graph with vertex partition $V = L \cup R$, and let $G' = (V', E')$ be its corresponding flow network.

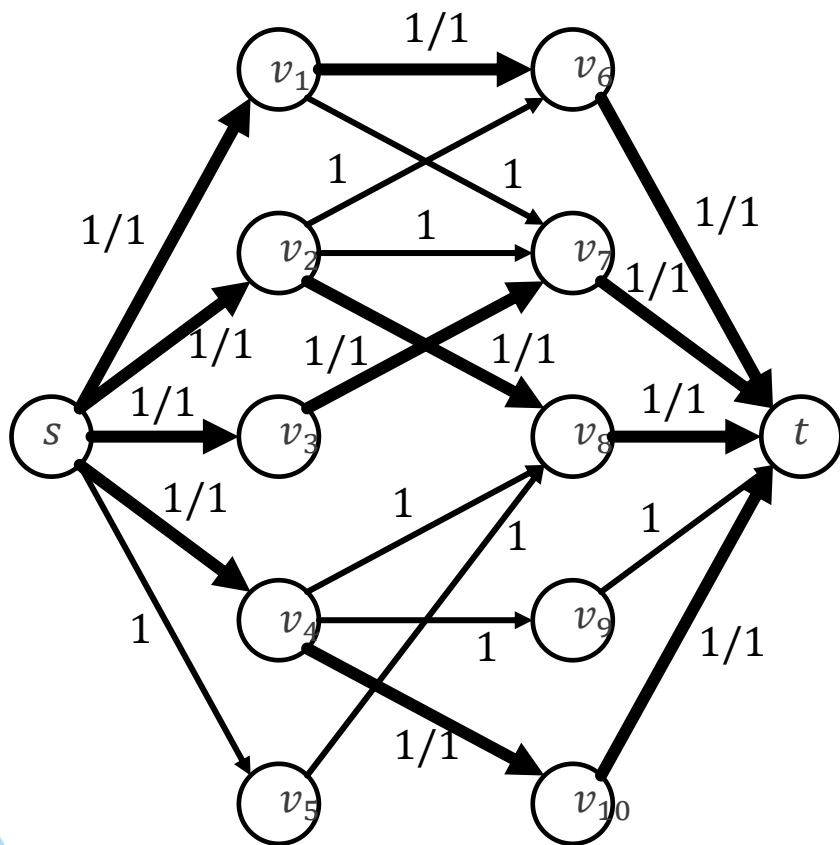
If M is a matching in G , then there is an **integer-valued flow** f in G' with value $|f| = |M|$.

Conversely, if f is an integer-valued flow in G' , then there is a matching M in G with cardinality $|M| = |f|$. 整数流和匹配的大小是对应的,

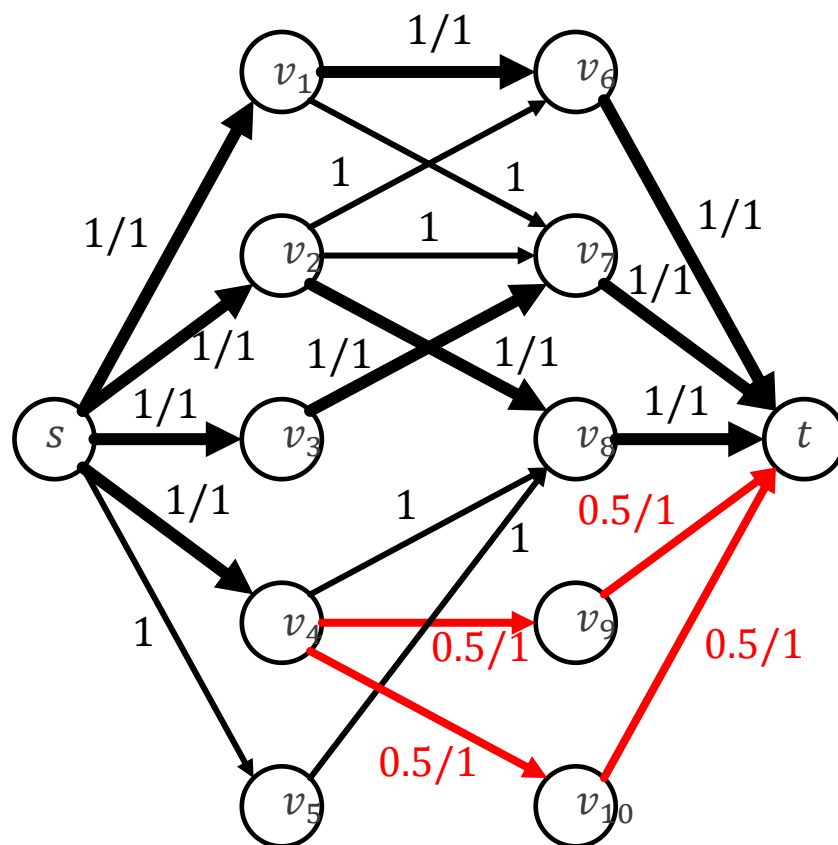
- **This Theorem tells you:** Just run max-flow algorithm and you will obtain maximum matching. 跑最大流算法，之后就得到最大匹配了
- The value of flow needs to be integer because floating point flow does not correspond to a match in G .



Example



Integer-valued flow



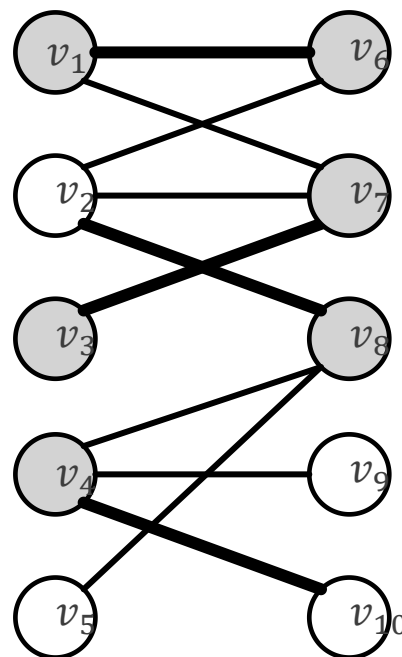
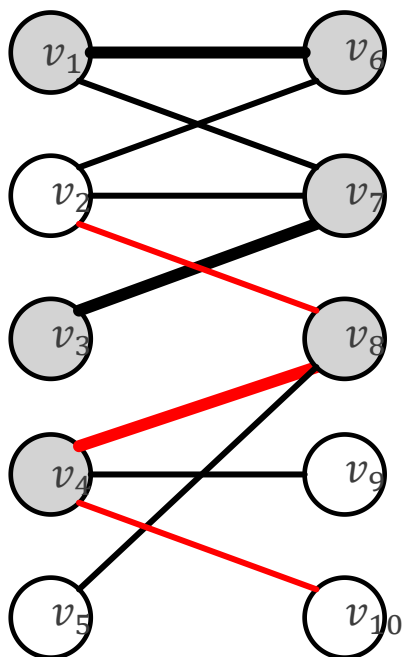
Non integer-valued flow



Maximum Matching Algorithms for Bipartite Graph

Solution 2: Hungarian tree algorithm (匈牙利树算法).

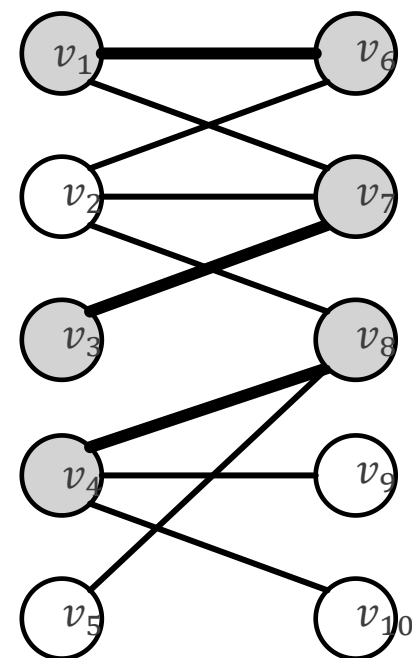
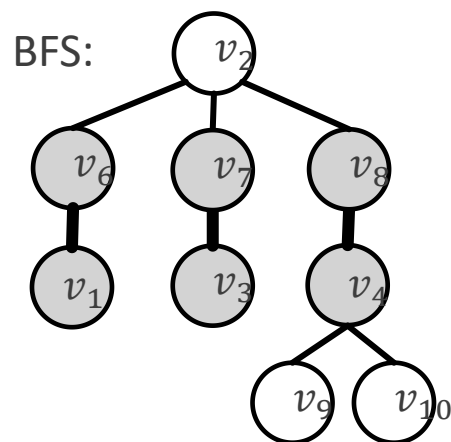
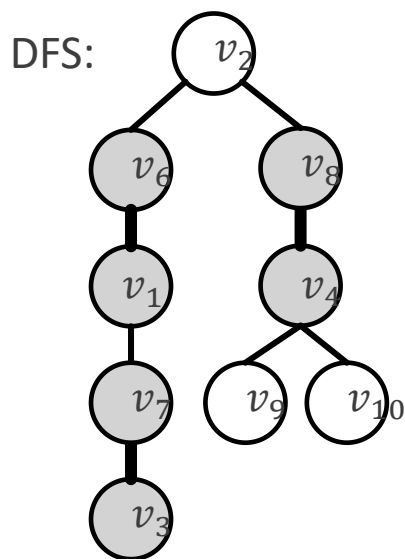
- **Idea:** Find **augmenting path** and **add** the path to **increase matching** by MaximumMatching.





Hungary Tree Algorithm

- But how to find an augmenting path?
- We can start from an **unmatched vertex** and use BFS or DFS to search all **alternating paths**. The generated tree is called **alternating path tree (交错路径树)**.
- If all leaves in the alternating path tree are matched, this tree is called **Hungarian tree (匈牙利树)**.





Hungary Tree Algorithm

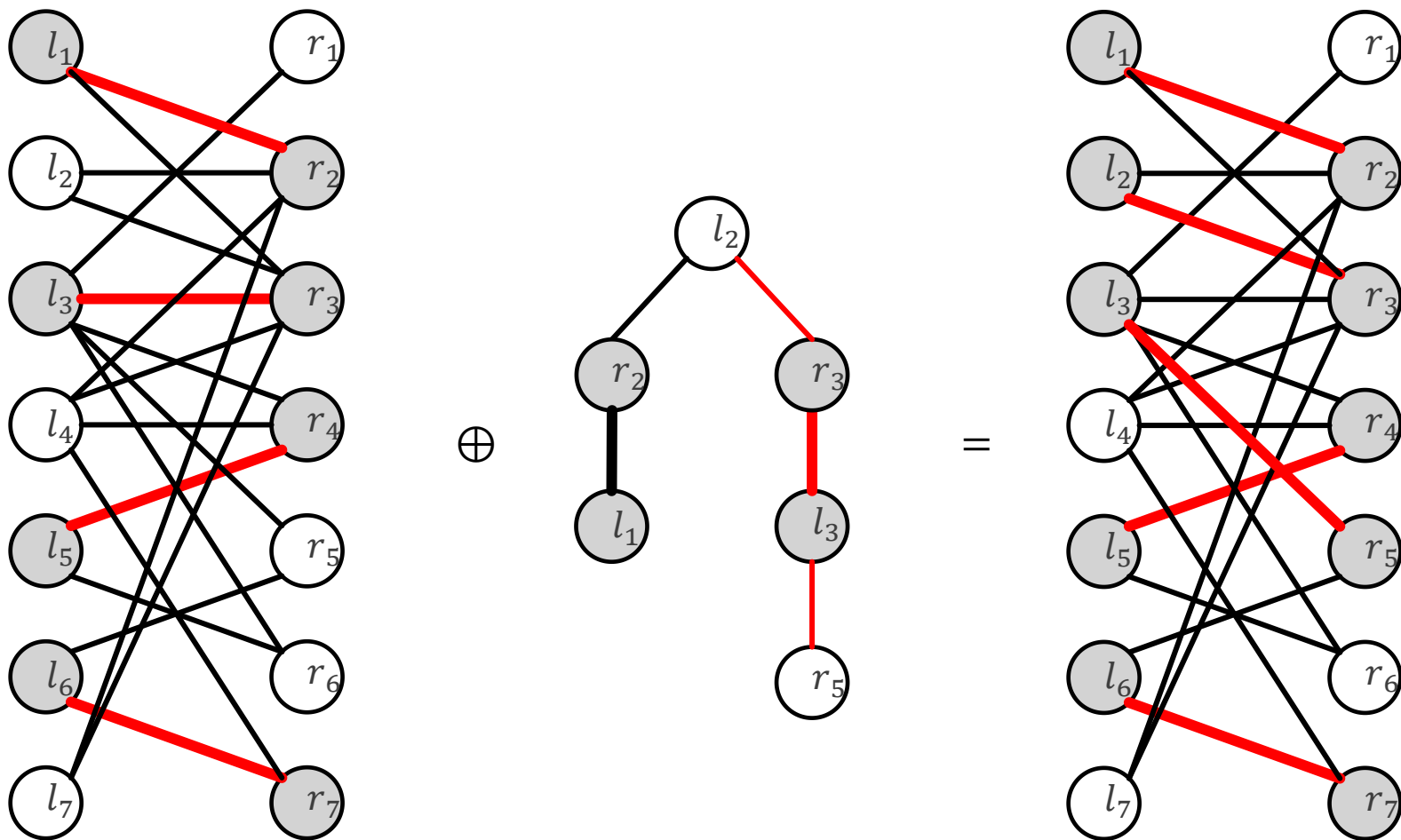
最大二分匹配的匈牙利算法

HungaryBipartiteGraph(G)

- 1 begin with any matching M
- 2 **while** there exists an unmatched vertex in L and R respectively **do**
- 3 let r is an unmatched vertex in L
- 4 grow an alternating path tree T with root r using breadth first search
- 5 **if** T is a Hungarian tree **then**
- 6 $G \leftarrow G - T$
- 7 **else**
- 8 find an augmenting path p in T
- 9 $M \leftarrow M \oplus p$
- 10 **return** M

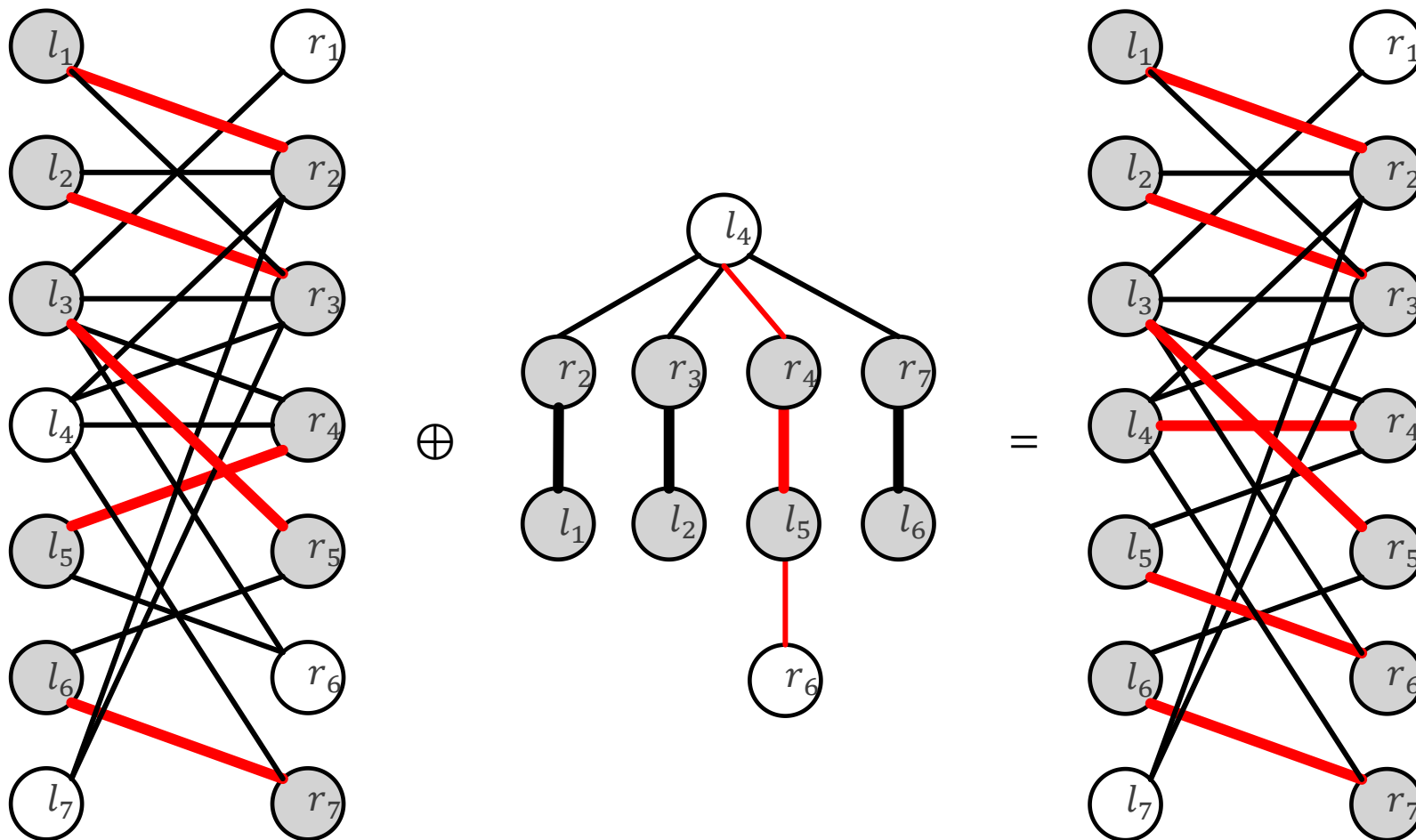


Example



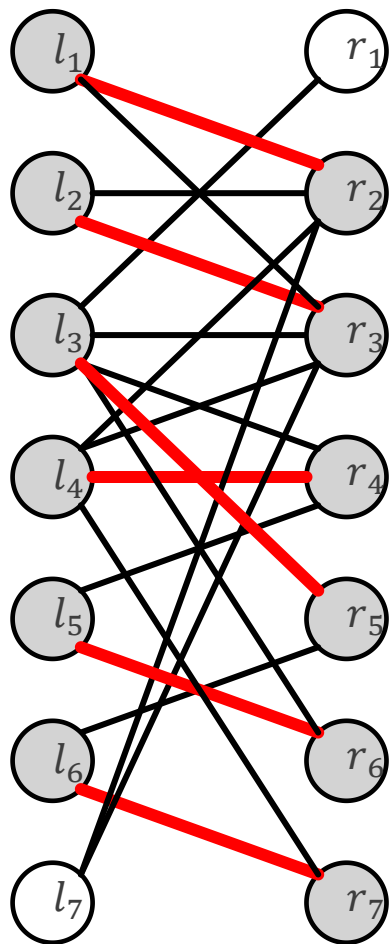


Example

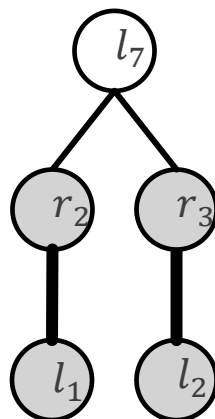




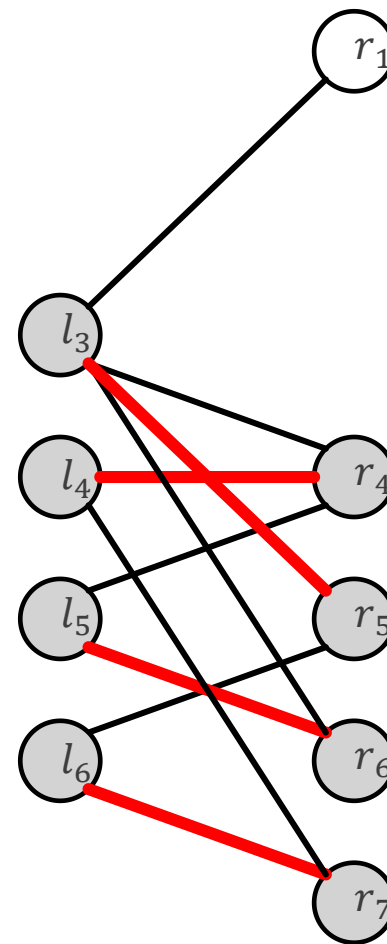
Example



—



=

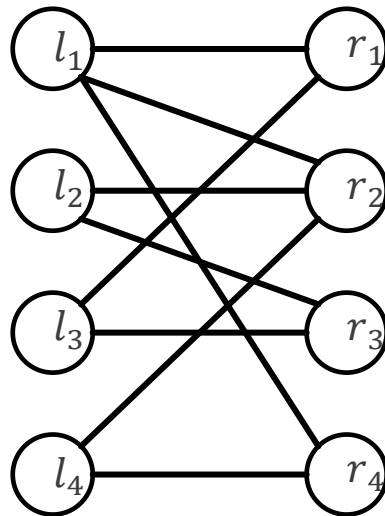


Get Hungarian tree: do
 $G \leftarrow G - T$, and then
the algorithm exits the
while loop in Line 2.



Classroom Exercise

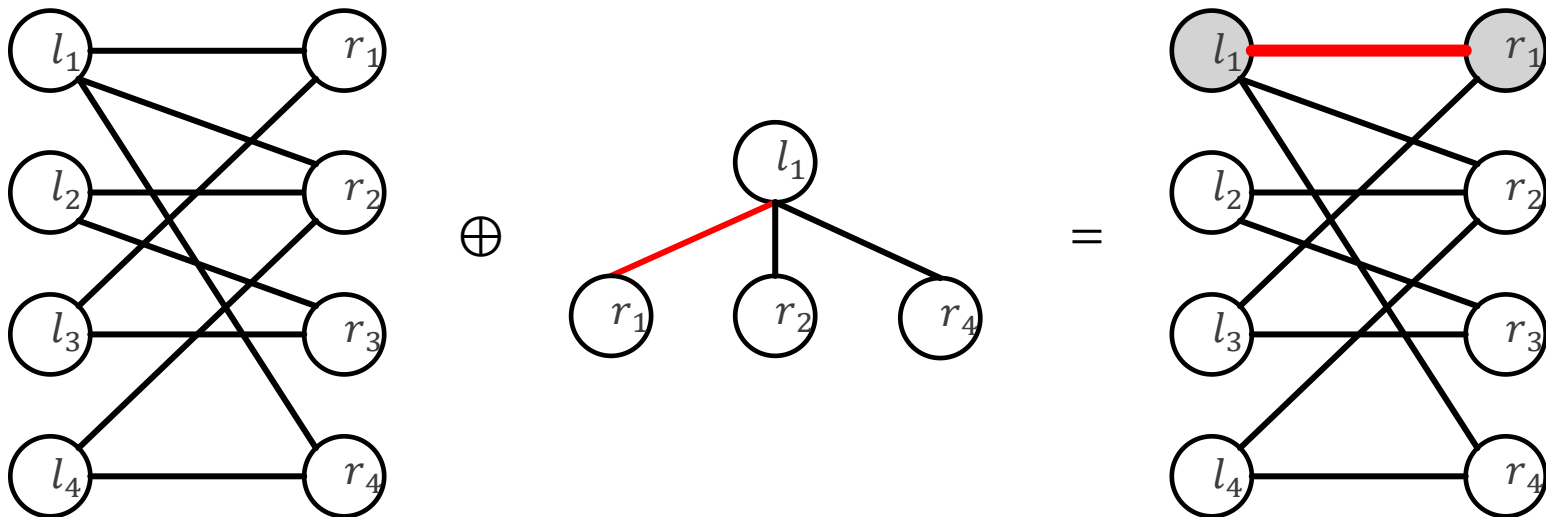
Use **Hungarian tree algorithm** to find the maximum matching of the following bipartite graph.





Classroom Exercise

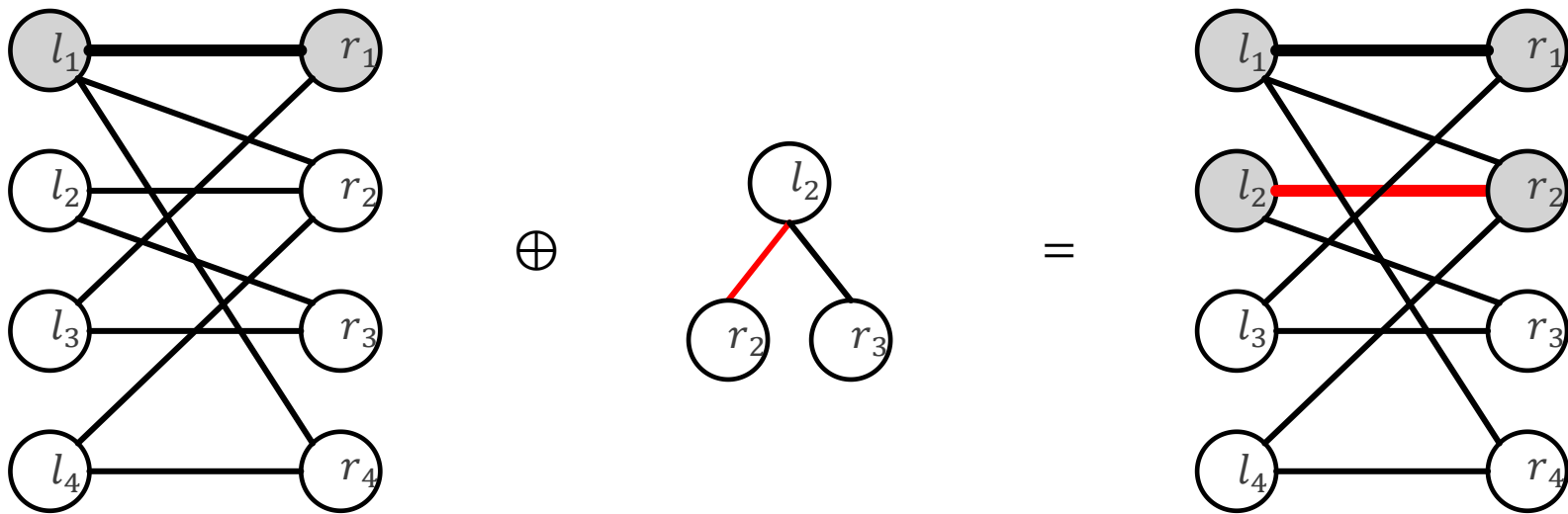
Pick l_1 to generate **alternating path tree**, and
Select an augmenting path and add into G





Classroom Exercise

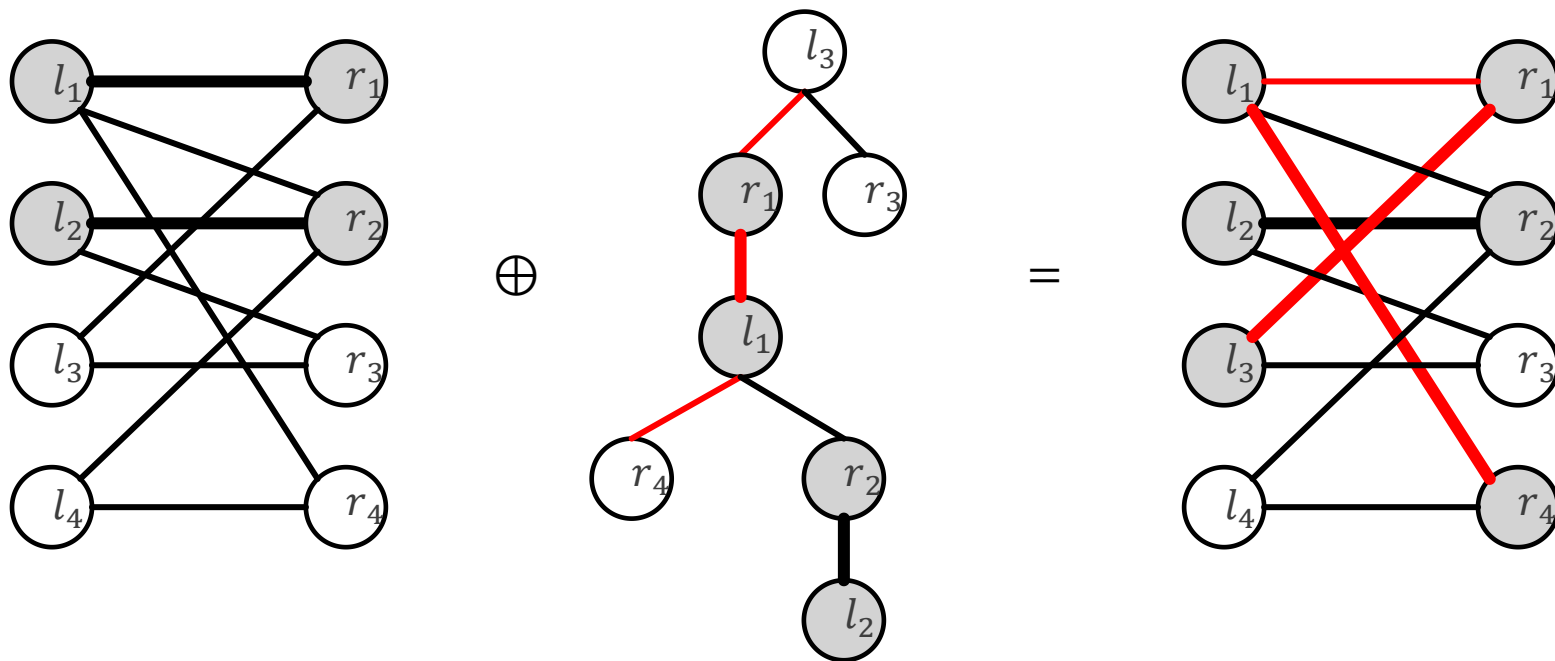
Pick l_2 to generate alternating path tree, and
Select an augmenting path and add into G





Classroom Exercise

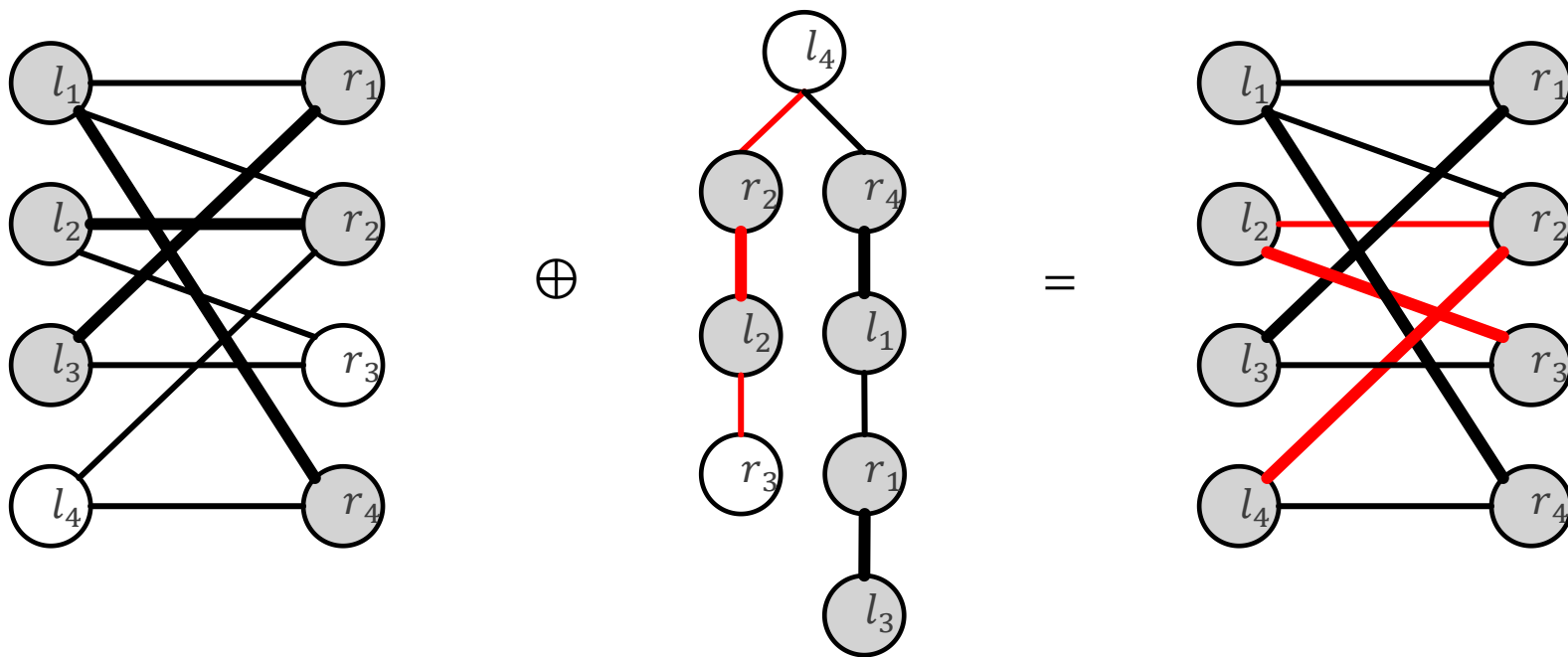
Pick l_3 to generate alternating path tree, and
Select an augmenting path and add into G





Classroom Exercise

Pick l_4 to generate alternating path tree, and
Select an augmenting path and add into G





Conclusion

After this lecture, you should know:

- What is network, capacity and flow in terms of **graph**.
- What **properties** does flow have.
- How to obtain the **residual network**.
- How to find an **augmenting path** and its **bottleneck capacity**.
- How to use the **augmenting path** to increase value of flow.
- How to use **shortest path** to improve max-flow problem.
- How to solve **matching problem**.



Homework 课后习题

Page 172-174

9.1

9.3

9.4

9.6

9.10

9.12

9.19



谢谢

有问题欢迎随时跟我讨论